Algol 68 in Modern Terms

An Introduction for Programmers

Version 0.5.0

3 November 2026

This book is dedicated to the public domain under the
Creative Commons CC0 1.0 Universal Public Domain Dedication:
https://creativecommons.org/publicdomain/zero/1.0/

Contents

Preface vii

1. Introduction 1
1.1. Algol 68 . . . 1
1.2. A first program . . . 1
1.3. Compiling and running . . . 2
1.4. Reserved words, type names and operators . . . 2
1.5. A second program . . . 2

2. Values, types and variables 5
2.1. Types . . . 5
2.2. Literals . . . 6
2.3. Names . . . 6
2.4. Constants . . . 7
2.5. Variables . . . 7
2.6. What a variable is: references . . . 8
2.7. Arithmetic . . . 9
2.8. Precedence . . . 10
2.9. Comparisons and logical operators . . . 11
2.10. Implicit conversions . . . 12
2.11. Assignment . . . 13
2.12. Explicit conversions and other operators . . . 14
2.13. Comments and layout . . . 15
2.14. Simple input and output . . . 15
2.15. A complete program . . . 16
Exercises . . . 17
Programming problems . . . 17

3. Expressions and control flow 19
3.1. Everything is an expression . . . 19
3.2. Sequences and discarded values . . . 20
3.3. Scope . . . 20
3.4. Unordered evaluation . . . 21
3.5. **if** expressions . . . 21
3.6. **case** expressions . . . 22
3.7. Loops . . . 23
3.8. **skip** . . . 26
3.9. Jumps . . . 26
3.10. The structure of a program . . . 28

3.11. A complete program 28
Exercises 28
Programming problems 29

4. Arrays and records **31**
4.1. Arrays 31
4.2. Array literals 32
4.3. Arrays are values 33
4.4. Multi-dimensional arrays 33
4.5. Slices 34
4.6. Strings are arrays 35
4.7. Records 36
4.8. Type declarations 36
4.9. Arrays of records 37
4.10. A single value as an array 38
4.11. Printing arrays and records 38
4.12. Two complete programs 38
Exercises 39
Programming problems 40

5. Functions and operators **43**
5.1. Declaring functions 43
5.2. Parameters 44
5.3. Functions without parameters 45
5.4. Recursion 46
5.5. Functions as values 47
5.6. The standard functions 48
5.7. Defining operators 48
5.8. How an operator is chosen 50
5.9. A complete program 50
Exercises 51
Programming problems 52

6. More built-in types and the standard library **55**
6.1. Complex numbers 55
6.2. Precision variants 56
6.3. Bit sets 58
6.4. Byte strings 59
6.5. Environment enquiries 59
6.6. Mathematical functions 60
6.7. Extending the standard operators 60
Exercises 61
Programming problems 61

7. Advanced types **63**
7.1. Resizable arrays 63

7.2. Where **flex** can stand . . . 64
7.3. Growing an array efficiently . . . 65
7.4. Strings . . . 66
7.5. References into resizable arrays . . . 67
7.6. Sharing arrays . . . 68
7.7. Unions . . . 68
7.8. Working with unions . . . 70
7.9. References to references . . . 72
7.10. Comparing references . . . 72
7.11. Records with reference fields . . . 73
7.12. Type annotations . . . 73
7.13. Orthogonality . . . 74
7.14. Implicit conversions and contexts . . . 74
7.15. Lifetimes . . . 76
7.16. Lifetimes of functions . . . 77
7.17. Recursive types . . . 78
7.18. Unions and recursive types . . . 80
7.19. When two types are the same . . . 82
7.20. A complete program . . . 82
Exercises . . . 83
Programming problems . . . 84

8. Concurrency **87**
8.1. The problems . . . 87
8.2. Concurrent execution . . . 87
8.3. Semaphores . . . 88
8.4. Mutual exclusion . . . 89
8.5. Producer and consumer . . . 90
8.6. The dining philosophers . . . 91
Exercises . . . 91
Programming problems . . . 92

9. Input and output **93**
9.1. Books, channels and files . . . 93
9.2. Reading and writing files . . . 94
9.3. Reading the standard input . . . 94
9.4. Events . . . 96
9.5. Strings as files . . . 98
9.6. Terminators and other settings . . . 98
9.7. Printing arrays and records . . . 99
9.8. Converting numbers to text . . . 99
9.9. Formatted output . . . 100
9.10. Formatted input . . . 104
9.11. Binary transput . . . 105
9.12. What Algol 68 Genie lacks . . . 106
9.13. A complete program . . . 106

Exercises 107
Programming problems 108

A. The standard library **109**

B. The syntax in brief **113**

C. Implementation notes **115**

D. Glossary **119**

E. Modules **123**

F. Calling C **125**

G. Partial parametrization **129**

Answers to the exercises **133**

Bibliography **141**

Index **143**

Preface

Algol 68 was designed in the late 1960s, and many of its ideas are now everyday features of programming languages: every construct is an expression, types can be combined freely, unions are tagged and checked at run time, operators can be defined and overloaded, and the program can run parts of itself concurrently. What kept many programmers away from the language was less its ideas than the way they were described. The defining document, the Revised Report on the Algorithmic Language Algol 68, has a vocabulary of its own: a type is a *mode*, a variable is a *name*, an expression is a *unit*, an implicit conversion is a *coercion*. Most textbooks of the time followed the Report.

This book describes the language of the Revised Report in the words a programmer uses today. It assumes that the reader can program in some current language; it does not assume any particular one. Where a concept is hard, such as references and their lifetimes, it is explained as carefully as it needs to be, but it is not made harder by an unfamiliar name.

The Report's own terms are not hidden, though. Error messages of the compilers use them, and so do the Report and the older literature. Where a concept is introduced, its name in the Report is given once, set in a sans-serif font like this; the index lists these names, and Appendix D is a glossary of all of them.

The implementations

The programs in this book are written for two implementations:

- *ga68*, the Algol 68 front end of the GNU Compiler Collection [10], which compiles to native code, and
- *a68j*, which compiles the same language to the Java virtual machine.

Both accept the same language, apart from a few features that only one of them provides, and give the same results. Every example program in this book was run when the book was built, and the output shown after a program is the output of that run. Most examples were run with both ga68 and a68j; the few that use a feature one of them lacks (semaphores, formats, **bytes**) were run with Algol 68 Genie and, where it has the feature, with a68j, and the text says so. The examples of Appendix F, which calls C, were run with ga68 only, and those of Appendix G, which describes an extension that neither implements, with Algol 68 Genie only. Where the two implementations, or Algol 68 Genie [9], a widely used interpreter, differ from the Report or from each other, a note says so; Appendix C collects these notes.

The versions used for this book are ga68 from the development version of GCC 17.0.0 of 31 October 2026 (commit 3f35c3d2d116), a68j 0.5.0 on OpenJDK 25, and Algol 68 Genie 3.1.2.

How the book is organized

Chapters 1 to 5 cover the core of the language: values and variables, expressions and control flow, arrays and records, functions and operators. They are enough to write substantial programs. Chapters 6 to 9 cover the rest: more built-in types, the advanced features of the type system, concurrency, and input and output. Each chapter ends with exercises, whose answers are at the end of the book, and with programming problems, which have no single answer.

The model for this book was Andrew McGettrick's textbook *ALGOL 68: A First and Second Course* [4], written for students in the years after the Revised Report appeared. The choice of topics and their order follow it closely, and many of the exercises and programming problems are modeled on it. The Revised Report [1] is the authority on every question of the language; the bibliography at the end of the book lists it together with the other works a reader may want to consult.

1. Introduction

1.1. Algol 68

Algol 68 was designed by Working Group 2.1 of the International Federation for Information Processing (IFIP), the group that had earlier produced Algol 60. The language was defined in the Report on the Algorithmic Language Algol 68 of 1968, published in 1969 [2]; a revised definition, the Revised Report [1], was published in 1975. This book describes the language of the Revised Report, which is what all current implementations follow. Lindsey's history of the language [8] tells how it came about.

Algol 68 is a compiled, statically typed, imperative language. Its designers aimed for a small number of general concepts that combine without special cases, a property they called *orthogonality*. The result is a language in which, for example, a block, an **if** and a **case** are all expressions with a value, any type can be the element type of an array or the result type of a function, and a function is a value like any other.

1.2. A first program

Here is a complete program:

```
access Genie_Transput
begin
   print (("Hello, world!", newline))
end
```

```
Hello, world!
```

The line after the program is its output. The program consists of a *block*, the text from **begin** to **end**. The block contains one call of the procedure `print`, which writes its argument to the standard output. The argument is a list of two items in parentheses: the string `"Hello, world!"` and `newline`, which ends the line. That is why there are two pairs of parentheses: the outer pair belongs to the call, the inner pair encloses the list. A call with a single item needs only one pair, as in `print (newline)`.

The first line, **access** `Genie_Transput`, makes `print` and the other input and output procedures available. The Revised Report includes `print` in the standard library, and Algol 68 Genie provides it that way. The standard library of ga68 provides only low-level input and output, and a68j follows ga68. The module `Genie_Transput` fills the gap: it provides the procedures of the Report that this book uses. It comes with a68j, and its source can be compiled with ga68 as well. Every program in this book that reads or writes begins with this line; with Algol 68 Genie, the line is left out. Modules, a GNU extension of the language, are described in Appendix E.

1.3. Compiling and running

With a68j, a program is compiled and run in one step:

```
$ java -jar a68j-backend.jar -run hello.a68
Hello, world!
```

With ga68, the module `Genie_Transput` is compiled once, and the program is compiled and linked with it:

```
$ ga68 -c genietransput.a68
$ ga68 -I. hello.a68 genietransput.o -o hello
$ ./hello
Hello, world!
```

The source of the module, `genietransput.a68`, comes with a68j.

1.4. Reserved words, type names and operators

Some words in an Algol 68 program are *reserved words*: **`begin`**, **`end`**, **`int`**, **`if`** and some sixty others. In this book they are printed in bold, a tradition that goes back to the Report, where these words are written in a typeface of their own. In a source file they are written in lower case.

How the reserved words are told apart from other words in a source file is called *stropping* (stropping). Several conventions have been used. This book uses the default convention of ga68, which is also understood by a68j:

- reserved words are written in lower case: **`begin`**, **`int`**;
- a type name that the program declares begins with a capital letter: `Node`, `Point`;
- operators that are words are written in capitals: `ABS`, `MOD`, or a user-defined `MAX`;
- everything else, such as the names of constants, variables and procedures, is written in lower case, with digits and underscores: `max_value`, `x2`.

Algol 68 Genie uses a different convention by default, in which reserved words, type names and word operators are written in capitals (`BEGIN INT n = 1; ... END`) and names may contain spaces.

1.5. A second program

The next program computes the area and the circumference of a circle:

```
access Genie_Transput
begin
   { The area and the circumference of a circle. }
   real r = 2.5;
   real area = pi * r ** 2;
   real circumference = 2 * pi * r;
   print (("radius        ", fixed (r, 0, 2), newline));
```

```
    print (("area          ", fixed (area, 0, 2), newline));
    print (("circumference ", fixed (circumference, 0, 2), newline))
end
```

```
radius        2.50
area          19.63
circumference 15.71
```

The text between braces is a *comment*. The program declares three names for values: `r` for the radius, and `area` and `circumference` for the results; `pi` is a constant of the standard library. The operator `**` raises to a power. The procedure `fixed` converts a number into a string with a given number of digits after the decimal point; the 0 asks for a string just as wide as needed. All of this is explained in the next chapter.

2. Values, types and variables

This chapter introduces the building blocks of every program: values and their types, constants and variables, the arithmetic and logical operators, assignment, and simple input and output. One idea in it is more important than the others and is the key to much of the language: a variable in Algol 68 is a name for a *reference* to a memory cell, and the language says so openly. Section 2.6 explains what this means.

2.1. Types

Every value in a program has a *type* (mode). The type determines which values are possible and which operations apply. Algol 68 has four *primitive types*:

Type	Values	Examples
int	whole numbers	0, 42
real	floating-point numbers	3.5, 1e6
bool	truth values	**true**, **false**
char	single characters	"a", "7"

The type **string** for sequences of characters is not primitive: it is a resizable array of **char** (Chapter 7). It can be used in the same simple ways as the primitive types, though, and this chapter does so.

The range of **int** and the precision of **real** depend on the implementation. The standard library provides constants that describe them, among them max_int, the largest **int**, and small_real, the smallest **real** that gives a result different from 1 both when it is added to 1 and when it is subtracted from 1:

```
access Genie_Transput
begin
   print (("max_int    ", max_int, newline));
   print (("small_real ", small_real, newline));
   print (("long max_int ", long_max_int, newline))
end
```

```
max_int    +2147483647
small_real +1.192093e -7
long max_int +9223372036854775807
```

> **Implementation note.** In ga68 and a68j, **int** has 32 bits and **real** is a 32-bit floating-point number (single precision, about 7 decimal digits). The types **long int** and **long real** have 64 bits. In Algol 68 Genie, **int** has 64 bits and **real** is a 64-bit floating-point number. Precision variants are the subject of Chapter 6.

2.2. Literals

A value written directly in the program text is a *literal* (denotation):

```
access Genie_Transput
begin
   print ((0, 42, 1 000 000, newline));
   print ((3.5, " ", .25, " ", 1e6, " ", 2.5e-3, newline));
   print (("a", "Z", "_", """", newline));
   print ((true, false, newline))
end
```

```
         +0        +42   +1000000
+3.500000e +0 +2.500000e -1 +1.000000e +6 +2.500000e -3
aZ_"
TF
```

An **int** literal is a sequence of digits. It has no sign: in `-5`, the minus is an operator applied to the literal `5`. Blanks may appear between the digits, so `1 000 000` is a single literal; commas may not.

A **real** literal has a decimal point, an exponent, or both: `3.5`, `.25`, `1e6`, `2.5e-3`. There must be at least one digit after the point, so `5.` is not a literal; `5.0` is. The letter of the exponent may be written `e` or `E`.

A **char** literal is a single character between double quotes. A double quote itself is written twice: `""""` is the literal for the character `"`. The same rule applies to strings: `"say ""hi"""` is the string `say "hi"`.

The two **bool** literals are **true** and **false**. When `print` writes a truth value, it writes `T` or `F`.

2.3. Names

The names that a program gives to its constants, variables and procedures are *identifiers* (identifier, tag). An identifier begins with a lower-case letter and continues with lower-case letters, digits and underscores: `x`, `x2`, `max_value`. Reserved words cannot be used as identifiers.

Capital letters have a meaning of their own (Section 1.4): a word that begins with a capital letter is a type name or an operator. A name in the style of other languages, such as `maxValue`, is therefore read as the identifier `max` followed by a type name `Value`, and the compiler complains about the type:

```
access Genie_Transput
begin
   int maxValue = 100;
   print ((maxValue, newline))
end
```

```
identifier-case.a68:3:11: error: indicant Value has not been declared properly
```

2.4. Constants

A declaration of the form

```
int days_per_week = 7
```

binds the identifier `days_per_week` to the value 7, for the rest of the block in which the declaration appears. Such a binding cannot be changed; `days_per_week` is a *constant*. The Report calls this form an identity declaration, since it declares that the identifier and the value are identical: wherever `days_per_week` is used, it stands for 7. It corresponds to `const` in C and JavaScript, to `final` in Java and to `let` in Rust.

The value need not be known when the program is compiled. Any expression of the right type can stand on the right-hand side, including one that uses other constants or reads input. Several constants can be declared at once, separated by commas; declarations are separated by semicolons:

```
int days_per_week = 7;
int weeks = 52, extra_days = 1;
int days_per_year = days_per_week * weeks + extra_days;
real g = 9.81;
char separator = ",";
bool verbose = false;
print ((days_per_year, separator, fixed (g, 0, 2), separator, verbose, newline))
```

```
       +365,9.81,F
```

An attempt to change a constant is an error, found by the compiler:

```
access Genie_Transput
begin
   int n = 10;
   n := 11;
   print ((n, newline))
end
```

```
constant-assign.a68:4:4: error: int secondary does not yield a name
```

The message is phrased in the terms of the Report: a name is what this book calls a reference, and the message says that `n` is not a reference, so nothing can be assigned to it. Section 2.6 explains this.

2.5. Variables

A *variable* is declared with `:=` instead of `=`, or with no initial value at all:

```
access Genie_Transput
begin
   int count := 0;
   real total := 0.0;
   count := count + 1;
   total := total + 2.5;
```

```
    count := count + 1;
    total := total + 4.0;
    print (("count = ", count, ", total = ", total, newline))
END
```

```
count =          +2, total = +6.500000e +0
```

The declaration `int count := 0` creates a variable that holds an `int`, with the initial value 0. The *assignment* `count := count + 1` replaces the value held by `count` with a new value. Several declarations can be joined with commas, but the type may be left out only if the next declaration is of the same kind: `int x = 5, y := 6` is an error ("mixed identity-declaration and variable-declaration"), whereas `int x = 5, int y := 6` is allowed. The declarations joined by commas are carried out in an unspecified order, so one of them must not use another (Section 3.4).

A variable declared without an initial value, as in `real x`, holds no defined value until a value is assigned to it. Using it before then is an error that the compiler does not detect. A `string` variable, and a `string` inside an array or record variable, is the exception: it starts out as the empty string, since a string is a resizable array whose initial bounds are 1 and 0 (Section 7.1), and an empty array has no elements that could be undefined.

> **Implementation note.** ga68 and a68j give such a variable an initial value: zero, `false` or a blank character. A program should not rely on this, since other implementations do not.

2.6. What a variable is: references

In most languages a variable is a primitive concept: a name that refers to a place in memory whose content can change. Algol 68 builds variables from two simpler concepts, and makes both visible.

The first concept is *allocation*. The expression `loc int` allocates a fresh memory cell that can hold an `int`, on the stack of the current block, and yields a *reference* to that cell. The type of a reference to an `int` cell is `ref int`, read "reference to int". A reference is a value like any other: it can be bound to an identifier, passed to a procedure or stored in another cell. The Report calls a reference a name and an allocation a generator.

The second concept is the constant declaration of the previous section. Together they give a variable:

```
int a := 5;                    { the short form }
ref int b = loc int := 5;      { what the short form means }
a := a + 1;
b := b + 1;
print ((a, b, newline))
```

```
          +6         +6
```

The declaration `ref int b = loc int := 5` allocates a cell, assigns 5 to it, and binds `b` to the reference. The short form `int a := 5` means exactly the same, and a program normally

uses the short form. But the long form shows what the identifier of a variable stands for: not the number in the cell, but the reference to the cell. The type of `a` is **`ref int`**, not **`int`**.

Since a reference is a value, it can be bound to a second name. The second name then refers to the same cell, and an assignment through it changes the variable:

```
int a := 5;
ref int alias = a;          { another name for the cell of a }
alias := 99;
print ((a, newline))
```

```
        +99
```

prints 99. Nothing has been copied: the declaration binds the name `alias` to the reference that `a` stands for.

This explains the error message of the previous section. The left-hand side of an assignment must be a reference, of type **`ref int`** when an **`int`** is assigned. A constant declared as **`int`** `n = 10` has the type **`int`**; it is the number 10 itself, and there is nothing to assign to.

Programmers who know C++ will recognize the idea: `b` behaves like a C++ reference `int& b` to a cell. In Rust, `b` would be a `&mut i32`, which needs an explicit `*` to read the cell; Algol 68 reads the cell implicitly where a value is needed (Section 2.10). Programmers who know Java or Python can think of the reference as the hidden storage location behind every variable; Algol 68 simply gives it a type. The distinction between a value and a reference to a cell holding that value is the source of most of the power of the language's type system, and later chapters return to it often: references to references, references in records, references returned by functions, and how long the cell behind a reference lives.

2.7. Arithmetic

The arithmetic operators on **`int`** and **`real`** are:

Operator	Meaning	Operand types
`+ - *`	addition, subtraction, multiplication	**`int`**, **`real`**
`/`	division	**`int`**, **`real`**; the result is **`real`**
`%`	integer division, truncating toward zero	**`int`**
`MOD`	remainder, never negative	**`int`**
`**`	power	**`int`** or **`real`** base, **`int`** exponent
`-` (unary)	negation	**`int`**, **`real`**

Note that `/` always yields a **`real`**, even for two **`int`** operands, so `n := 4 / 2` is rejected for an **`int`** variable `n`; integer division is `%`. The remainder `a MOD b` lies between 0 and `ABS b - 1` even if `a` is negative, unlike the `%` operator of C, Java or JavaScript:

```
access Genie_Transput
begin
   int a = 17, b = 5;
   print ((a + b, a - b, a * b, newline));
   print ((a / b, newline));
```

```
    print ((a % b, a MOD b, newline));
    print ((-a % b, -a MOD b, newline));
    print ((a ** 2, " ", 2.5 ** 2, newline))
end
```

```
        +22        +12        +85
+3.400000e +0
         +3         +2
         -3         +3
       +289 +6.250000e +0
```

The power operator requires an **int** exponent, which may be negative if the base is a **real**: `2.0 ** -1` is 0.5. A **real** power, such as a square root, is computed with the functions of the standard library (`sqrt`, `exp`, `ln`), which Chapter 6 describes.

> **Implementation note.** An **int** result that does not fit into an **int** is undefined by the Report. In ga68 and a68j, it wraps around silently: `max_int + 1` gives −2147483648. The result of an **int** raised to a negative power, such as `2 ** -1`, is undefined as well.

2.8. Precedence

Every binary operator has a *precedence* (priority) from 1 to 9. An operator of higher precedence binds more tightly: `2 + 3 * 4` is `2 + (3 * 4)`. Operators of equal precedence are applied from left to right: `10 - 4 - 3` is `(10 - 4) - 3`. The standard operators have the following precedences:

Precedence	Operators
9	`I` (complex numbers, Chapter 6)
8	`**`, `UP`, `LWB`, `UPB`
7	`*`, `/`, `%`, `MOD`, `ELEM`
6	`+`, `-`
5	`<`, `<=`, `>=`, `>`
4	`=`, `/=`
3	`AND`
2	`OR`
1	`+:=`, `-:=` , `*:=`, `/:=`, `%:=`, …

Two rules differ from what most current languages do. First, `**` is applied from left to right like every other operator, so `2 ** 3 ** 2` is $(2^3)^2 = 64$, not $2^9 = 512$. Second, every unary operator binds more tightly than every binary operator. In `-2 ** 2`, the minus applies to the 2 first, so the result is $(-2)^2 = 4$:

```
access Genie_Transput
begin
    print ((2 + 3 * 4, (2 + 3) * 4, newline));
    print ((10 - 4 - 3, 2 ** 3 ** 2, newline));
    print ((-2 ** 2, -(2 ** 2), newline))
end
```

```
        +14        +20
         +3        +64
         +4         -4
```

When in doubt, parentheses make the intended order explicit, and the programs in this book use them wherever the order might surprise a reader. Older programs, and many on Rosetta Code, write some operators as words: `OVER` for `%`, `PLUSAB` for `+:=`, and so on (Appendix A). Chapter 5 shows how a program declares operators of its own and their precedences.

2.9. Comparisons and logical operators

The comparison operators `=`, `/=` (not equal), `<`, `<=`, `>`, `>=` compare two numbers, or two characters in the order of their character codes, and yield a **`bool`**. The logical operators are `AND`, `OR` and the unary `NOT`:

```
access Genie_Transput
begin
   int x = 7;
   print ((x > 5, " ", x = 7, " ", x /= 7, newline));
   print ((x > 0 AND x < 10, " ", x < 0 OR x > 100, " ", NOT (x = 7), newline));
   print (("a" < "b", " ", "Z" < "a", newline))
end
```

```
T T F
T F F
T T
```

Since `AND` has a lower precedence than the comparisons, `x > 0 AND x < 10` needs no parentheses. Applied to two truth values, `=` tests whether they are equal and `/=` is the exclusive or; `(a < 0) = (b < 0)` tests whether `a` and `b` have the same sign.

> **Pitfall.** `NOT` is a unary operator and binds more tightly than every binary operator, so `NOT x = 7` means `(NOT x) = 7`, which the compiler rejects for an **`int`** `x`:
>
> ```
> not-prec.a68:4:12: error: monadic operator NOT int has not been declared
> ```
>
> The test is written `NOT (x = 7)` or `x /= 7`. Programmers who know Python, where `not` binds loosely, fall into this easily. Similarly, `x < y < z` is not a chained comparison: `x < y` is a **`bool`**, which cannot be compared with a number.

`AND` and `OR` are ordinary operators: both operands are evaluated before the operator is applied, as with every operator. They do not "short-circuit" like `&&` and `||` in C or Java. The GNU dialect of Algol 68, which ga68 and a68j implement, adds the short-circuit forms **`andth`** and **`orel`**, which evaluate their right operand only when it is needed. They bind more weakly than every operator, and a chain of them needs parentheses: `a andth (b andth c)` (Appendix B). The following program shows the difference; it uses a small procedure `check` that prints a line before it returns its argument (procedures are the subject of Chapter 5):

```
access Genie_Transput
```

```
begin
   proc check = (bool b) bool: (print (("checked", newline)); b);
   print ((false AND check (true), newline));
   print ((false andth check (true), newline))
end
```

```
checked
F
F
```

2.10. Implicit conversions

In the program of Section 2.6, `a` has the type **`ref int`**, and yet `a + 1` is allowed, although `+` is defined for **`int`** and not for references. The compiler inserts an operation that reads the cell: it converts the reference into the value it refers to. Such an automatic change of type is an *implicit conversion* (coercion), and this one is called *dereferencing* (dereferencing).

Algol 68 has six kinds of implicit conversion; this chapter needs two. The second is *widening* (widening): an **`int`** is converted into a **`real`** where a **`real`** is required. Both conversions can apply one after the other: in `r := n` below, the variable `n` is first dereferenced to an **`int`**, which is then widened to a **`real`**:

```
access Genie_Transput
begin
   int n := 3;
   real r := 2;                 { int to real: widening }
   r := n;                      { again: n is read, then widened }
   print ((n + 1, " ", r / 2, newline));
   print ((n + 0.5, " ", n < 3.5, newline))    { mixed operands }
end
```

```
         +4 +1.500000e +0
+3.500000e +0 T
```

There is no implicit conversion in the other direction, from **`real`** to **`int`**, since it would lose information:

```
access Genie_Transput
begin
   int i = 2.5;
   print ((i, newline))
end
```

```
no-narrowing.a68:3:12: error: real cannot be coerced to int in a strong-unit
```

The message, again in the terms of the Report, says that a **`real`** cannot be converted to **`int`** in a "strong-unit". A unit is an expression; strong describes the place where the expression appears.

This is the last important idea of the chapter: which implicit conversions are allowed depends on where an expression stands. Algol 68 distinguishes five kinds of place, called

conversion contexts (syntactic positions, contexts). The right-hand side of a declaration or an assignment is a *strong* context, where all conversions are allowed, since the required type is known. An operand of an operator is a *firm* context, where widening is *not* allowed, since the operator to apply is chosen by the types of its operands and is not known in advance. The operator `ROUND`, for example, is defined for a **`real`** operand only, and an **`int`** operand is not widened to make it fit:

```
access Genie_Transput
begin
   int n = 3;
   print ((ROUND n, newline))
end
```

```
round-int.a68:4:12: error: monadic operator ROUND int has not been declared
```

How then can `n + 0.5` work, with an **`int`** and a **`real`** operand? Not by widening: the standard library simply defines the arithmetic and comparison operators for each combination of **`int`** and **`real`** operands (Revised Report, Section 10.2.3.5). Chapter 7 describes all six implicit conversions and all five contexts.

2.11. Assignment

An assignment `x := e` evaluates `e` and stores its value in the cell that the reference `x` refers to. An assignment is itself an expression: its value is the reference on the left-hand side. That makes `a := b := 7` possible: it assigns 7 to `b`, and the result, the reference `b`, is dereferenced and its value assigned to `a`. The two sides of an assignment are evaluated in an unspecified order (Section 3.4).

The *compound assignment* `x +:= e` adds `e` to the value of `x` and stores the result in `x`; it is short for `x := x + e`, except that `x` is evaluated only once. Similar operators exist for the other arithmetic operators: `-:=`, `*:=`, `/:=` (for a **`real`** variable), `%:=` and `%*:=` (the `MOD` form). They are operators of precedence 1, so `x +:= 1 + 2` adds 3. Their value, like that of an assignment, is the reference on the left-hand side.

Unlike the assignment, these are ordinary operators, and like all operators of the same precedence they are applied from left to right:

```
int a := 2, b := 1;
a +:= b +:= 3;              { (a +:= b) +:= 3 }
print ((a, b, newline))
```

```
         +6         +1
```

prints 6 and 1: `a +:= b +:= 3` is `(a +:= b) +:= 3`, and `b` is not changed. A chain of assignments `a := b := 3`, in contrast, groups to the right. And `n +:= x` for an **`int`** variable `n` and a **`real`** `x` is rejected, for the same reason as `n := n + x`.

```
access Genie_Transput
begin
   int a, b;
   a := b := 7;                    { assigns 7 to b, then to a }
```

```
    print ((a, b, newline));
    int total := 10;
    total +:= 5;  total *:= 2;  total -:= 1;
    print ((total, newline));
    int k := 17;
    k %:= 5;
    print ((k, newline))
end
```

```
         +7         +7
        +29
         +3
```

2.12. Explicit conversions and other operators

A **real** is converted into an **int** with one of two operators. `ENTIER x` is the largest integer not greater than `x` (the floor). `ROUND x` is the integer nearest to `x`. Characters are converted to and from their integer codes with `ABS` and `REPR`. A few more operators complete the set used in this chapter:

Operator	Result
`ENTIER x`	the largest **int** not greater than the **real** `x`
`ROUND x`	the **int** nearest to the **real** `x`
`ABS x`	the absolute value of an **int** or **real**
`SIGN x`	-1, 0 or 1, according to the sign of `x`
`ODD n`	**true** if the **int** `n` is odd
`ABS c`	the character code of the **char** `c`
`REPR n`	the character with the code `n`

```
access Genie_Transput
begin
    print ((ROUND 2.5, ROUND -2.5, ROUND 2.4, newline));
    print ((ENTIER 2.5, ENTIER -2.5, newline));
    print ((ABS -7, " ", ABS -2.5, SIGN -7, " ", ODD 7, newline));
    print ((ABS "A", " ", REPR 66, newline))
end
```

```
         +3         -3         +2
         +2         -3
         +7 +2.500000e +0         -1 T
        +65 B
```

Implementation note. The Report does not say whether a number exactly halfway between two integers, such as 2.5, is rounded up or down. ga68 and a68j round it away from zero: `ROUND 2.5` is 3 and `ROUND -2.5` is -3.

2.13. Comments and layout

In the convention of this book, a *comment* is text between braces, `{ like this }`. Comments can be nested: `{ a { nested } comment }` is one comment. The layout of a program, its spaces and line breaks, is free: a program can be written on one line or spread over many. In the stropping of this book, spaces are not allowed inside an identifier or a reserved word, but may appear between the digits of a number. The Report allows spaces inside identifiers as well, which is why it and older texts write `max int` where this book writes `max_int`.

> **Implementation note.** The Report encloses a comment in two `#` symbols, or between two of the bold words **comment** or **co**. In the stropping convention of this book, ga68 and a68j accept only braces. Algol 68 Genie accepts `#…#`, `CO…CO` and `COMMENT…COMMENT`, but not braces. These comments cannot be nested: a comment ends at the next symbol of the kind that began it.

2.14. Simple input and output

The procedure `print` writes a list of values to the standard output. Each value is written in a standard layout: an **int** with its sign, in a field wide enough for `max_int` (11 characters in ga68 and a68j), a **real** in scientific notation, a **bool** as `T` or `F`, and a **char** or a string as it is. Apart from the padding of an **int**, no separators are inserted; a program that wants spaces between numbers prints them. For more control, three procedures convert a number into a string:

- `whole (n, w)` converts the **int** `n` into a string of width `w`. A width of 0 gives a string just as wide as needed; a positive width gives a string of exactly that width, with a sign; a negative width gives that width without a plus sign.
- `fixed (x, w, d)` converts the number `x` into a string of width `w` with `d` digits after the decimal point. Again, width 0 means "as wide as needed".
- `float (x, w, d, e)` converts `x` into scientific notation with an exponent of `e` digits (Chapter 9).

```
access Genie_Transput
begin
   int n = 42;
   real x = 3.14159;
   print ((n, " ", x, " ", true, newline));
   print ((whole (n, 0), "|", whole (n, 6), "|", whole (-n, 0), newline));
   print ((fixed (x, 0, 2), "|", fixed (x, 8, 3), "|", fixed (-x, 0, 1), newline))
end
```

```
        +42 +3.141590e +0 T
42|   +42|-42
3.14|  +3.142|-3.1
```

A number that does not fit into the given width is not printed in a wider field: `whole (12345, 3)` yields three asterisks (Section 9.8).

> **Implementation note.** The `print` of the Report writes a blank before every number that does not begin a line. ga68, a68j and Algol 68 Genie leave it out, which is why the outputs in this book show numbers directly after a preceding string.

The procedure `read` reads values from the standard input into variables. Numbers are separated by blanks or line breaks. The procedures `read_int` and `read_real`, an extension of Algol 68 Genie that `Genie_Transput` also provides, read a single value and return it, which is convenient for initializing a constant:

```
access Genie_Transput
begin
   int a, b;
   real c;
   read ((a, b, c));
   print ((a + b, " ", c * 2, newline));
   int d = read_int;
   print ((d, newline))
end
```

With the input

```
3 4
1.5
  99
```

the program writes

```
          +7 +3.000000e +0
         +99
```

A number is read after skipping blanks and line breaks, but a **`char`** is read as it is: a blank is a character. At the end of a line, reading a **`char`** moves on to the first character of the next line; the line break itself is never read as a character. The layout procedure `space` skips a single character, so a date written as `7/3/1975` can be read into three numbers:

```
int day, month, year;
read ((day, space, month, space, year));
print ((whole (year, 0), "-", whole (month, 0), "-", whole (day, 0), newline))
```

```
1975-3-7
```

prints `1975-3-7`.

2.15. A complete program

The last program of this chapter reads a temperature in degrees Fahrenheit and converts it to degrees Celsius. The output shown is for the input `98.6`:

```
access Genie_Transput
begin
   { Read a temperature in degrees Fahrenheit and print it in Celsius. }
   real fahrenheit;
```

```
    read (fahrenheit);
    real celsius = (fahrenheit - 32) * 5 / 9;
    print ((fixed (fahrenheit, 0, 1), " F is ",
            fixed (celsius, 0, 1), " C", newline))
end
```

```
98.6 F is 37.0 C
```

Writing `(fahrenheit - 32) * (5 / 9)` would also have worked, since `/` yields a **`real`** even for two **`int`** operands; with integer division, `5 % 9` would have been 0.

Exercises

The answers are at the end of the book.

1. Which of the following are **`int`** literals, which are **`real`** literals, and which are neither? `12`, `-3`, `1 000`, `1,000`, `2.`, `.5`, `3e2`, `0.0`.
2. What are the values of `17 % 5`, `-17 % 5`, `17 MOD 5`, `-17 MOD 5` and `17 / 5`?
3. What are the values of `2 + 3 * 4 ** 2` and `-3 ** 2`?
4. Explain the difference between **`int`** `n = 10` and **`int`** `n := 10`. What is the type of `n` in each case?
5. Write the declaration **`real`** `x := 1.5, y` without the short form, using **`loc`**.
6. Which of the following are correct, given **`int`** `i := 1` and **`real`** `r := 2.5`? (a) `r := i` (b) `i := r` (c) `i := ROUND r` (d) `r := ROUND i`
7. After **`int`** `a := 1, b := 2`, what are the values of `a` and `b` after `a := b := a + b`?
8. Why does the compiler reject `maxCount := 0`, even if a variable `max_count` has been declared?
9. What does **`int`** `x := 1; ref int r = x; r := r + 1; print (x)` print?
10. Add parentheses to show how `a < b AND x > 0 = y > 0 OR NOT done` is grouped.
11. What are the values of `2.0 ** -2` and `ABS (-7 MOD 3)`? Why is `NOT n = 0` rejected for an **`int`** `n`?

Programming problems

1. Read a number of seconds and print it as hours, minutes and seconds.
2. Read the three coefficients of a quadratic equation whose roots are real, and print the roots (`sqrt` computes a square root).
3. Read a character and print its code, the next character, and whether it is a digit (`"0" <= c AND c <= "9"`).

4. Read an amount of money in cents and print the smallest number of coins of 50, 20, 10, 5, 2 and 1 cents that make it up.

5. **Easter.** Read a year and print the date of Easter Sunday, computed with the anonymous Gregorian algorithm (Meeus/Jones/Butcher), which needs only `%` and `MOD`.

6. **Day of the week.** Read a date written as `dd/mm/yyyy` and print the day of the week as a number, using Zeller's congruence. Where does it matter that `MOD` never yields a negative result?

7. **Clock.** Read an hour from 0 to 23 and print it on a 12-hour clock with `AM` or `PM`, without **`if`**: the hour is `(h + 11) MOD 12 + 1`. (Hint: `REPR` and `ABS` can compute the letter.)

8. **Rounding money.** Read an amount as a **`real`**, round it to cents with `ROUND`, and print euros and cents with `%` and `MOD`. Try a negative amount, and an amount above 100000 with the 32-bit **`real`** of ga68 or a68j. What goes wrong, and why should money not be kept in a **`real`**?

3. Expressions and control flow

The programs of the previous chapter ran from top to bottom. This chapter introduces the constructs that choose between alternatives and repeat actions, and the rules that say where a declared name can be used. It begins with the idea that makes Algol 68 different from most languages of its time and from many of ours: there is no separate category of statements. Every construct, including a block, a choice and a loop, is an expression.

3.1. Everything is an expression

In C, Java or Python, a program is made of statements, and expressions are one kind of statement. Algol 68 has only expressions, which the Report calls units. Constructs that look like statements, such as an assignment or a call of `print`, are expressions too; most of them simply have a value that nobody uses.

A block, written either **`begin`**…**`end`** or `(…)`, contains a sequence of declarations and expressions separated by semicolons, and its value is the value of the last expression. A block can therefore stand wherever a value is needed:

```
int square = (int t = 7; t * t);
real mean = begin
               real a = 2.0, b = 3.5;
               (a + b) / 2
            end;
print ((square, " ", fixed (mean, 0, 2), newline))
```

```
        +49 2.75
```

The two spellings of a block mean exactly the same; **`begin`** and **`end`** are usually preferred for long blocks, parentheses for short ones. The Report calls a block a closed clause and the sequence inside it a serial clause. Rust and the functional languages work the same way; in C, the closest equivalent is the "statement expression" of the GNU C compiler.

A block must end with an expression, not with a declaration, since it must have a value. The block **`begin int`** `x = 1` **`end`** is an error ("clause does not yield a value"), even in a place where its value is not used.

The value of a block is computed before the block ends, so a block may yield the value of one of its own variables: **`int`** `n = (`**`int`** `r; read (r); r)` reads a number through a local variable and yields it. What a block may not yield is the variable itself, a reference to a cell that disappears with the block; Chapter 7 explains this rule about lifetimes.

3.2. Sequences and discarded values

When an expression is followed by a semicolon, its value is discarded. The Report calls this voiding, and it is one of the implicit conversions: the value is converted to the type **void**, which has a single, uninteresting value. Discarding is what happens to the value of every assignment and every call of `print` in the programs so far:

```
int x := 3;
x + 1;                    { computed, then discarded }
x := x + 1;               { the reference x is discarded }
print ((x, newline))
```

```
         +4
```

The value of `x := x + 1` is the reference `x` (Section 2.11); it is discarded, and so is the value of `x + 1` in the line before, which therefore does nothing useful.

3.3. Scope

A name declared in a block can be used anywhere in that block, including in blocks nested inside it, but not outside. The part of the program in which a declaration is valid is its *scope* (the Report says range for the block, and reach for the block without the blocks nested in it). A nested block can declare a name that is already declared outside; inside the nested block, the new declaration hides the outer one:

```
int x = 1;
begin
   real x = 2.5;          { a new x, hiding the outer one }
   print ((x, newline))
end;
print ((x, newline))      { the outer x again }
```

```
+2.500000e +0
         +1
```

The same name cannot be declared twice in the same block; this is an error found by the compiler ("multiple declaration of identifier").

Strictly speaking, the scope of a declaration is the whole block, including the part before the declaration. The name can be mentioned there, but its value cannot be used, since the declaration has not been carried out yet. A program that uses a name before its declaration is therefore wrong, but compilers do not always detect it: ga68 and a68j accept such a program and use an undefined value. The rule matters for procedures that call each other (Chapter 5); for everything else, a declaration should simply come before its uses.

A word about the term: in this book, *scope* means the region of the program text in which a name can be used, as in every current language. The Report uses the word scope for something else, the lifetime of a value, which is the subject of Chapter 7.

3.4. Unordered evaluation

A list of expressions separated by commas and enclosed in parentheses is evaluated in an unspecified order:

```
int a, b;
(a := 1, b := 2);                  { in either order }
print ((a + b, newline))
```

```
         +3
```

Here the two assignments may be carried out in either order, or even interleaved; the program must not depend on the order. The Report calls this collateral elaboration, and the list a collateral clause. The same holds wherever several expressions separated by commas are all evaluated: the operands of an operator, the arguments of a procedure call, the items of a list given to `print`, the elements of an array literal, and the declarations in **`int`** `a = f, b = g`. (The commas between the alternatives of a **`case`** are different: only one alternative is evaluated.) A program whose result depends on the order in which `f` and `g` are called is wrong, even if it happens to work with a particular compiler. Some places without commas have the same freedom: the two sides of an assignment, an array and its indexes in `a[i]`, and a function and its arguments in a call.

A semicolon, by contrast, always means "first this, then that".

> **Pitfall.** Code that looks harmless can depend on the order. In `(k +:= 1) * k`, the right operand `k` may be read before or after the left one has incremented it. In **`int`** `a = 1, b = a + 1`, the two declarations are carried out in an unspecified order, so `b` may be computed before `a` exists; C programmers write this all the time, and C defines it, but Algol 68 does not. The same holds for two reads in one expression, as in `read_int - read_int` with Algol 68 Genie's `read_int`, and for `a[i] := (i +:= 1)` with `i` = 1, which may store 2 into `a[1]` or into `a[2]`. ga68 and a68j evaluate from left to right (here into `a[1]`), so testing does not reveal the error. Writing **`int`** `a = 1;` **`int`** `b = a + 1` with a semicolon makes the order definite. The items of a `print` list are evaluated in an unspecified order, but they are always printed in the order in which they are written.

3.5. if expressions

The conditional expression has the form

```
if condition then expressions else expressions fi
```

where **`fi`**, **`if`** spelled backwards, closes the construct. The **`else`** part may be omitted, and further conditions may be tested with **`elif`**. Like a block, an **`if`** has a value, so it serves both as a statement and as the conditional expression `c ? a : b` of C:

```
int n = 7;
if n MOD 2 = 0 then print (("even", newline)) else print (("odd", newline)) fi;
string size = if n < 10 then "small" elif n < 100 then "medium" else "large" fi;
print ((size, newline));
print (((n > 0 | "positive" | "not positive"), newline))
```

```
odd
small
positive
```

The last line shows the brief form: `(c | a | b)` means **`if`** `c` **`then`** `a` **`else`** `b` **`fi`**, and `|:` stands for **`elif`**. The brief form is convenient in short expressions; for anything longer, the words are easier to read.

The condition is itself a sequence and may contain declarations, which are then visible in all parts of the **`if`**:

```
if int k = n * n; k > 100 then print (k) fi
```

If the **`else`** part is omitted and the condition is false, the **`if`** does nothing, which is what is wanted in a statement. If such an **`if`** is used for its value, the value is undefined. The Report calls this undefined value **`skip`** (Section 3.8).

When the branches have different types, the compiler looks for a common type to which all branches can be converted, and converts them. If one branch is an **`int`** and the other a **`real`**, the common type is **`real`**:

```
int n = 7;
real r = if n > 5 then 1 else 2.5 fi;     { the int 1 is widened to real }
print ((r, newline))
```

```
+1.000000e +0
```

The Report calls this balancing. It is what C does for `c ? 1 : 2.5`, and what the type inference of a modern language does when it finds the least common type of the branches.

Since an **`if`** yields a value of any type, it can also yield a variable, and stand on the left-hand side of an assignment:

```
int a := 3, b := 7;
(a > b | a | b) := 0;           { sets the larger of a and b to 0 }
print ((a, b, newline))
```

```
          +3         +0
```

sets `b` to 0. Here both branches yield a variable of type **`ref int`**. With a **`real`** variable `x` and an **`int`** variable `n`, `(c  x | n) := 4|` is rejected: the branches yield a **`ref real`** and a **`ref int`**, and neither can be converted into the other.

3.6. case expressions

The **`case`** expression selects one of several alternatives by an integer: the first alternative for 1, the second for 2, and so on. If the integer is out of range, the **`out`** part is chosen; without an **`out`** part, nothing happens (or the value is undefined).

```
for day to 8 do
   string name = case day in "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"
                 out "???"
                 esac;
```

```
    print ((name, " "))
od;
print (newline);
int code = 12;
print (((code | "one", "two" | "many"), newline))
```

```
Mon Tue Wed Thu Fri Sat Sun ???
many
```

The alternatives are separated by commas, and each can be a single expression or a block. The Report requires at least two alternatives (for one, an **if** does the job); the three implementations also accept one. The brief form is `(i | a, b, c | d)`. A second selection can follow the **out** with **ouse**, much as **elif** follows the **then** part of an **if**.

This **case** corresponds to the `switch` of C, except that the alternatives are numbered from 1 and cannot be labeled with arbitrary constants. Chapter 7 introduces a second, more powerful form of **case** that selects by the type of a value.

3.7. Loops

Algol 68 has a single loop construct with several optional parts:

```
for i from first by step to last while condition do expressions od
```

Every part except **do**…**od** may be left out. The parts have the following meaning:

- **for** `i` names the *loop variable*; without it, the loop still counts, but the count has no name.
- **from** `first` gives the initial value, 1 if omitted.
- **by** `step` gives the increment, 1 if omitted; it may be negative.
- **to** `last` gives the final value; without it, there is no limit.
- **while** `condition` is tested before each iteration; the loop ends when it is false.

The values of `first`, `step` and `last` are computed once, before the first iteration. The loop variable is an **int** constant, not a variable: it is declared anew for each iteration, it cannot be assigned to, and it does not exist after the loop. The usual forms of the loop result from the parts that are present:

```
int sum := 0;
for i from 1 to 10 do sum +:= i od;
print ((sum, newline));
for i from 10 by -3 to 0 do print (i) od;
print (newline);
to 3 do print ("ha") od;
print (newline)
```

```
        +55
        +10         +7         +4         +1
hahaha
```

to 3 **do** …**od** repeats the body three times. A loop with only a **while** part is the `while` loop of other languages; Euclid's algorithm for the greatest common divisor is a classic example:

```
{ Euclid's algorithm: the greatest common divisor of a and b. }
int a := 1071, b := 462;
while b /= 0 do
   int r = a MOD b;
   a := b;
   b := r
od;
print (("gcd = ", whole (a, 0), newline))
```

```
gcd = 21
```

The **for** and **while** parts can be combined: the loop then ends when the counter passes the final value or the condition becomes false, whichever comes first. Like the condition of an **if**, the **while** part may contain declarations, which are visible in the body. This replaces the `do`…`while` loop of C, whose body runs before its test: the part of the body that must run first is written in the **while** part.

```
{ The first square greater than 50. }
int n := 1;
for i while i * i <= 50 do n := i + 1 od;
print ((n, n * n, newline));
{ The same, with a declaration in the while part. }
int m := 0;
while int square = m * m; square <= 50 do m +:= 1 od;
print ((m, newline))
```

```
         +8        +64
         +8
```

Finally, **do**…**od** alone repeats forever; the loop is left with a jump (Section 3.9). A loop has no value: its type is **void**.

Since the loop variable is a constant, an attempt to assign to it fails with the message we met in Chapter 2:

```
access Genie_Transput
begin
   for i to 10 do
      if i = 5 then i := 10 fi;
      print (i)
   od
end
```

```
loop-variable.a68:4:21: error: int secondary does not yield a name
```

A few details of the loop are worth knowing. Since the final value is computed once, changing a variable that appears in the **to** part inside the body does not change the number of iterations. The **while** part is not evaluated any more once the counter has passed the final value, which matters if it reads input or has other effects. With a step of 0 the final value

is ignored, and the loop runs forever unless the **while** part stops it. (ga68 and a68j instead skip such a loop if the start value is above the final value.)

The loop variable is always an **int**; there is no loop over real numbers with a step of 0.1. That is just as well, since real numbers are approximations, and adding 0.1 ten times does not give exactly 1.0:

```
real sum := 0;
to 10 do sum +:= 0.1 od;
print ((sum = 1.0, " ", ABS (sum - 1.0) < 1e-6, newline));
for i from 0 to 4 do
   real x = i / 4;                    { 0, 0.25, 0.5, 0.75, 1 }
   print ((fixed (x, -4, 2), " "))
od;
print (newline)
```

prints

```
F T
0.00 0.25 0.50 0.75 1.00
```

The right way is to count with an integer and compute each real value from the counter, as in the second loop, so that the errors do not accumulate. For the same reason, real numbers should not be compared with `=`, but tested for a small difference, as in `ABS (sum - 1.0) < 1e-6`. With the 32-bit **real** of ga68 and a68j, which has about 7 decimal digits, the problem appears sooner than with the 64-bit `double` that most programmers are used to.

The **while** part is the place for an iteration that must compute a value before it can decide whether to continue. Newton's method for the square root improves an approximation until the improvement is small relative to the value:

```
proc root = (real a) real:
begin
   real x := (a > 1 | a | 1);
   while real better = (x + a / x) / 2;
         ABS (better - x) > 1e-6 * x
   do x := better od;
   x
end;
print ((fixed (root (2), 0, 5), " ", fixed (root (1e6), 0, 2), " ",
        fixed (root (0.25), -7, 5), newline))
```

```
1.41421 1000.00 0.50000
```

prints `1.41421 1000.00 0.50000`. The test is relative, `1e-6 * x`, because a fixed tolerance such as `1e-6` is far too coarse for very small roots, and for large ones, with only 7 significant digits, it is reached only if the iteration happens to arrive at an exact fixed point.

> **Implementation note.** Algol 68 Genie has two extensions that appear in Rosetta Code programs: `downto` counts downwards (`FOR i FROM 3 DOWNTO 1`), and an `UNTIL` part before **od** ends the loop after the body if its condition is true. ga68 and a68j have neither; **by** `-1` and a **while** part do the same.

3.8. skip

The reserved word **skip** stands for an undefined value of whatever type is needed. It is used where the syntax requires an expression but no particular value is meaningful, for example as an alternative of a **case** that has nothing to do, or as the body of a loop that does all its work in the **while** part:

```
int n := 0;
while n +:= 1; n * n < 1000 do skip od;
print ((n, newline))
```

```
         +32
```

The loop finds the smallest `n` whose square is at least 1000, and prints 32. The semicolon in the **while** part makes sure that `n` is incremented before it is squared; the shorter `(n +:= 1) * n < 1000` would depend on the order of the operands.

3.9. Jumps

An expression in a sequence can be given a *label*, a name followed by a colon, and the **goto** expression continues the program at the labeled expression. **go to** may also be written as two words, and in many places the label alone is enough. A typical use is to leave a loop that runs forever:

```
    { Search for the first multiple of 7 and 11 above 500. }
    int n := 500;
    do
       n +:= 1;
       if n MOD 7 = 0 andth n MOD 11 = 0 then goto found fi
    od;
found:
    print ((n, newline))
```

```
        +539
```

A jump can only go to a label in the same block or in an enclosing block, never into a block, a loop or a branch, since the label is not visible there (Section 3.3).

A jump never yields a value, so it is acceptable wherever the type is determined by the context or by the other branches of an **if** or a **case**. This makes it easy to give up when the data are bad:

```
    [] real data = (4, 2.25, -1, 9);
    for i to UPB data do
       real r = if data[i] >= 0 then sqrt (data[i]) else goto bad_input fi;
       print ((fixed (r, 0, 2), newline))
    od;
    stop;
bad_input:
    print (("negative input", newline))
```

```
2.00
1.50
negative input
```

prints `2.00`, `1.50` and `negative input`. The **if** has the type **real**, and its **else** part, a jump, is acceptable as a **real**. The predefined label `stop` ends the whole program.

One case is different. Where a function without parameters is expected, as for a parameter of type **proc void**, a jump is not carried out: it becomes a function that carries out the jump when it is called. A function can thus be given an exit to take when something goes wrong:

```
   proc mean = ([] real v, proc void on_empty) real:
   begin
      if UPB v < LWB v then on_empty fi;
      real s := 0;
      for i from LWB v to UPB v do s +:= v[i] od;
      s / (UPB v - LWB v + 1)
   end;
   [1:0] real none;
   print ((fixed (mean ((1, 2, 3, 4), no_data), 0, 2), newline));
   print ((fixed (mean (none, no_data), 0, 2), newline));
   goto stop;
no_data:
   print (("no data", newline))
```

```
2.50
no data
```

The first call returns normally. In the second, `mean` calls `on_empty`, and the jump leaves `mean` and continues at the label `no_data`. In the same way, `[] proc void switch = (l1, l2, l3)` is an array of functions that jump, and `switch[i]` jumps to the `i`-th label, as the switch of Algol 60 did.

Labels have two restrictions. In a block, no declaration may follow a label, so that a jump can never skip a declaration whose name is used after the label; a label before a declaration is rejected with a message about an invalid sequence. And the condition of an **if**, a **case** or a **while** may not contain labels.

A block may also end before its last expression. The reserved word **exit** after an expression ends the sequence with that expression's value; the part after **exit** can only be reached by a jump to a label that follows it:

```
int x = -4;
int magnitude = begin
                   if x >= 0 then goto non_negative fi;
                   -x exit
                non_negative:
                   x
                end;
print ((magnitude, newline))
```

```
         +4
```

Jumps were common in programs of the 1960s, and Algol 68 still has them. With its

general loop and the value-yielding **if** and **case** they are rarely needed, and the programs in this book use them only to leave a loop early. The keyword **exit** should not be confused with the `exit` or `break` of other languages: it does not leave a loop.

3.10. The structure of a program

A program is a block, or more generally any construct in brackets: an **if**, a **case** or a loop is a program too. Everything it uses that it does not declare itself comes from the standard library, which can be thought of as a block enclosing the program. In this book, a program is preceded by **access** `Genie_Transput`, which places the declarations of that module around the program as well.

> **Implementation note.** Algol 68 Genie also accepts a program without the enclosing **begin** and **end**, a convenience that many Rosetta Code programs use. ga68 and a68j require the brackets.

3.11. A complete program

The program below lists the prime numbers below 60. For each number `n`, it tries the divisors from 2 upwards while no divisor has been found and the divisor squared does not exceed `n`. The short-circuit **andth** (Section 2.9) is not essential here, but it expresses the intent: once `prime` is false, the rest of the condition does not matter.

```
access Genie_Transput
begin
   { The prime numbers below 60, by trial division. }
   int count := 0;
   for n from 2 to 59 do
      bool prime := true;
      for d from 2 while prime andth d * d <= n do
         if n MOD d = 0 then prime := false fi
      od;
      if prime then
         print (whole (n, -4));
         count +:= 1;
         if count MOD 10 = 0 then print (newline) fi
      fi
   od;
   print (newline)
end
```

```
   2   3   5   7  11  13  17  19  23  29
  31  37  41  43  47  53  59
```

Exercises

The answers are at the end of the book.

1. What are the values of `(int a = 3; a * a + 1)` and of `(2 > 1 | 10 | 20) + 1`?

2. What does the following program print?

```
int x = 1;
begin int y = x + 1; begin int x = y * 10; print (x) end; print (x) end
```

3. How many times is the body of each loop executed?
   (a) `for i from 5 to 1 do skip od`
   (b) `for i from 1 by 2 to 10 do skip od`
   (c) `for i from 10 by -2 to 1 do skip od`
   (d) `to 0 do skip od`

4. Write `(x > 0 | "plus" |: x < 0 | "minus" | "zero")` with reserved words instead of the brief form.

5. What type has `(n > 0 | 1 | 0.5)`, and why?

6. Why may a program not rely on the order of the two calls in `f + g`, if `f` and `g` print something?

7. Write a loop that prints the powers of 2 up to 1000 without using a **`for`** part.

8. What does `int k := 3; to k do k *:= 2; print (k) od` print?

9. What does `for i from 1 by 0 to 3 do print (i) od` do?

10. Is `string name = case k in "one", "two", "three" out goto bad esac` a correct declaration, if `bad` is a label of the enclosing serial clause? What happens for `k = 5`?

11. With `real x := 1.5; int n := 2; bool b = x > n`, the declaration `real v = (b | x | n)` is accepted, but `ref real r = (b | x | n)` is rejected. Why?

12. What is wrong with `int a = 1, b = a + 1`, and how is it corrected?

13. The loop `int n; while read (n); n < 1 AND n > 9 do ... od` is meant to repeat until a number from 1 to 9 is read. Why does it stop after the first number, whatever it is?

Programming problems

1. Print a multiplication table for the numbers 1 to 9, with the numbers aligned in columns.

2. Read numbers until a 0 is read, and print their count, their sum and their largest value.

3. The Collatz sequence starting at n continues with $n/2$ if n is even and with $3n + 1$ if n is odd, until it reaches 1. Print the starting value below 1000 with the longest sequence.

4. Print the perfect numbers below 10000 (numbers that are equal to the sum of their divisors other than themselves, such as $6 = 1 + 2 + 3$).

5. Print a calendar for a month, given the number of days and the weekday of the first day.

6. **Number bases.** Read a number written in a base between 2 and 16 (the base first, then the digits, with `A` to `F` for 10 to 15) character by character, and print its value. Report a digit that is not valid for the base and stop with **`goto`** `stop`.

7. **Brackets.** Read a line and decide whether its round brackets are properly nested. Why can the same method not check two kinds of brackets, `()` and `[]`, without an array?

8. **Arithmetic without brackets.** Read an expression of integers with `+`, `-` and `*`, such as `2+3*4-5`, and evaluate it with the usual precedence. Keep a running sum and a pending product, without arrays.

9. **Fibonacci and overflow.** Print the Fibonacci numbers up to the largest one that fits in an **`int`**, testing before each addition whether it would overflow (`a > max_int - b`). How many are there with a 32-bit and with a 64-bit **`int`**?

10. **Cube roots.** Compute cube roots with Newton's iteration $x_{n+1} = (2x_n + a/x_n^2)/3$, with a relative test for the end, and compare the results with `exp (ln (a) / 3)`. What happens for negative `a`?

11. **Prime factors.** Print the factorization of a number as a product of prime powers, such as `360 = 2^3 * 3^2 * 5`.

12. **Armstrong numbers.** 153 equals the sum of the cubes of its digits. Find all three-digit numbers with this property.

4. Arrays and records

The values of the previous chapters were single numbers, characters and truth values. Real programs work with collections of values: a list of measurements, a matrix, a table of names and dates. Algol 68 provides two ways of building such values. An *array* holds a number of values of the same type, selected by position; a *record* holds a fixed set of values of possibly different types, selected by name. Both can be combined freely: arrays of records, records containing arrays, arrays of arrays. This chapter also introduces type declarations, which give a name to a type.

4.1. Arrays

An array variable is declared by giving its bounds in square brackets before the element type:

```
[1:5] int squares;
for i from LWB squares to UPB squares do squares[i] := i * i od;
print ((squares, newline));
print ((squares[3], " ", LWB squares, UPB squares, newline));
[0:2] char letters;
letters[0] := "x"; letters[1] := "y"; letters[2] := "z";
print ((letters[0], letters[2], newline))
```

```
         +1         +4         +9        +16        +25
         +9          +1         +5
xz
```

The declaration `[1:5] int squares` allocates five cells for `int` values, numbered 1 to 5. Unlike in C, Java or Python, the numbering need not start at 0 or 1: `[0:2] char letters` has the indices 0, 1 and 2, and an array of the years 1990 to 2025 could be declared as `[1990:2025] real`. The lower bound is the first number and the upper bound the second; the operators `LWB` and `UPB` give them. A loop over all elements should use them rather than repeat the numbers. If the lower bound is 1, it can be left out: `[n] real` means `[1:n] real`.

An element is selected by an *index* in square brackets, as in `squares[3]`. The Report calls an array a multiple value or row, and the selection of an element a slice.

The bounds are evaluated when the declaration is carried out, so the size of an array can depend on input. The following program reads a count and then that many numbers:

```
int n = read_int;
[n] real values;                    { the same as [1:n] real values }
for i to n do values[i] := read_real od;
real sum := 0;
for i to n do sum +:= values[i] od;
print (("mean of ", whole (n, 0), " values: ", fixed (sum / n, 0, 3), newline))
```

```
mean of 4 values: 3.000
```

The input was:

```
4
2.5 3.5
1.0 5.0
```

Once declared, the array keeps its bounds; an array that can grow or shrink is the subject of Chapter 7. Every index is checked: an index outside the bounds stops the program with an error.

```
access Genie_Transput
begin
   [1:3] int a := (10, 20, 30);
   int i := 4;
   print ((a[i], newline))
end
```

```
out-of-bounds.a68:5: runtime error: bound 4 out of range [1:3]
```

4.2. Array literals

A list of values in parentheses, separated by commas, is an *array literal* (row display) wherever an array is expected. A constant array can take its bounds from the literal: the declaration `[] int primes = (2, 3, 5, 7, 11)` gives no bounds, and `primes` has the bounds 1 to 5. The Report calls a type written without bounds, such as `[] int`, a formal declarer; one with bounds, as in the declaration of a variable, an actual declarer.

```
[] int primes = (2, 3, 5, 7, 11);
[] real weights = (1, 0.5, 0.25);         { each int is widened }
[] int none = ();
print ((UPB primes, UPB weights, UPB none, newline));
[1:3] int v := (7, 8, 9);
v := (1, 2, 3);                           { replaces all elements }
print ((v, newline))
```

```
         +5         +3         +0
         +1         +2         +3
```

The elements of a literal are converted to the element type where necessary, so the integers of `weights` are widened to `real`. The empty literal `()` gives an array with the bounds 1 to 0, that is, with no elements. A literal can also be assigned to an array variable, which replaces all its elements. A literal always has the lower bound 1, so its bounds match those of the variable only if the variable has the lower bound 1 and as many elements as the literal. For a variable `[0:2] int w`, the assignment `w := (1, 2, 3)` fails, but `w[@1] := (1, 2, 3)` works: `w[@1]` is the whole of `w`, renumbered to start at 1 (Section 4.5).

4.3. Arrays are values

An array in Algol 68 is a value, like a number. Assigning one array variable to another copies the elements:

```
[1:3] int a := (1, 2, 3), b;
b := a;                    { copies the elements }
b[1] := 99;
print ((a[1], b[1], newline))
```

```
         +1        +99
```

After the assignment, `a` and `b` are separate arrays; changing `b` does not change `a`. This is the behavior of arrays in C when they are inside a `struct`, or of the arrays of Fortran, but not that of Java, Python or JavaScript, where an assignment makes two names refer to the same array. In Algol 68, two names refer to the same array only if a reference to it is shared explicitly (Chapter 7). A constant declared with an array value, `[] int c = b`, is a copy as well: later changes of `b` do not show in `c`. The same holds for a parameter of type `[] int` (Chapter 5).

> **Implementation note.** Algol 68 Genie does not always copy: if a procedure has a `[] int` parameter and a `ref [] int` parameter, and both are given the same array, a change through the reference shows in the value parameter during the call. ga68 and a68j copy, as the Report requires.

Since an array variable has fixed bounds, an assignment must supply an array with the same bounds. A mismatch is detected when the program runs:

```
access Genie_Transput
begin
   [1:3] int a;
   [1:4] int b := (1, 2, 3, 4);
   a := b;
   print ((a, newline))
end
```

```
bounds-mismatch.a68:5: runtime error: multiple bounds mismatch in assignation: dim 1:
    [1:3] /= [1:4]
```

4.4. Multi-dimensional arrays

An array can have several dimensions; the bounds of each dimension are separated by commas, and so are the indices:

```
[1:2, 1:3] int m := ((1, 2, 3),
                     (4, 5, 6));
print ((whole (1 UPB m, 0), " rows, ", whole (2 UPB m, 0), " columns", newline));
for i to 1 UPB m do
   for j to 2 UPB m do print (whole (m[i, j], -4)) od;
   print (newline)
```

```
od
```

```
2 rows, 3 columns
   1   2   3
   4   5   6
```

`1 UPB m` is the upper bound of the first dimension and `2 UPB m` that of the second; plain `UPB m` means `1 UPB m`. A literal for a two-dimensional array is a list of rows, each of which is a list.

An array of arrays is something different: `[] [] int` is an array whose elements are themselves arrays, and those may have different lengths, as in `((1), (2, 3), (4, 5, 6))`. Its elements are selected with two pairs of brackets, `x[3][2]`. A two-dimensional array, by contrast, is always rectangular.

4.5. Slices

An index range in place of an index selects a part of an array, a *slice*:

```
[1:6] int a := (10, 20, 30, 40, 50, 60);
print ((a[2:4], newline));
print ((LWB a[2:4], UPB a[2:4], newline));        { bounds 1 to 3 }
print ((LWB a[2:4 @ 2], UPB a[2:4 @ 2], newline)); { bounds 2 to 4 }
print ((a[:2], " ", a[5:], newline));
a[1:3] := (0, 0, 0);                               { assigns into a }
print ((a, newline))
```

```
        +20        +30        +40
         +1         +3
         +2         +4
        +10        +20         +50        +60
         +0         +0         +0        +40        +50        +60
```

The slice `a[2:4]` consists of the elements 2 to 4 of `a`, but its own bounds are 1 to 3: a range is renumbered from 1 unless a new lower bound is given after `@`, as in `a[2:4 @ 2]`. Either end of the range can be left out; it then defaults to the corresponding bound of the array. The Report calls an index range a trimmer.

A slice of an array variable is not a copy: it refers to the elements of the original array, so an assignment to the slice changes the array, as the last line shows. In the terms of Section 2.6, the slice of a reference to an array is a reference to part of the array. An assignment between overlapping parts of an array is safe, since the right-hand side is a value that is complete before anything is stored:

```
[6] int a := (1, 2, 3, 4, 5, 6);
a[2:6] := a[1:5];              { shift right by one place }
print ((a, newline))
```

```
         +1         +1         +2         +3         +4         +5
```

shifts the elements one place to the right, which in C would need `memmove` rather than `memcpy`.

In a multi-dimensional array, an index can be given for some dimensions and left out for others. Leaving out the second index of a matrix selects a row, leaving out the first selects a column:

```
[1:3, 1:3] int m := ((1, 2, 3), (4, 5, 6), (7, 8, 9));
print ((m[2, ], newline));            { the second row }
print ((m[, 3], newline));            { the third column }
m[, 1] := (0, 0, 0);                  { clears the first column }
print ((m[3, ], newline))
```

```
         +4         +5         +6
         +3         +6         +9
         +0         +8         +9
```

A dimension whose index is left out entirely keeps its bounds; only a range with a colon is renumbered:

```
[0:9, 2:11] int m;
print ((LWB m[1, ], UPB m[1, ], newline));        { 2 and 11: kept }
print ((LWB m[1, :], UPB m[1, :], newline));      { 1 and 10: renumbered }
print ((LWB m[1, 3:], UPB m[1, 3:], newline));    { 1 and 9 }
print ((LWB m[1, 3:10 @ 3], UPB m[1, 3:10 @ 3], newline))
```

```
         +2        +11
         +1        +10
         +1         +9
         +3        +10
```

For a matrix whose bounds do not start at 1, a loop over a row `m[i, ]` must therefore run from `LWB m[i, ]`, not from 1.

A row or an element that a loop body uses several times can be given a name of its own. Inside a loop over `i`, the declaration **`ref`** `[] int row = m[i, ]` makes `row[j]` the same cell as `m[i, j]`, so assignments to it change the matrix. For an element that is only read, **`int`** `x = v[i]` names its current value; this is a copy, which later assignments to `v[i]` do not change.

4.6. Strings are arrays

The type **`string`** is an array of characters. Strings normally have the lower bound 1; a string variable takes the bounds of whatever is assigned to it, so a slice with `@` can give it another, and loops should use `LWB s`. Everything said about arrays applies to it: `UPB s` is normally its length, `s[i]` its `i`-th character, and `s[2:4]` a part of it:

```
string s = "Algol 68 report";
print ((UPB s, " ", s[1], " ", s[7:8], newline));
int vowels := 0;
for i to UPB s do
   char c = s[i];
   if c = "a" OR c = "e" OR c = "i" OR c = "o" OR c = "u" then vowels +:= 1 fi
od;
```

```
print ((vowels, newline))
```

```
        +15 A 68
         +3
```

Strings are compared with `=` and `<` in the order of their character codes, which is alphabetical for letters of the same case (in ASCII, every capital letter comes before every small letter), and joined with `+`. A string variable can hold strings of different lengths, which an ordinary array cannot; Chapter 7 explains how.

4.7. Records

A *record* groups values of possibly different types into one value. Its type, written with **`struct`**, lists the types and names of its *fields*:

```
struct (string name, int born, bool alive) p := ("Ada", 1815, false);
print ((name of p, " was born in ", whole (born of p, 0), newline));
born of p := 1816;                          { assigns to one field }
p := ("Grace", 1906, false);                { assigns all fields }
print ((name of p, " ", whole (born of p, 0), newline))
```

```
Ada was born in 1815
Grace 1906
```

The record type of `p` has three fields: `name`, a **`string`**, `born`, an **`int`**, and `alive`, a **`bool`**. A field is selected with **`of`**: `name of p` is what most languages write as `p.name`. If `p` is a variable, `born of p` is a reference to the field, so the field can be assigned to on its own. The Report calls a record a structure, a field name a field selector, and `name of p` a selection.

A record value can be written as a list of field values in parentheses, in the order of the fields (structure display). Such a literal needs at least two fields; a record with a single field must be built by assigning to the field.

Selection with **`of`** binds less tightly than indexing: `name of class[i]` means `name of (class[i])`, the name of the `i`-th element of an array of records. If a field is itself an array, parentheses are needed to index it: `(marks of class[i])[k]`. The same holds for a field that is a function: `(f of s) (x)` calls it, while `f of s (x)` selects the field `f` of `s (x)` and is rejected, with a message that `s` must yield a function.

4.8. Type declarations

Writing out a record type at each use would be tedious and error-prone. A *type declaration* (mode declaration) gives a type a name:

```
mode Point = struct (real x, y);
mode Circle = struct (Point center, real radius);
Circle c := ((0, 0), 1.5);
x of center of c := 2;
print ((fixed (x of center of c, 0, 1), " ",
        fixed (radius of c, 0, 1), newline))
```

```
2.0 1.5
```

The names of declared types begin with a capital letter (Section 1.4). A declared type can be used wherever a type is expected, including in other type declarations: a `Circle` contains a `Point`. The field `x of center of c` is the `x` of the `center` of `c`.

A type declaration does not create a new, distinct type; it gives a name to a type that is determined by its structure. Two record types with the same field types and field names, in the same order, are the same type, whatever they are called:

```
mode Meters = struct (real value, int precision);
mode Seconds = struct (real value, int precision);
Meters distance = (100, 2);
Seconds time = distance;                { allowed: the same type }
print ((fixed (value of time, 0, 1), newline))
```

```
100.0
```

This *structural equivalence* differs from the *nominal typing* of C, Java, Rust and most current languages, where two separately declared `struct`s are different types even if their fields are identical. In Algol 68, a `Meters` and a `Seconds` as declared above are interchangeable, and the type system does not keep them apart. Chapter 7 returns to the question of when two types are the same.

A type declaration can also name an array type, such as `mode Vector = [1:3] real`. It must then give the bounds: a type without bounds, such as `[] int`, is allowed for a parameter or a constant, which take their bounds from a value, but not in a type declaration, since a type declaration may be used to declare variables.

4.9. Arrays of records

When the fields of an array of records are selected with `of`, the result is an array of fields:

```
mode Point = struct (int x, y);
[1:4] Point path := ((0, 0), (3, 4), (6, 0), (9, 4));
print ((x of path, newline));           { all x fields: a [] int }
y of path[2] := 5;                      { one field of one element }
print ((y of path, newline))
```

```
         +0         +3         +6         +9
         +0         +5         +0         +4
```

`x of path` is an array of the four `x` values. If `path` is a variable, it is even a reference to them, so that `x of path := (1, 2, 3, 4)` assigns to the `x` field of every element. The Report calls this multiple selection. Few current languages have it; the closest analog is selecting a column of a table in a data-frame library.

4.10. A single value as an array

Where an array is required and a single value is given, the value is converted into an array with one element, the bounds 1 to 1:

```
[] int single = 42;                 { a one-element array }
print ((LWB single, UPB single, single[1], newline));
print (newline)                     { a list of one item }
```

```
         +1         +1        +42
```

This implicit conversion is called *rowing* (rowing) in the Report, since an array is a row. It is why `print (newline)` needs only one pair of parentheses: `print` expects a list, and the single item `newline` is turned into a list of one. It is also why `y of ps := 0`, for an array `ps` of three points, fails when the program runs: `0` becomes an array with one element, which does not fit the three `y` fields.

4.11. Printing arrays and records

`print` accepts arrays of numbers, truth values and characters, and prints their elements one after the other, as the examples of this chapter show. A matrix is printed row by row, without line breaks between the rows. The Report's `print` also accepts records, which it prints field by field, as long as they contain no references, functions or unions. A **`string`** field is printed as its characters, and an array of strings as its strings one after the other.

> **Implementation note.** `Genie_Transput` does not accept records in `print`; their fields must be printed one by one. Algol 68 Genie accepts them. All three implementations print arrays of strings.

4.12. Two complete programs

The first program sorts an array with *insertion sort*: each element in turn is moved to the left until the elements before it are not greater. The constant array `data` is copied into a variable with the same bounds, which the program then rearranges.

```
access Genie_Transput
begin
   { Insertion sort. }
   [] int data = (31, 41, 59, 26, 53, 58, 97, 93, 23, 84);
   [LWB data : UPB data] int a := data;
   for i from LWB a + 1 to UPB a do
      int v = a[i];
      int j := i - 1;
      while j >= LWB a andth a[j] > v do
         a[j + 1] := a[j];
         j -:= 1
      od;
      a[j + 1] := v
   od;
```

```
   for i to UPB a do print (whole (a[i], -4)) od;
   print (newline)
end
```

```
  23  26  31  41  53  58  59  84  93  97
```

The condition of the inner loop uses **andth** (Section 2.9): when `j` has passed the lower bound, `a[j]` must not be evaluated, since the index would be out of range. With `AND`, both operands would be evaluated and the program would stop with an index error.

The second program keeps a table of records, each with a name and three marks, and finds the student with the best average:

```
access Genie_Transput
begin
   { The student with the best average mark. }
   mode Student = struct (string name, [1:3] int marks);
   [] Student class = (("Ann",  (72, 85, 90)),
                       ("Ben",  (88, 79, 81)),
                       ("Cleo", (95, 91, 70)));
   real best := -1;
   int best_index := 0;
   for i to UPB class do
      int sum := 0;
      for k to 3 do sum +:= (marks of class[i])[k] od;
      real average = sum / 3;
      print ((name of class[i], ": ", fixed (average, 0, 2), newline));
      if average > best then best := average; best_index := i fi
   od;
   print (("best: ", name of class[best_index], newline))
end
```

```
Ann: 82.33
Ben: 82.67
Cleo: 85.33
best: Cleo
```

The record type `Student` has an array field with the bounds 1 to 3, and the array `class` takes its bounds from the literal. Note the parentheses in `(marks of class[i])[k]`, explained in Section 4.7. The bounds in a type apply only where a variable is created: they are computed anew for each variable, and a constant of type `Student` takes the bounds of its value. A literal with four marks would be accepted in `class`, and the error would appear only when such a record is assigned to a `Student` variable.

> **Implementation note.** ga68 and a68j do not check the bounds of an array field when a record is assigned: a `Student` variable then holds four marks. Algol 68 Genie reports the error.

Exercises

The answers are at the end of the book.

1. Declare an array that holds one `real` for each year from 1900 to 2000, and write a loop that sets all elements to 0.
2. Given `[] int a = (5, 6, 7, 8, 9)`, what are `a[2:3]`, `LWB a[2:3]`, `UPB a[3:]` and `a[2:3 @ 0][0]`?
3. Given `[1:3, 1:4] int m`, what are the types and bounds of `m[2, ]`, `m[, 3]` and `m[2:3, 2]`?
4. After `[1:3] int a := (1, 2, 3), b := (4, 5, 6)`, what does `a := b; b[1] := 0` leave in `a`?
5. Declare a type `Date` with fields `day`, `month` and `year`, and a constant for 3 November 1975.
6. Why does `name of class[1][1]` not give the first character of the first name in an array `class` of records with a `string` field `name`? How must it be written?
7. Given `[1:10] Point ps` with `mode Point = struct (real x, y)`, write a single assignment that sets the `y` field of all ten points to 0.
8. For `[0:9, 2:11] int m`, what are the bounds of `m[1, ]`, `m[1, 3:5]` and `m[ , 2]`?
9. What does `a[2:5] := a[1:4]` leave in `[6] int a := (1, 2, 3, 4, 5, 6)`?

Programming problems

1. Read ten numbers and print them in reverse order.
2. Read a matrix of 3×3 integers and print its transpose and the sum of its diagonal.
3. Multiply two matrices whose sizes are read from the input.
4. Use the sieve of Eratosthenes, an array of `bool`, to print the prime numbers below 1000.
5. Keep a table of records of the planets (name, distance from the sun, diameter) and print them sorted by diameter.
6. **Binary search.** Write a binary search for a value in a sorted array that works for any bounds, using `LWB` and `UPB`. Test it on arrays with the bounds 1 to 10, 0 to 9 and -5 to 4.
7. **Merging.** Merge two sorted arrays into one sorted array, so that a value that occurs in both appears only once.
8. **Rotation.** Rotate a square matrix by 90 degrees using row and column slices (`r[ , n + 1 - i] := m[i, ]`), and rotate an array by r places with slice assignments only.
9. **Game of Life.** Play Conway's Game of Life on a `[0:n+1, 0:m+1] bool` board whose border stays empty, so that the loop over the neighbors needs no special cases. Compute each generation into a second board and assign it to the first.

10. **Generalized Fibonacci.** In the Fibonacci numbers of order m, each number is the sum of the m before it, starting with $m - 1$ zeros and a one. Print the first 30 for a given m, keeping only the last m numbers in an array `[0 : m - 1] int` indexed with `MOD`.

11. **League table.** Read the results of the games of a league, keep a table of records (team, games, points, goals for and against), and print it sorted by points, then by goal difference.

5. Functions and operators

A program of any size is built from parts that each solve one task and that can be used again: functions. In Algol 68, a function is a value like a number or an array. It has a type, it can be bound to a name, stored in a variable, passed to another function and returned from one. The Report calls a function a procedure; this book uses both words, since Algol 68 programs declare functions with the reserved word **proc**.

The second half of the chapter shows how a program defines operators of its own. Operators are functions with a different calling syntax, and they make it possible to extend the arithmetic of the language to new types, such as fractions or vectors.

5.1. Declaring functions

A function declaration gives the function a name, its parameters with their types, its result type and its body:

```
proc max = (int a, b) int: (a > b | a | b);
proc hypotenuse = (real a, b) real: sqrt (a * a + b * b);
proc greet = (string name) void:
   print (("Hello, ", name, "!", newline));
print ((max (3, 7), " ", fixed (hypotenuse (3, 4), 0, 1), newline));
greet ("Algol")
```

```
         +7 5.0
Hello, Algol!
```

The declaration `proc max = (int a, b) int: (a > b | a | b)` has the same form as the constant declarations of Chapter 2: a name, an equals sign and a value. The value is a *function literal* (routine text), the part `(int a, b) int: (a > b | a | b)`. It lists the parameters in parentheses, then the result type, a colon and the body, which is a single expression, usually a block. The type of `max` is `proc (int, int) int`, the type of functions that take two `int` values and return an `int`. Parameters of the same type can share the type, as in `(int a, b)`; parameters of different types are separated by commas: `(string s, int n)`.

The declaration `proc max = (int a, b) int: ...` is short for `proc (int, int) int max = (int a, b) int: ...`, which names the type in full. The short form requires a function literal on the right-hand side. A function value computed in some other way, such as a choice between two functions, needs the full type: `proc (real) real f = (c | sin | cos)`. The short form with such a value is rejected with a message about a missing symbol that does not point to the cause.

A function whose result is of no interest has the result type `void`, as `greet` shows. There is no `return` statement: the result of a function is the value of its body, which is the value of the last expression in the block. A function that must finish early jumps to a label at the

end of its body (Section 3.9), which is rarely necessary since **if** and **case** yield values.

A call gives the arguments in parentheses after the name. The Report calls the parameters formal parameters and the arguments actual parameters.

5.2. Parameters

When a function is called, each parameter is bound to its argument exactly as a constant is bound to its value in a declaration: the call `max (3, 7)` behaves like the declarations **int** `a = 3, b = 7` followed by the body. This explains the rules for parameters, which differ from those of most languages.

A parameter is a constant. It cannot be assigned to:

```
access Genie_Transput
begin
   proc countdown = (int n) void:
      while n > 0 do print (n); n := n - 1 od;
   countdown (3)
end
```

```
param-constant.a68:4:33: error: int secondary does not yield a name
```

A function that needs a variable for its work declares one, for example **int** `k := n`. A function that is to change a variable of its caller takes a reference to that variable as its parameter:

```
proc swap = (ref int a, b) void: (int t = a; a := b; b := t);
int x := 1, y := 2;
swap (x, y);
print ((x, y, newline))
```

```
          +2         +1
```

The parameters of `swap` are of type **ref int**. The call `swap (x, y)` passes the references `x` and `y`, and the assignments in the body go to the caller's variables. In C this would be done with pointers, in C++ with reference parameters, in Pascal with `var` parameters. Algol 68 needs no special kind of parameter for it: the reference is an ordinary value, passed like any other.

Both kinds of parameter can be arrays. A parameter of type `[]` **real** accepts an array of **real** with any bounds, and the function finds them with `LWB` and `UPB`. A parameter of type **ref** `[]` **real** accepts an array variable, whose elements the function can then change:

```
proc sum = ([] real v) real:
begin
   real s := 0;
   for i from LWB v to UPB v do s +:= v[i] od;
   s
end;
proc scale = (ref [] real v, real factor) void:
   for i from LWB v to UPB v do v[i] *:= factor od;
[0:3] real w;
```

```
w[@1] := (1, 2, 3, 4);
scale (w, 10);
print ((fixed (sum (w), 0, 1), " ", fixed (sum ((0.5, 0.25)), 0, 2), newline))
```

```
100.0 .75
```

The second call of `sum` passes an array literal. Since a literal has the bounds 1 to n, the function sees those bounds; for the variable `w`, it sees the bounds 0 to 3.

A function can also return a reference to a cell that already exists, such as an element of an array it receives by reference. Its call can then stand on the left of an assignment, as in C++ a function that returns a reference:

```
proc max_elem = (ref [] real a) ref real:
begin
   int k := LWB a;
   for i from LWB a + 1 to UPB a do
      if a[i] > a[k] then k := i fi
   od;
   a[k]
end;
[4] real v := (3, 9, 4, 1);
max_elem (v) := 0;
max_elem (v) *:= 10;
for i to 4 do print ((fixed (v[i], 0, 1), " ")) od;
print (newline)
```

```
3.0 .0 40.0 1.0
```

The parameter must be a **`ref [] real`**. With **`[] real a`**, the element `a[k]` would be a value, not a cell of the caller's array, and could not be returned as a **`ref real`**.

5.3. Functions without parameters

A function may have no parameters. Its type is then **`proc int`** (or **`proc void`**, and so on), and its literal has no parameter list: **`int`**`: expression`. Such a function is called simply by writing its name; no empty parentheses are needed, or even allowed:

```
int seed := 1;
proc next = int: (seed := seed * 75 MOD 65537; seed);
int a = next;              { next is called: no parentheses needed }
int b = next;
print ((a, b, newline))
```

```
         +75      +5625
```

Writing `next` where an **`int`** is expected calls the function. This is another implicit conversion, which the Report calls deproceduring. Where a function itself is expected, the name is not a call: `newline` in a call of `print` is a function with a parameter, passed as a value, and `print` calls it at the right moment.

Each call of `next` changes the variable `seed`, so the two calls give different results. In a list

such as `print ((next, next))` the order of the two calls would be unspecified (Section 3.4); the program above therefore calls the function in two separate declarations.

A parameterless function whose value is discarded is called as well, which is how a function of type **`proc void`** is called as a statement. (`newline` is not such a function: it has a file parameter, and `newline;` alone does nothing; `print (newline)` writes a line break.) There is one exception that is easy to overlook:

```
proc hello = void: print (("hello", newline));
proc void p;
p := hello;           { assigns the function; nothing is called }
print (("assigned", newline));
p;                    { a name alone: the function is called }
hello
```

```
assigned
hello
hello
```

prints `assigned` and then `hello` twice. The assignment `p := hello` stores the function in the variable `p` and does not call it, although its value, the variable `p`, is discarded. A discarded assignment, a reference comparison, an allocation, a type annotation, a literal and a format are discarded as they are; any other expression that yields a function without parameters is called.

A parameter of type **`proc real`** differs from one of type **`real`** in when the argument is evaluated. A **`real`** parameter receives a value, computed once before the call; a **`proc real`** parameter receives a function, which is called at every use of the parameter:

```
proc twice_value = (real x) real: x + x;
proc twice_call = (proc real x) real: x + x;
int n := 0;
proc count = real: n +:= 1;
print ((fixed (twice_value (count), 0, 1), newline));
print ((fixed (twice_call (count), 0, 1), newline))
```

```
2.0
5.0
```

`twice_value` gets the value 1 and doubles it. `twice_call` gets the function `count` and calls it twice, which gives 2 and 3, and returns their sum, 5. Passing a literal such as **`real`**`: v[i] * w[i]` for a **`proc real`** parameter gives the effect of Algol 60's call by name: the expression is evaluated anew at each use, with the current value of `i`. Algol 60 programs used this for summation functions, a trick known as Jensen's device; in Algol 68 a parameter of type **`proc`**` (`**`int`**`) `**`real`** does the same more plainly.

5.4. Recursion

A function may call itself. The classic examples are the factorial and the Fibonacci numbers:

```
proc factorial = (int n) int: (n <= 1 | 1 | n * factorial (n - 1));
```

```
proc fib = (int n) int: (n < 2 | n | fib (n - 1) + fib (n - 2));
print ((factorial (10), fib (20), newline))
```

```
   +3628800      +6765
```

Two functions can also call each other. Since the scope of a declaration is the whole block (Section 3.3), `is_even` can mention `is_odd` before `is_odd` is declared. This is allowed because the call only happens later, when both declarations have been carried out:

```
proc is_even = (int n) bool: (n = 0 | true | is_odd (n - 1));
proc is_odd = (int n) bool: (n = 0 | false | is_even (n - 1));
print ((is_even (10), " ", is_odd (10), newline))
```

```
T F
```

5.5. Functions as values

A function is a value, so there are also variables that hold functions. The declaration **`proc (real) real`** `f := sqrt` declares a variable for functions from **`real`** to **`real`** and stores in it the square root of the standard library. A later assignment can store another function, for example a function literal written directly in the program:

```
proc (real) real f := sqrt;
print ((fixed (f (2), 0, 4), newline));
f := (real x) real: x * x;              { a function literal }
print ((fixed (f (3), 0, 4), newline))
```

```
1.4142
9.0000
```

A function literal is what other languages call a *lambda* or an anonymous function. Functions that take other functions as parameters are called *higher-order functions*. A typical example computes an integral numerically:

```
{ The integral of f from a to b, by the trapezoidal rule with n steps. }
proc integrate = (proc (real) real f, real a, b, int n) real:
begin
   real h = (b - a) / n;
   real sum := (f (a) + f (b)) / 2;
   for i to n - 1 do sum +:= f (a + i * h) od;
   sum * h
end;
proc bell = (real x) real: exp (-x * x);
print ((fixed (integrate (bell, -3, 3, 100), 0, 4), newline));
print ((fixed (integrate ((real x) real: x * x, 0, 3, 100), 0, 4), newline))
```

```
1.7724
9.0004
```

The first call passes a declared function, the bell curve e^{-x^2}, whose integral is the square

root of π, about 1.7725; the second passes a literal for x^2. The function in a call can itself be the result of an expression: `(b  sin | cos) (x)|` chooses the function first and then calls it with `x`.

The body of a function can use the names of the blocks around it, not only its parameters. A function can thus read and change variables of the enclosing blocks:

```
int calls := 0;
proc counted_square = (int n) int: (calls +:= 1; n * n);
int total := 0;
for i to 4 do total +:= counted_square (i) od;
print ((total, calls, newline))
```

```
        +30         +4
```

Such a function is called a *closure* in current terminology. Algol 68 closures have a restriction that the closures of Lisp, JavaScript or Python do not have: a function may not be used after the block that declares a name it uses has ended. A function that uses a variable, a constant or a parameter of a block therefore cannot be returned from that block or stored in a variable outside it. Section 7.16 explains this rule, which is about the lifetimes of values, and how the implementations check it.

5.6. The standard functions

The standard library provides the usual mathematical functions for **`real`** arguments: `sqrt`, `exp`, `ln` (the natural logarithm), `sin`, `cos`, `tan`, `arcsin`, `arccos` and `arctan`, together with the constant `pi`. The parameterless function `random` returns a pseudo-random **`real`** between 0 and 1. Chapter 6 describes the library in more detail.

5.7. Defining operators

An operator is declared like a function, with **`op`** instead of **`proc`**. A binary operator has two parameters, a unary one has one. The name of an operator is either a word in capitals, such as `MAX`, or a symbol made of characters such as `+ - * / < > =` and a few others. A symbol has one or two characters, and the second must be one of `< > / = *`; either may be followed by `:=` or `=:`. A unary operator may not begin with one of these five. Thus `//` and `+:=` can be declared, but `++` cannot ("invalid operator tag"):

```
prio MAX = 9;
op MAX = (int a, b) int: (a > b | a | b);
op MAX = (real a, b) real: (a > b | a | b);
op SQUARE = (int n) int: n * n;
print ((3 MAX 9, " ", fixed (2.5 MAX 1.5, 0, 1), newline));
print ((SQUARE 4 + 1, " ", 2 * 5 MAX 6, newline))
```

```
         +9 2.5
        +17         +12
```

A binary operator needs a precedence, declared with **prio** (priority declaration); without it, the compiler reports an error. Here MAX gets the precedence 9, higher than that of *, so 2 * 5 MAX 6 means 2 * (5 MAX 6). A unary operator needs no precedence: like the standard unary operators, it binds more tightly than every binary operator, so SQUARE 4 + 1 is 17.

The same operator can be declared several times for different operand types; the example declares MAX for two **int** and for two **real** operands. This is *overloading*. The standard operators are declared in the same way in the standard library, and a program can add new meanings to them: + for vectors, * for matrices, < for dates.

The declarations of one operator in the same block must differ enough that the compiler can always tell them apart, even after the implicit conversions allowed for operands. Two declarations clash if every operand could be converted, by the conversions allowed for operands, to the parameter types of both; for a binary operator this must hold for both operands. Declarations for (**int**, **real**) and (**real**, **int**) can therefore coexist. The Report calls clashing types firmly related. In practice the operand types must really differ, not merely in whether an operand is a reference, and a union is related to each of its members (Section 7.18); the compiler reports a violation as "firmly related".

An operator can also be defined for arrays. The result of the following + is a new array, which is copied out of the function like any other value:

```
op + = ([] real a, b) [] real:
begin
   if LWB a /= LWB b orel UPB a /= UPB b then
      print (("bounds differ", newline)); stop
   fi;
   [LWB a : UPB a] real sum;
   for i from LWB a to UPB a do sum[i] := a[i] + b[i] od;
   sum
end;
[] real v = (1.5, 2.5, 3.5);
[] real w = v + [] real (1, 1, 1);
for i to UPB w do print ((fixed (w[i], 0, 1), " ")) od;
print (newline)
```

```
2.5 3.5 4.5
```

The second operand is an array literal, which must be annotated with its type: a literal is allowed only where its type is known, and the operand of an operator is not such a place (Section 7.12).

Since type names are only abbreviations, **mode** Meters = **real** does not create a type of its own, and an operator for Meters would be an operator for **real**. A record with a single field does create a distinct type, the "newtype" of current languages. The record types must have different field names, or they would be the same type:

```
mode Meters = struct (real m), Seconds = struct (real s);
proc meters = (real x) Meters: (Meters r; m of r := x; r);
proc seconds = (real x) Seconds: (Seconds r; s of r := x; r);
op + = (Meters a, b) Meters: meters (m of a + m of b);
op / = (Meters d, Seconds t) real: m of d / s of t;
Meters distance = meters (100) + meters (300);
```

```
Seconds time = seconds (50);
print ((fixed (distance / time, 0, 1), " m/s", newline))
```

```
8.0 m/s
```

A record literal needs at least two fields, so the functions `meters` and `seconds` build the values.

5.8. How an operator is chosen

When the compiler meets an operator, it chooses the declaration by the types of the operands. Operands are in a firm context (Section 2.10): a variable is dereferenced and a parameterless function is called, but an **int** is not widened to a **real**. An operator declared only for **real** operands therefore does not accept an **int**:

```
access Genie_Transput
begin
   op HALF = (real x) real: x / 2;
   print ((HALF 3, newline))
end
```

```
no-widening-op.a68:4:12: error: monadic operator HALF int has not been declared
```

The standard operators avoid this problem by declaring every combination of **int** and **real** operands.

The precedence belongs to the symbol, not to one of its declarations: all the meanings of `+` in a block have the same precedence, and a **prio** declaration changes it for all of them, the standard ones included:

```
print ((3 + 4 * 7, newline));
begin
   prio + = 7;              { now + binds as tightly as * }
   print ((3 + 4 * 7, newline))
end
```

```
        +31
        +49
```

prints 31 and 49. Inside the inner block, `3 + 4 * 7` is `(3 + 4) * 7`. A program that wants the same convenience must do the same, or call the operator with a **real** operand. The Report calls the choice of the declaration the identification of the operator; current languages call it *overload resolution*.

5.9. A complete program

The final program of this chapter defines a type of exact fractions and the arithmetic operators for it. The function `rat` reduces a fraction to lowest terms with the greatest common divisor and makes the denominator positive. A new operator `//` builds a fraction from two

integers; it is given the precedence of `/`. The unary operator `STR` converts a fraction into a string.

```
access Genie_Transput
begin
   { Exact arithmetic with fractions. }
   mode Rat = struct (int num, den);
   proc gcd = (int a, b) int: (b = 0 | ABS a | gcd (b, a MOD b));
   proc rat = (int n, d) Rat:
   begin
      int g = gcd (n, d) * SIGN d;
      (n % g, d % g)
   end;
   op + = (Rat a, b) Rat:
      rat (num of a * den of b + num of b * den of a, den of a * den of b);
   op - = (Rat a, b) Rat:
      rat (num of a * den of b - num of b * den of a, den of a * den of b);
   op * = (Rat a, b) Rat: rat (num of a * num of b, den of a * den of b);
   op / = (Rat a, b) Rat: rat (num of a * den of b, den of a * num of b);
   op - = (Rat a) Rat: (-num of a, den of a);
   op +:= = (ref Rat a, Rat b) ref Rat: a := a + b;
   op STR = (Rat a) string:
      whole (num of a, 0) + (den of a = 1 | "" | "/" + whole (den of a, 0));
   prio // = 7;
   op // = (int n, d) Rat: rat (n, d);
   Rat sum := 0 // 1;
   for k to 6 do sum +:= 1 // k od;
   print (("1 + 1/2 + ... + 1/6 = ", STR sum, newline));
   print ((STR (3 // 4 * (2 // 3 - 1 // 6)), " ", STR - (6 // 8), " ",
           STR (1 // 2 / (1 // 4)), newline))
end
```

```
1 + 1/2 + ... + 1/6 = 49/20
3/8 -3/4 2
```

Note how the standard symbols `+`, `-`, `*` and `/` get a new meaning for the type `Rat` without losing their meaning for numbers, and how the unary and binary `-` are declared separately. Defining `+` does not define `+:=`: the program declares it separately, with a reference to the variable as its left operand and the variable as its result, like the standard `+:=`. The expression `STR - (6 // 8)` applies two unary operators, the minus first.

Exercises

The answers are at the end of the book.

1. What is the type of each of these functions? (a) `proc f = (real x, int n) real: x ** n` (b) `proc g = bool: random < 0.5` (c) `proc h = (ref [] int a) void: a[1] := 0`

2. Write a function `digit_sum` that returns the sum of the decimal digits of a non-negative `int`, once with a loop and once recursively.

3. Why does `proc inc = (int n) void: n +:= 1` not compile, and how must it be written to increase the caller's variable?

4. Write a function `apply_twice` that takes a function of type `proc (int) int` and an `int` and applies the function twice. Call it with a function literal that doubles its argument.

5. Declare an operator `ABOVE` so that `a ABOVE b` is `true` if `a` is greater than `b`, for `int` operands, with the precedence of `>`. What is the value of `1 + 2 ABOVE 2`?

6. With the operators of this chapter, what is `SQUARE 2 MAX 3`?

7. Why does the assignment `p := hello` not call the function `hello`, although its value is discarded?

8. In a block that contains `prio + = 7`, what is the value of `1 + 2 * 3`?

9. May `op MAX = (int a, real b) real` and `op MAX = (real a, int b) real` be declared in the same block?

10. Write an operator `*` that multiplies a `[] real` by a `real` and returns a new array with the same bounds.

11. A function `month = (ref int day) string` takes a day of the year, returns the name of its month and changes `day` to the day within that month. Why is `print ((month (d), " ", d))` wrong, and how is it corrected?

12. Write a function `sum = (int lo, hi, proc (int) real term) real` and use it to compute the sum of $1/k^2$ for k from 1 to 100 and the scalar product of two arrays `v` and `w`. Why can the second call not simply pass `v[k] * w[k]`?

Programming problems

1. Write a function that returns the n-th prime number, and use it to print the first 20 primes.

2. Solve the Towers of Hanoi recursively and print the moves for four discs.

3. Write a function that finds a root of a function `proc (real) real` in an interval by bisection, and use it to compute the cube root of 2.

4. Define a type `Vector` with three `real` fields and the operators `+`, `-`, `*` (scalar product) and a unary `ABS` (length).

5. Extend the rational numbers of this chapter with the comparison operators `=` and `<`, and use them to sort an array of fractions.

6. **Merge sort.** Sort an array by recursive merge sort, passing the halves to the recursive calls as slices of a `ref [] int`.

7. **Permutations.** Print all permutations of the numbers 1 to n, and all ways of choosing k of them, with a recursive procedure that fills an array position by position.

8. **Plotting.** Write a procedure that takes a function `proc (real) real`, an interval and a height, and plots the function with characters. Plot `sin` and a function literal.

9. **Horner's rule.** Write a function that evaluates a polynomial whose coefficients are given in an array with any lower bound, and count the multiplications compared with computing each power separately.

10. **The 91 function.** McCarthy's function is $f(n) = n - 10$ for $n > 100$ and $f(f(n+11))$ otherwise. Compute it recursively for n from 1 to 110, and find a simple formula for its values.

11. **Monte Carlo.** Estimate π by choosing random points in a square with `random` and counting those inside the circle, for 10^3, 10^4 and 10^5 points.

12. **Matrices.** Declare `+`, `*` and `*:=` for `[,] real`, checking that the bounds fit.

13. **Resistors.** Declare an operator `PAR` that combines two resistances in parallel, $ab/(a+b)$, choose a precedence for it relative to `+` (resistances in series), and add an assigning form `PARAB` in the style of `+:=`. Compute a network of several resistors with one formula.

14. **A maze.** Describe a maze by a function `blocked = (int x, y) bool` and find a way from the entrance to the exit with a recursive procedure that tries each direction, marks the cells it has visited, and backs out of dead ends. Print the path as the recursion returns, and explain why it comes out in reverse order.

15. **An evaluator.** Evaluate integer expressions with `+ - * /` and parentheses, given as a string, with mutually recursive functions for expressions, terms and factors that share a position variable.

6. More built-in types and the standard library

Chapters 2 to 5 used four primitive types. This chapter introduces the remaining built-in types of the language: complex numbers, numbers of greater or smaller precision, bit sets and byte strings. It closes with a survey of the constants and functions of the standard library. Much of the chapter depends on the implementation, since the precision of numbers does; where it does, the text says what ga68, a68j and Algol 68 Genie provide.

6.1. Complex numbers

The type **compl** holds a complex number, a pair of **real** values for the real and the imaginary part. The binary operator `I` builds a complex number from its two parts: `1 I 2` is $1 + 2i$. The arithmetic operators `+`, `-`, `*` and `/` work on complex numbers, also mixed with **int** and **real** operands, and `=` and `/=` compare them. Further operators give the parts and properties of a complex number:

Operator	Result
`RE z`, `IM z`	the real and the imaginary part
`ABS z`	the absolute value $\|z\|$
`ARG z`	the argument (angle), between $-\pi$ and π
`CONJ z`	the complex conjugate
`a I b`	the complex number $a + bi$

```
proc show = (compl z) string:
   fixed (RE z, 0, 3) + (IM z < 0 | " - " | " + ")
      + fixed (ABS IM z, 0, 3) + "i";
compl z = 1 I 2, w = 3 I -1;
print ((show (z + w), "   ", show (z * w), "   ", show (z / w), newline));
print ((fixed (ABS (3 I 4), 0, 3), " ", fixed (ARG (0 I 1), 0, 4), " ",
        show (CONJ z), newline));
compl r := 2.5;                 { a real is widened to compl }
r *:= 0 I 1;
print ((show (r), newline))
```

```
4.000 + 1.000i   5.000 + 5.000i   .100 + .700i
5.000 1.5708 1.000 - 2.000i
.000 + 2.500i
```

The function `show` formats a complex number for printing. A **`real`** (and therefore an **`int`**) is widened to **`compl`** where a complex number is required, so **`compl`** `r := 2.5` is allowed; this is the second step of the widening that Chapter 2 introduced.

The standard library declares **`compl`** as a record, **`mode compl = struct (real re, im)`**, so `re` **`of`** `z` selects the real part of a complex variable and can be assigned to, and **`compl`** `z = (1, 2)` is a record literal. Two details catch the unwary: `I` has the highest precedence, 9, so `2 * x I y` means `2 * (x I y)`; and complex numbers have no order, so there is no `<`.

> **Implementation note.** With ga68 a program that uses the complex operators must be linked with the static libga68, which is the default: the shared library (`-shared-libga68`) does not export them, and the program fails when it is linked. `Genie_Transput` does not print complex numbers directly, which is why the example formats them itself; the `print` of the Report and of Algol 68 Genie prints the two parts.

As an application, the following program solves quadratic equations, whose roots may be complex. The discriminant decides whether the square root is real or imaginary; the **`if`** balances its two branches to **`compl`**.

```
access Genie_Transput
begin
   { The roots of a x^2 + b x + c, real or complex. }
   proc show = (compl z) string:
      fixed (RE z, 0, 3) + (IM z < 0 | " - " | " + ")
         + fixed (ABS IM z, 0, 3) + "i";
   proc roots = (real a, b, c) void:
   begin
      real d = b * b - 4 * a * c;
      compl root = (d >= 0 | sqrt (d) | 0 I sqrt (-d));
      compl x1 = (-b + root) / (2 * a), x2 = (-b - root) / (2 * a);
      print ((show (x1), "   ", show (x2), newline))
   end;
   roots (1, -3, 2);
   roots (1, 2, 5);
   roots (2, 0, 8)
end
```

```
2.000 + .000i   1.000 + .000i
-1.000 + 2.000i   -1.000 - 2.000i
.000 + 2.000i   .000 - 2.000i
```

6.2. Precision variants

The types **`int`**, **`real`**, **`compl`** and **`bits`** come in several sizes. The prefix **`long`** gives a variant with more bits, **`long long`** one with still more, and **`short`** and **`short short`** variants with fewer. How many variants there are and how large they are depends on the implementation. A variant that the implementation does not distinguish has the same values as the next larger or smaller one, but it is still a type of its own: a **`long long int`** must be converted with `SHORTEN` to become a **`long int`**, even where both have 64 bits.

Type	ga68	a68j	Algol 68 Genie
short short int	8 bits	8 bits	64 bits
short int	16 bits	16 bits	64 bits
int	32 bits	32 bits	64 bits
long int	64 bits	64 bits	128 bits
long long int	64 bits	64 bits	72 decimal digits
real	32 bits	32 bits	64 bits
long real	64 bits	64 bits	128 bits
long long real	80 bits	64 bits	63 decimal digits
bits	32	32	64
long bits	64	64	128

A literal of a long type is written with the prefix: **long** 1, **long** 2.5, **long long** 0. The operator LENG converts a value to the next longer type and SHORTEN to the next shorter one; there is no implicit conversion between the sizes, so a **long int** and an **int** cannot be added without LENG. The names of the standard constants and functions for the long types begin with long_: long_max_int, long_pi, long_sqrt.

```
long int big = long 123456789 * long 1000000;
print ((big, newline));
print ((long_max_int, newline));
long real third = long 1 / long 3;
print ((third, newline));
print ((LENG max_int + long 1, newline));       { no overflow in long }
print ((SHORTEN (long 2 ** 20), newline))
```

```
    +123456789000000
+9223372036854775807
+3.33333333333333e  -1
         +2147483648
   +1048576
```

The last two lines show the conversions: LENG max_int + **long** 1 computes in **long int** and so does not overflow, and SHORTEN converts a value that fits back to **int**.

The long types are what a program needs for exact large integers and for precise floating-point computations. With 32-bit integers, 13! already overflows; with **long int**, all factorials up to 20! are exact:

```
access Genie_Transput
begin
   { Factorials with long int, and their numbers of digits. }
   long int f := long 1;
   for n to 20 do
      f *:= LENG n;
      if n MOD 5 = 0 then
         string digits = whole (f, 0);
         print ((whole (n, -2), "! = ", whole (f, -20),
                 "  (", whole (UPB digits, 0), " digits)", newline))
```

```
      fi
    od
end
```

```
 5! =                  120  (3 digits)
10! =              3628800  (7 digits)
15! =        1307674368000  (13 digits)
20! =  2432902008176640000  (19 digits)
```

Implementation note. `Genie_Transput` prints **`long int`** and **`long real`** values, but not the **`short`** and **`long long`** variants; they are converted with `LENG` or `SHORTEN` before printing. In Algol 68 Genie, **`long long int`** and **`long long real`** are computed in software with a very large precision, which Rosetta Code programs use for big numbers.

6.3. Bit sets

A value of type **`bits`** is a fixed-size sequence of bits, as many as the constant `bits_width` says. It is the Algol 68 equivalent of an unsigned integer used as a set of flags or a bit mask. Literals are written in base 2, 4, 8 or 16 with a prefix that gives the base: `2r1100`, `8r14` and `16rc` are the same value. The operators on **`bits`** are:

Operator	Result
`a AND b`, `a OR b`, `NOT a`	bitwise and, or, complement
`a SHL n`, `a SHR n`	shift left or right by `n` bits
`n ELEM a`	bit `n` of `a`, as a **`bool`**
`a = b`, `a /= b`	equality
`a <= b`, `a >= b`	subset, superset (not numerical order)
`ABS a`, `BIN n`	conversion to and from **`int`**

```
bits a = 2r1100, b = 16r0a;                 { 12 and 10 }
print ((ABS (a AND b), ABS (a OR b), ABS (a SHL 2), ABS (a SHR 2), newline));
print ((a, newline));
print ((bits_width, " ", bits_width ELEM a, (bits_width - 2) ELEM a, newline));
bits c = BIN 37;                            { from int to bits }
print ((ABS c, " ", ABS bits_pack ((true, true, false)), newline))
```

```
         +8        +14        +48         +3
FFFFFFFFFFFFFFFFFFFFFFFFFFFFTTFF
        +32 FT
        +37          +6
```

Two details differ from current conventions. First, `print` writes a **`bits`** value as a row of `T` and `F` characters, one per bit. Second, the bits are numbered from the left: bit 1 is the most significant bit and bit `bits_width` the least significant one. In `2r1100`, the two set bits are therefore numbers 29 and 30 of 32. The function `bits_pack` builds a **`bits`** value from an array of **`bool`**, whose last element becomes the least significant bit.

Pitfall. The comparisons of **bits** values are set comparisons: `2r100 >= 2r011` is false, since the first value does not contain the bits of the second. For numerical order, compare `ABS a` and `ABS b`. `ELEM` has the precedence 7, so `i + 1 ELEM b` means `i + (1 ELEM b)` and is rejected; write `(i + 1) ELEM b`. A negative shift count shifts in the other direction. And `ABS` of a value whose first bit is set does not fit into an **int** of the same width: the Report leaves the result undefined, and a68j gives a negative number.

Implementation note. ga68, a68j and Algol 68 Genie also provide `XOR` for **bits**, which the Report does not have; it can be written as `(a OR b) AND NOT (a AND b)`.

6.4. Byte strings

The type **bytes** is a string of fixed length, `bytes_width` characters, stored compactly. It was meant for machines on which a **string** was expensive. The function `bytes_pack` converts a string to **bytes**, and `n ELEM b` selects a character.

```
bytes code = bytes_pack ("ALGOL");
print ((2 ELEM code, " ", 5 ELEM code, newline))
```

```
L L
```

Implementation note. ga68 and a68j do not yet support **bytes**; the example was run with Algol 68 Genie. Programs rarely need this type today, since **string** serves the same purpose.

6.5. Environment enquiries

The standard library contains constants that describe the implementation, which the Report calls environment enquiries. A program that wants to be portable uses them instead of assuming particular sizes:

```
print (("max_int        ", max_int, newline));
print (("long_max_int   ", long_max_int, newline));
print (("small_real     ", small_real, newline));
print (("long_small_real", long_small_real, newline));
print (("bits_width     ", bits_width, newline));
print (("int_lengths    ", int_lengths, newline));
print (("int_shorths    ", int_shorths, newline))
```

```
max_int        +2147483647
long_max_int   +9223372036854775807
small_real     +1.192093e -7
long_small_real+2.22044604925031e -16
bits_width             +32
int_lengths             +2
int_shorths             +3
```

The most important ones are:

Constant	Meaning
`max_int`, `long_max_int`	the largest integer of each size
`max_real`, `long_max_real`	the largest real of each size
`small_real`	the smallest **`real`** x with $1 + x > 1$ and $1 - x < 1$
`int_lengths`, `int_shorths`	1 + the number of **`long`**, **`short`** sizes of **`int`**
`real_lengths`, `real_shorths`	the same for **`real`**
`bits_width`, `bytes_width`	the number of bits in **`bits`**, characters in **`bytes`**
`int_width`, `real_width`	the digits needed to print an **`int`**, a **`real`**
`max_abs_char`	the largest character code
`blank`, `null_character`	the space character, a filler character (code 0 here)
`flip`, `flop`	the characters for true and false (`"T"` and `"F"` here)

`int_lengths` and `int_shorths` count **`int`** itself, so `int_lengths = 2` means that there is one distinct **`long`** size (and `int_shorths = 2` one **`short`** size); a larger prefix gives a type with the same values as the largest distinct one.

6.6. Mathematical functions

The standard library provides the mathematical functions of Section 5.6, each also in a version for each long size: `sqrt`, `exp`, `ln`, `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`; for **`long real`**, they are `long_sqrt`, `long_exp`, and so on. The function `random` returns a pseudo-random **`real`** between 0 and 1.

```
for i from 0 to 3 do
   real x = i * pi / 8;
   print ((fixed (x, -6, 4), fixed (sin (x), -8, 4), fixed (cos (x), -8, 4),
           fixed (exp (x), -8, 4), newline))
od;
print ((fixed (long_sqrt (long 2), 0, 15), newline))
```

```
0.0000  0.0000  1.0000  1.0000
0.3927  0.3827  0.9239  1.4810
0.7854  0.7071  0.7071  2.1933
1.1781  0.9239  0.3827  3.2482
1.414213562373095
```

The negative widths ask `fixed` for a field of six or eight characters without a plus sign (Section 2.14).

6.7. Extending the standard operators

A program can declare a standard operator for operand types that the library does not cover, as Chapter 5 did for fractions. The standard `MOD`, for example, is defined for integers only; a declaration adds a version for **`real`** operands, while the standard `MOD` keeps its meaning:

```
op MOD = (real a, b) real: a - b * ENTIER (a / b);
print ((fixed (7.5 MOD 2.0, 0, 2), " ", fixed (-7.5 MOD 2.0, 0, 2), newline));
```

```
print ((7 MOD 2, newline))                    { the standard MOD }
```

```
1.50 .50
         +1
```

Exercises

The answers are at the end of the book.

1. What are `RE ((1 I 1) * (1 I -1))` and `ABS (0 I 2)`?
2. Why is **`long int`** `n := 5` not allowed, and how is it written correctly?
3. Assuming `bits_width` is 32, what are `ABS (16rff AND 2r1010)`, `ABS (2r1 SHL 4)` and `32 ELEM 2r1`?
4. Write a function `popcount` that counts the set bits in a **`bits`** value, using `ELEM`.
5. How can a program find out whether **`long long int`** is longer than **`long int`** in the implementation it runs on?
6. Is `2r100 >= 2r011` true? Why or why not?
7. What does `2 * 3 I 4` mean?

Programming problems

1. Compute 2^{62} and 3^{39} exactly with **`long int`**.
2. Print the first 90 Fibonacci numbers with **`long int`**.
3. Use the Newton iteration $z \leftarrow z - (z^3 - 1)/(3z^2)$ to find which of the three cube roots of 1 is reached from each starting point of a grid in the complex plane, and print the grid as characters.
4. Represent a set of the numbers 1 to 32 as **`bits`**, and write the operations union, intersection, membership and size.
5. **Mandelbrot.** Print the Mandelbrot set with characters, using **`compl`** arithmetic.
6. **A bit sieve.** Write the sieve of Eratosthenes on an array of **`bits`** words, splitting a number into a word index and a bit number, and compare its size with the **`[] bool`** version of Chapter 4.
7. **Gray code.** Convert numbers to their Gray code (`n XOR (n SHR 1)` on **`bits`**) and back, and print the codes of 0 to 15.
8. **An adder.** Add two positive numbers given as **`bits`** bit by bit, as hardware does, with a carry, and report an overflow.

9. **Machine epsilon.** Find the smallest power of 2 that, added to 1, gives a result different from 1, by halving, for **real** and **long real**, and compare it with small_real.

10. **Exact factorials.** Compute 50! exactly with an array of **int** digits in base 10000, and compare the result with **long long int** in Algol 68 Genie.

11. **Safe fractions.** Make the rational numbers of Chapter 5 safe from overflow: compute the products in **long int** with LENG, reduce with a **long** greatest common divisor, and convert back with SHORTEN only after checking against LENG max_int. Use them for the first terms of a power series with exact coefficients, such as that of e^x.

7. Advanced types

The types of the previous chapters were built in simple ways: arrays of numbers, records of strings and numbers, functions of numbers. This chapter shows the full power of the type system. It introduces arrays that change their size, unions of types, references to references, and recursive types such as lists and trees, and it combines them in larger examples. It then collects the rules that earlier chapters introduced piece by piece: all implicit conversions and the places where each is allowed, and the rules that govern how long a value lives.

This is the chapter where Algol 68 differs most from the languages that most programmers know. Many of its rules are easy to state and have consequences that are not obvious, so the chapter shows those consequences, including the ones that the compilers do not report.

The guiding principle behind all of it is *orthogonality* (Section 7.13): every way of building a type can be applied to every type, and every construct works the same way whatever the types involved.

7.1. Resizable arrays

The arrays of Chapter 4 have fixed bounds. A variable declared with **flex** before the bounds can hold arrays of any size; each assignment gives it the bounds of the assigned array:

```
flex [1:0] int list;                        { empty at first }
print ((UPB list, newline));
list := (3, 1, 4);
print ((UPB list, ":", list, newline));
list := list[2:3];                          { shrinks to two elements }
print ((UPB list, ":", list, newline));
[] int snapshot = list;                     { a fixed copy }
list := (1, 5, 9, 2, 6);
print ((UPB snapshot, UPB list, newline))
```

```
         +0
         +3:         +3         +1         +4
         +2:         +1         +4
         +2         +5
```

flex `[1:0]` **int** `list` declares a variable that holds an array of **int**, initially empty. The bounds in the declaration only determine the initial array. The Report calls such a variable a flexible name.

The value of a resizable array is an ordinary array: `snapshot` is a copy with the bounds 1 to 2, which does not change when `list` changes. The type of such a value is the array type without **flex**; the Report says that the type deflexes to it.

There is no operation that adds an element to a resizable array in place. An array literal cannot help either: each element of the literal must be an **int**, and `squares` is an array, so

the compiler rejects `(squares, i * i)`:

```
flex [1:0] int squares;
for i to 6 do
   squares := (squares, i * i)    { not allowed: see the text }
od
```

```
append.a68:6:7: error: ref flex [] int cannot be coerced to 'int' in a strong-unit
```

Instead, a new, longer array is built and assigned. A procedure does this once and for all:

```
proc append = (ref flex [] int list, int value) void:
begin
   [1 : UPB list + 1] int longer;
   longer[: UPB list] := list;
   longer[UPB list + 1] := value;
   list := longer
end;
flex [1:0] int squares;
for i to 6 do append (squares, i * i) od;
print ((squares, newline))
```

```
         +1         +4         +9        +16        +25        +36
```

The parameter has the type **ref flex [] int**: a reference to a resizable array variable, the only kind of variable whose bounds may change. A **ref [] int** parameter would accept fixed arrays as well, and could therefore not be given a longer array. In the Report, the two are even different types: a **ref flex [] int** cannot be passed as a **ref [] int**. This is the price of Algol 68's value semantics for arrays: what other languages call a dynamic array or list (`std::vector`, `ArrayList`, a Python `list`) is here written out explicitly.

> **Implementation note.** ga68, a68j and Algol 68 Genie accept a **ref flex [] int** argument for a **ref [] int** parameter, contrary to the Report. A program that relies on this is not portable.

7.2. Where flex can stand

flex is a property of variables, not of values: an array value always has fixed bounds, and only a variable can be given a new array of a different size. **flex** therefore belongs in the type of a variable, that is, under a **ref**. It can appear in a variable declaration, as above, in the type **ref flex [] int** of a parameter, and in a type declaration that is used to declare variables:

```
mode Buffer = flex [1:0] int;
Buffer b := (1, 2, 3);
b := (4, 5)
```

The standard type **string** is exactly such a declaration (Section 7.4). A type declaration needs bounds, even if they are only initial ones: **mode B = flex [] int** is rejected with "actual bounds expected". A constant cannot be resizable, so **flex [] int b = (1, 2, 3)** is not allowed by the Report; the implementations accept it and treat `b` as a **[] int**.

Deflexing also applies to the fields of a record. A variable of type `struct (string name, int age)` holds records whose `name` can have any length, but the record value that the variable holds has a field of type `[] char`, with the bounds of its current name.

Jagged arrays

A two-dimensional array `[,] int` is rectangular: all its rows have the same length. An array of arrays, `[] [] int`, is a different type, whose elements are arrays in their own right and may have different lengths, like a Java `int[][]`. With `flex` at both levels, both the number of rows and each row can change:

```
[] [] int jag = ((1, 2), 3, (4, 5, 6));
for i to UPB jag do
   print ((whole (UPB jag[i], 0), " elements:"));
   for j to UPB jag[i] do print ((" ", whole (jag[i][j], 0))) od;
   print (newline)
od;
flex [1:0] flex [1:0] int rag := ((1, 2), 3, (4, 5, 6));
rag[2] := (7, 8, 9, 10);       { one row gets longer }
rag := rag[1:2];               { the array loses its last row }
print ((whole (UPB rag, 0), " rows, row 2 has ",
        whole (UPB rag[2], 0), " elements", newline))
```

```
2 elements: 1 2
1 elements: 3
3 elements: 4 5 6
2 rows, row 2 has 4 elements
```

In the literal `((1, 2), 3, (4, 5, 6))`, the element `3` is converted to an array with one element (rowing, Section 4.10). The assignment to `rag[2]` gives the second row a new length, and `rag := rag[1:2]` drops the last row. In between lie the type `[3] flex [1:0] int`, three rows of which each can change, and `flex [1:0] [3] int`, a varying number of rows of fixed length 3.

> **Implementation note.** ga68 stops with an internal compiler error when a value of type `[] [] int` that is not a literal is assigned to a variable of type `flex [] flex [] int`, as in `rag := jag`. Assigning a literal, as in the example, works.

7.3. Growing an array efficiently

The procedure `append` of Section 7.1 copies the whole array each time it adds an element. Building an array of n elements this way copies about $n^2/2$ elements, which is slow for large n. The dynamic arrays of other languages avoid this by keeping some spare room: the array is larger than the number of elements in use, and when it is full, it is replaced by one twice as large. Each element is then copied only a few times on average.

In Algol 68 the same structure is a record with a resizable array and a count:

```
mode Vector = struct (flex [1:0] int data, int size);
proc push = (ref Vector v, int x) void:
```

```
begin
   if size of v = UPB data of v then
      { full: move the elements into an array twice as large }
      [(UPB data of v = 0 | 4 | 2 * UPB data of v)] int bigger;
      bigger[1 : size of v] := (data of v)[1 : size of v];
      data of v := bigger
   fi;
   (data of v)[size of v +:= 1] := x
end;
Vector v := ((), 0);
for i to 10 do
   push (v, i * i);
   print ((whole (size of v, 3), whole (UPB data of v, 4), newline))
od
```

The program prints the number of elements and the capacity after each `push`:

```
 +1  +4
 +2  +4
 +3  +4
 +4  +4
 +5  +8
 +6  +8
 +7  +8
 +8  +8
 +9 +16
+10 +16
```

The field `data` of a `Vector` variable is itself a resizable array variable: if `v` has the type `ref Vector`, then `data of v` has the type `ref flex [] int` and can be assigned to, indexed, or passed to `append` of Section 7.1. The parentheses in `(data of v)[i]` are needed because indexing binds more tightly than selection (Section 4.9).

The same cost applies to strings: `s +:= c` in a loop builds a new string each time. For long texts, collecting the characters in a buffer of this kind and converting them at the end is much faster.

7.4. Strings

The type `string` is defined in the standard library as `flex [1:0] char`, a resizable array of characters. That is why a `string` variable can hold strings of any length. The library adds a few operators:

```
string s := "Algol";
s +:= " 68";                        { append }
"The " +=: s;                       { prepend }
print ((s, ", length ", whole (UPB s, 0), newline));
print ((3 * "ab", " ", "-" * 10, newline));
string letters := "";
for i to 5 do letters +:= REPR (ABS "a" + i - 1) od;
print ((letters, " ", "abc" < "abd", " ", "abc" = "ab" + "c", newline))
```

```
The Algol 68, length 12
ababab ----------
abcde T T
```

`+` joins strings and characters; `s +:= t` appends `t` to `s`, and `t +=: s` puts `t` in front of `s`. `*` repeats a string a given number of times, and `s *:= n` replaces `s` by `n` copies of itself. The comparisons `=`, `/=`, `<`, `<=`, `>`, `>=` compare strings in dictionary order, by the codes of their characters. They are defined for `[] char`, so they apply to every array of characters, not only to `string` variables.

7.5. References into resizable arrays

An element of an array variable is itself a variable: `a[1]` is a reference to the first element. For a resizable array, such a reference must not be kept, since the next assignment to the array may give it a different size and place the elements elsewhere; the reference would then refer to a cell that no longer belongs to the array. The language therefore forbids binding such a reference to a name or storing it:

```
access Genie_Transput
begin
   flex [1:3] int a := (1, 2, 3);
   ref int first = a[1];
   first := 10;
   print ((a, newline))
end
```

```
transient.a68:4:20: error: attempt at storing a transient name
```

The Report calls a reference into a resizable array a transient name. The same problem is known in C++ as iterator invalidation; Algol 68 rules it out at compile time.

The rule is stricter than it looks. A transient reference may be used only directly, for an assignment or to obtain its value. It may not be bound to a name, stored, returned, or passed to a procedure or an operator, since a parameter is bound like a constant declaration and no type can be written for a transient reference. That excludes some everyday code:

```
flex [1:10] int count;
for i to 10 do count[i] := 0 od;
count[3] +:= 1
```

```
transient-count.a68:6:4: error: attempt at storing a transient name
```

`+:=` is an operator whose left operand is a reference, so `count[3] +:= 1` passes a transient reference to it. For the same reason `read (count[3])` and `swap (count[i], count[j])` are rejected, and so is `s[i] +:= "x"` for an element of a `flex [] string`. Note that `string` is a resizable array, so the rule applies to the characters of every `string` variable. Selecting a field across all elements, `x of path` for a `flex [] Point` variable `path`, gives a transient reference as well.

There are three ways around it. The plain assignment `count[3] := count[3] + 1` is allowed. If the size is known once the array has been filled, a fixed array can take over: `[UPB count] int c := count` is an ordinary array whose elements may be passed anywhere. And the

restriction applies only to resizable arrays: the elements of fixed arrays and the fields of records are ordinary variables.

7.6. Sharing arrays

Arrays are values in Algol 68: assigning an array copies its elements (Section 4.3). An array is shared only when a reference to it is shared explicitly, which the type always shows:

```
[5] int a := (1, 2, 3, 4, 5);
ref [] int r = a;            { another name for a }
ref [] int s := a[2:3];      { a pointer to a slice of a }
r[1] := 10;
s[1] := 20;
int x := 7;
ref [] int one = x;          { x seen as an array of one element }
one[1] := 8;
print ((a, newline, x, newline))
```

prints

```
        +10        +20         +3         +4         +5
         +8
```

`r` is a constant of type **`ref [] int`** bound to the variable `a`: another name for the same array, so `r[1] := 10` changes `a`. `s` is a variable that holds a reference to a part of `a`; `s[1]` is `a[2]`. And `one` shows the rowing of a variable: the reference `x` becomes a reference to an array with one element, which is `x` itself.

The same distinction exists for single values. **`ref int`** `r = x` makes `r` another name for `x`, like a reference **`int`**`& r = x` in C++. **`ref int`** `p := x` creates a variable that holds a reference to `x` and can later refer to something else, like a pointer **`int`**`* p = &x` in C. In Java and Python, objects and lists are always shared and a copy must be made explicitly; in Algol 68, as in C++ (and, with moves instead of copies, in Rust), values are copied and sharing must be made explicit.

7.7. Unions

A *union* type holds a value of one of several types, together with an indication of which type it is. **`union (int, real, string)`** holds an **`int`**, a **`real`** or a **`string`**. A value of any of these types can be assigned to a union variable or passed as a union parameter; this implicit conversion is *uniting* (uniting).

To use the value inside a union, the program must find out its type. This is done with a special form of **`case`**, which the Report calls a conformity clause:

```
mode Value = union (int, real, string);
proc describe = (Value v) string:
   case v in
      (int n):    "the int " + whole (n, 0),
      (real x):   "the real " + fixed (x, 0, 2),
      (string s): "the string """ + s + """"
```

```
   esac;
[] Value items = (42, 2.5, "hello");
for i to UPB items do print ((describe (items[i]), newline)) od;
Value v := 7;
v := "seven";                          { now it holds a string }
print ((describe (v), newline))
```

```
the int 42
the real 2.50
the string "hello"
the string "seven"
```

Each alternative names a type and, optionally, a name for the value: `(int n): ...` is chosen if the union holds an **int**, and binds `n` to it. An **out** part handles the types that no alternative names. A union without a name in its alternative only tests the type:

```
union (int, string) u := "text";
bool is_int = case u in (int): true out false esac;
print ((is_int, newline));
u := 5;
case u in
   (int n): print ((n * 2, newline))
out
   print (("not an int", newline))
esac
```

```
F
        +10
```

The union is a *tagged union*, or *sum type*: the type of the current value is recorded with it and checked at run time, so a value can never be read as the wrong type. It corresponds to the `enum` of Rust and Swift, the variant types of ML and Haskell and `std::variant` of C++, and the conformity clause corresponds to their pattern matching. Unlike those, an Algol 68 union has no names for its alternatives; the alternatives are distinguished by their types only. Two alternatives that carry the same type of value must therefore be wrapped in different record types.

The order of the types in a union does not matter, and a union that contains a union contains the members of the inner one: **union** (**int**, **real**) and **union** (**real**, **int**) are the same type, and **union** (**int**, **union** (**real**, **string**)) is **union** (**int**, **real**, **string**). A union value can be converted implicitly to a larger union, but not to a smaller one, since the value might be of a type that the smaller union lacks.

The types of a union must be clearly distinct: **union** (**int**, **ref int**) is rejected ("firmly related components"), because an **int** variable could become either member. A union value needs the space of its largest member plus the tag, and every conformity clause is a test at run time.

The `print` procedure is an example of unions at work: its parameter is an array of a union of all the types it can print.

7.8. Working with unions

What a conformity clause binds

The name in an alternative is a constant that holds a copy of the value: in `(int n): n +:= 1` the compiler rejects the assignment. To change the value in a union variable `u`, assign a new value to `u` itself, as in `(int n): u := n + 1`. An alternative may also name a union of some of the members: `(union (int, real) x): ...` is chosen for an **int** or a **real**, and `x` is then a value of that smaller union.

The alternatives are chosen by the exact type of the value, without conversions. An **int** in a union does not match an alternative for **real**:

```
union (int, real) u := 3;
case u in
   (real r): print (("real ", fixed (r, 0, 1), newline))
   out print (("not a real", newline))
esac
```

```
not a real
```

> **Pitfall.** A conformity clause without an **out** part, none of whose alternatives matches, does nothing and yields an undefined value. There is no error at compile time or at run time:
>
> ```
> union (int, real, string) u := "seven";
> int n = case u in (int i): i, (real r): ROUND r esac
> ```
>
> ga68 and a68j give `n` the value 0, Algol 68 Genie the value 1. Rust rejects a match that does not cover every case, and a Haskell compiler warns about it and the program stops at run time; Algol 68 does neither. Every conformity clause that is not meant to ignore the remaining types should have an **out** part that reports the error.

Optional values

A union with **void** is the Algol 68 form of an optional value, the `Option<T>` of Rust or the `Optional<T>` of Java. The only value of type **void** is written **empty**:

```
mode Maybe = union (int, void);
proc find = ([] int a, int x) Maybe:
begin
   Maybe result := empty;
   for i from LWB a to UPB a while (result | (void): true | false) do
      if a[i] = x then result := i fi
   od;
   result
end;
[] int primes = (2, 3, 5, 7, 11);
for x from 4 to 7 do
   case find (primes, x) in
      (int i): print ((whole (x, 0), " at ", whole (i, 0), newline)),
      (void): print ((whole (x, 0), " not found", newline))
```

```
   esac
od
```

```
4 not found
5 at 3
6 not found
7 at 4
```

nil serves the same purpose for references, but only for them; a search that returns an index needs **union (int, void)**. The **while** part of the loop uses the short form of the conformity clause: `(result (void): true | false)` is **case** `result` **in (void): true out false esac**. Rosetta Code programs use the short form often.

Which values can be united

Implicit conversions happen in a fixed order (Section 7.14): uniting and widening are alternatives, and rowing can only follow them. A value is therefore united only if its own type is a member of the union, not a type that it could be widened or rowed to:

```
union (real, string) u := 1;
skip
```

```
union-order.a68:4:25: error: int cannot be coerced to union (real, string) in a
    strong-unit
```

```
union (int, string) u := "x";
skip
```

```
union-char.a68:4:24: error: char cannot be coerced to union (int, string) in a
    strong-unit
```

`1` is an **int**, not a **real**, and `"x"` is a **char**, not a **string**; neither is a member of the union. The remedies are to write the value with the right type (`1.0`) or to annotate it (**real** `(1)`, **string** `("x")`, Section 7.12). For the same reason, `print (1)` prints an integer: **int** is a member of the union that `print` accepts, so it is not widened. A one-character string literal must be annotated when it is to become a **string** in a union; this is a common surprise.

A record literal has no type of its own, so it cannot be united either: with **mode** `Pt = struct (int x, y), Sz = struct (int w, h)`, the declaration **union** `(Pt, Sz) r := (1, 2)` is rejected, and **union** `(Pt, Sz) r := Pt (1, 2)` says which member is meant. Uniting does not reach through a reference: a function with a parameter of type **ref union (int, real)** does not accept an **int** variable; the caller must pass a variable of the union type. Nor can a union whose members are arrays be indexed directly: a conformity clause must first say which member it holds. And no conversion applies to the elements of an array: a `[] int` is never converted to a `[] real` or to an array of a union. (Java allows an `Integer[]` to be used as an `Object[]`, which is why Java must check every store into an array at run time.)

7.9. References to references

A reference is a value, so there are variables that hold references. A variable of type **ref int** is declared like any other variable, and its type is **ref ref int**. Such a variable is what C calls a pointer:

```
int a := 1, b := 2;
ref int p := a;          { p holds a reference to a }
p := b;                  { now p refers to b; a and b are unchanged }
ref int (p) := 20;       { assigns to what p refers to, that is to b }
print ((a, b, newline));
print ((p is b, " ", p isnt a, newline))
```

```
         +1        +20
T T
```

The assignment `p := b` stores the reference `b` in `p`; it does not change `a` or `b`. Which cell an assignment changes is determined by the type of its left-hand side: the left-hand side of `p := b` is `p` itself, of type **ref ref int**, so a **ref int** is stored. To assign to the cell that `p` refers to, the left-hand side must be the **ref int** that `p` holds. The expression **ref int** `(p)` says exactly that: it is `p` converted to the type **ref int**, which dereferences it once.

The *null reference* is written **nil**; it refers to no cell, and any attempt to use it as a variable is an error. Two references are compared with **is** and **isnt** (also written `:=:` and `:/=:`); the Report calls these identity relations. They compare references, not the values in the cells: `p` **is** `b` is true because `p` refers to the same cell as `b`.

Comparing a pointer with **nil** needs care. In an identity relation, one side is in a soft context, where no dereferencing happens, and the other in a strong one, in either order. **nil** can only stand in a strong context, so in `q` **is nil** the variable `q` is the soft side and is not dereferenced at all: the comparison is between the variable `q` itself and **nil**, which is always false. The test for a null pointer is **ref int** `(q)` **is nil**:

```
ref int q := nil;
print ((q is nil, " ", ref int (q) is nil, newline))
```

```
F T
```

This is the most common pitfall of Algol 68 references, and a direct consequence of the rule that an implicit conversion never happens unless the context requires it.

7.10. Comparing references

The same rule applies to two pointer variables. If `p` and `q` are both of type **ref ref int** and both refer to `n`, then `p` **is** `q` compares the two variables themselves, which are different cells:

```
int n := 1;
ref int p := n, q := n;
print ((p is q, " ", ref int (p) is ref int (q), " ", ref int (p) is n, newline))
```

```
F T T
```

prints `F T T`. The question that is usually meant, whether the two pointers refer to the same cell, is **`ref int`** `(p)` **`is ref int`** `(q)`. The rule of thumb: annotate one side with the type of the references that are to be compared, and the other side is dereferenced to the same type. **`ref int`** `(p)` **`is`** `n` compares the reference held by `p` with the variable `n`.

An identity relation is not an operator. It cannot be redefined, and it binds more loosely than every operator, so it must be put in parentheses when it is combined with `AND`, `OR`, **`andth`** or **`orel`**: `p` **`is`** `q AND b` means `p` **`is`** `(q AND b)` and is rejected, and the condition of a loop over a list is written **`while`** `(`**`ref`** `Node (p)` **`isnt`** `no_node)` **`andth`** `value` **`of`** `p /= v` **`do`**.

7.11. Records with reference fields

A record field can be a reference, and a constant record can then contain something that changes. A gas meter has a fixed serial number and a reading that grows:

```
mode Meter = struct (int serial, ref int reading);
Meter m = (23146, heap int := 0);
reading of m := 10;          { m is a constant, its reading is not }
Meter v := m;                { copies the reference, not the cell }
ref int (reading of v) := 30;
print ((whole (reading of m, 0), newline))
```

```
30
```

prints `30`. `m` is a constant, but its field `reading` is a **`ref int`**, and the cell it refers to can be assigned to. Copying the record into `v` copies the reference, not the cell, so `m` and `v` share the reading. This is the behavior of a `final` field that refers to a mutable object in Java. The semaphores of Chapter 8 are records of this kind.

Assigning to the reading through the variable `v` needs an annotation. `reading` **`of`** `v` is a field of a variable, and therefore has the type **`ref ref int`**: the variable that holds the reference. `reading` **`of`** `v := 30` would try to store an **`int`** in it. The annotation **`ref int`** `(reading` **`of`** `v)` dereferences once and yields the cell to be changed. In contrast, selecting from or indexing through a reference needs no annotation: if `pa` is a variable of type **`ref [] int`**, then `pa[1] := 5` assigns to an element of the array it refers to, and `next` **`of`** `p` in the lists below selects from the record that `p` refers to. The Report calls this the weak dereferencing of Section 7.14.

7.12. Type annotations

An expression of the form `Type (expression)` is a *cast*. The word suggests a conversion as in C, but an Algol 68 cast converts nothing by force: it only places the expression in a strong context with the given target type, so that exactly the implicit conversions to that type happen. A cast is therefore a type annotation, like `e: T` in Rust or TypeScript. It cannot convert a **`real`** to an **`int`**; that needs `ROUND` or `ENTIER`.

```
union (int, real) u = real (1);          { a real, not an int }
print ((case u in (real): "real", (int): "int" esac, newline));
proc next = int: 42;
proc int saved = next;                   { the function itself }
```

```
int value = next;                          { the result of a call }
print ((saved, value, newline))
```

```
real
        +42        +42
```

The cast `real (1)` makes the united value a `real` rather than an `int`. The declaration `proc int saved = next` binds the function itself, since its type is a function type, whereas `int value = next` calls it.

An array or record literal is allowed only where the type it must have is known, that is, in a strong context. It cannot be the operand of an operator, where the operator is not yet known; an annotation provides the type:

```
print ((UPB [] int (1, 2, 3), newline))
```

```
         +3
```

`UPB (1, 2, 3)` would be rejected. Annotations are also the way to say which member of a union a record literal belongs to (Section 7.8), and `void (x := f)` discards a value explicitly.

7.13. Orthogonality

The constructions of types can be combined without restriction. There are arrays of functions (`[] proc (real) real`), functions that return functions (`proc (int) proc (int) int`), records with function fields, unions of records, references to arrays of references, and so on. Every type except `void` can be the type of a parameter, a result, a field or an element; `void` can be the result of a function and a member of a union.

The constructs of the language are equally orthogonal. Any expression can stand wherever a value of its type is required, and a block or an `if` can stand where other languages only allow a simple expression, as Chapter 3 showed. This is why Algol 68 needs few special rules, and why its few rules apply everywhere.

7.14. Implicit conversions and contexts

The chapters so far introduced the six implicit conversions one at a time. Here they are together:

Conversion	Report	Effect
dereferencing	dereferencing	a reference is replaced by the value it refers to
calling	deproceduring	a parameterless function is called
uniting	uniting	a value becomes a value of a union type
widening	widening	`int` to `real` to `compl`; `bits` to `[] bool`; `bytes` to `[] char`
rowing	rowing	a value becomes an array with one element, an array gains a dimension
discarding	voiding	a value is thrown away

Rowing also applies to references: a variable of type **ref int** can become a **ref [] int** whose only element is the variable itself (Section 7.6).

Where each conversion is allowed depends on the context in which the expression stands. There are five contexts, from the most to the least permissive:

Context	Where	Conversions allowed
strong	right-hand side of a declaration or assignment, argument, cast, function body, element of a literal, one side of an identity relation	all
firm	operand of an operator	dereferencing, calling, uniting
meek	condition of **if**, **case**, **while**; index, bound, **from**, **by**, **to**; the function in a call	dereferencing, calling
weak	the array in an indexing, the record in a selection	dereferencing (but not of the last reference), calling
soft	left-hand side of an assignment, one side of an identity relation	calling

In a strong context, the required type is known, and the compiler applies whatever conversions lead to it, always in the same order: first dereferencing and calling as often as needed, then either one uniting or widening (twice for **int** to **compl**), then rowing (once per dimension added). A parameterless function `f` of type **proc ref int** that stands where a **real** is required is called, its result is dereferenced, and the **int** is widened.

In a firm context, the operator is not known until the types of the operands are, so the conversions that would change the type substantially (widening, rowing) are excluded; uniting is allowed so that an operator can be declared for a union. A meek context requires a value of a particular kind, such as a **bool** for a condition or an **int** for an index, and only dereferencing and calling are allowed. A weak context keeps a reference: in `a[i] := 0`, the array `a` is not dereferenced to its value, since the element must remain a reference to be assigned to; this is also why a slice of an array variable is a reference (Section 4.5). And a soft context keeps the reference on the left-hand side of an assignment entirely.

The branches of an **if** or a **case** that yields a value are balanced: one of them is in the context of the whole expression, and the others are in a strong context, so they can be converted to a common type even where the context itself allows little. **if** `b` **then** `1` **else** `2.5` **fi** is a **real**, and so is `(b 1 | 2.5) + 1|`. On the left-hand side of an assignment, **if** `b` **then** `x` **else** `y` **fi** `:= 0` needs branches that yield references of one type, though a branch may be dereferenced to match the other.

The Report calls a context a syntactic position and its kind its sort. Most of the time, these rules are invisible: they make `x := x + 1` mean what it should. They become visible in error messages, which mention them ("in a strong-unit"), and in the cases of this chapter: the pointer comparisons, the order of uniting and widening, and the annotations that help in an ambiguous situation.

7.15. Lifetimes

Every value that a program creates exists for a certain time. A cell allocated by **loc**, which includes every variable declaration, lives as long as the block in which it is allocated is running; when the block ends, the cell is gone. A cell allocated by **heap** lives as long as the program can reach it; when no reference to it remains, its memory is reclaimed by a garbage collector. A function can therefore return a reference to a new heap cell:

```
proc counter = (int start) ref int: heap int := start;
ref int c = counter (10);
c +:= 5;
print ((c, newline))
```

```
        +15
```

The Report calls the lifetime of a value its scope, a word this book reserves for the visibility of names (Section 3.3). A block creates a lifetime that is newer, that is shorter, than the lifetime of the enclosing block; heap cells and the values of the standard library have the longest lifetime. The lifetime of a record or an array that contains references is that of its shortest-lived reference.

The *lifetime rule* says that a reference must never outlive the cell it refers to: a value may not be assigned to a variable that lives longer than the value. Storing a reference to a local cell in a variable of an enclosing block, or returning it from the block, breaks the rule:

```
ref int p;
begin
   int x := 1;
   p := x          { wrong: p outlives x }
end
```

Rust enforces a rule of the same kind with its borrow checker, entirely at compile time; in Algol 68 it can only partly be checked by the compiler and otherwise must be checked while the program runs.

> **Implementation note.** The Report says that a program that breaks the lifetime rule is wrong and that its further behavior is undefined, which allows but does not require an implementation to check it. Algol 68 Genie checks: it warns at compile time and stops the program with the error "value is exported out of its scope". ga68 and a68j do not check, and such a program may even appear to work, which makes the error all the more dangerous. Use **heap** for anything that must outlive its block.

The lifetime of a loc cell

loc is an expression like any other and can appear anywhere a reference is needed, for example as a dummy argument for a **ref** parameter whose result is not wanted. The precise rule for its lifetime has a surprising consequence. A **loc** cell belongs to the nearer of two things: the function or operator body in which it is allocated (every call counts), or the innermost block, loop body or other range that declares a variable, a constant, a function, an operator or a type. Ranges without such declarations do not count, nor do the loop counter, the name in an alternative of a conformity clause, labels and priority declarations.

Every call of a function counts, even if its body declares nothing: `proc f = ref int: loc int := 0` returns a reference to a cell that is gone when the call is over. The following program keeps references to cells allocated in a loop body:

```
[3] ref int cells;
for i to 3 do
   int square = i * i;
   cells[i] := loc int := square
od;
print ((ref int (cells[2]), newline))
```

Algol 68 Genie stops it:

```
6           cells[i] := LOC INT := square
                        1
a68g: runtime error: 1: REF INT value is exported out of its scope, in VOID loop-clause
    starting at "FOR" in line 4.
```

The loop body declares `square`, so every `loc int` belongs to one execution of the body and is gone when the next one starts. Without the declaration, writing `cells[i] := loc int := i * i`, the cells belong to the block around the loop, and the program is correct and prints 4. A harmless-looking change to the body thus changes the lifetime of the cells. The advice is simple: if a reference leaves the block or loop body in which its cell is allocated, allocate the cell with `heap`.

`heap` can also be used in declarations: `heap int n := 5` and `heap [10] int a` declare variables whose cells live on the heap. There is no operation to free a heap cell; that is left to the garbage collector, which costs time and memory. The distinction is the one between the stack and the heap in C++ and Rust (`Box`); Java and Python allocate every object on the heap.

7.16. Lifetimes of functions

The lifetime rule also applies to functions, and this is where Algol 68 differs most from current languages. A function value lives as long as the innermost block that declares a name the function uses but does not declare itself: a variable, a constant, a parameter of an enclosing function, another function, an operator, a label it jumps to, or a type name with bounds. A function that uses such a name cannot be returned from that block or stored in a variable outside it.

In JavaScript, Python or Haskell, returning a function that composes two other functions is an everyday idiom. In Algol 68 it is an error:

```
mode Fn = proc (real) real;
proc compose = (Fn f, g) Fn: (real x) real: f (g (x));
proc double = (real x) real: 2 * x;
proc inc = (real x) real: x + 1;
Fn h = compose (double, inc);
print ((fixed (h (1), 0, 1), newline))
```

Algol 68 Genie stops at the call of `compose`:

```
7        FN h = compose (double, inc);
                1
a68g: runtime error: 1: FN value is exported out of its scope, in VOID closed-clause
    starting at "BEGIN" in line 1.
```

The inner function uses `f` and `g`, the parameters of `compose`, which exist only while `compose` runs. Putting the data on the heap does not help either:

```
proc int next = (heap int count := 0; int: count +:= 1)
```

The cell of `count` would live long enough, but the name `count` belongs to the block, and the function uses the name. Algol 68 Genie stops this program as well, with the same message; ga68 and a68j run both programs and print plausible results. Here the lack of a check is most dangerous, since such a program appears to work.

The reason for the rule is that an Algol 68 function does not keep alive the variables and constants it uses, as a closure in Lisp, JavaScript or Python does by keeping them on the heap; it refers to the block in which they are declared, which lives on the stack. A C++ lambda that captures by reference has the same problem, without any check; Rust rejects such a closure at compile time unless it is declared `move`.

The usual way out is to pass the state explicitly. A counter becomes a heap cell and a function that takes it as a parameter:

```
proc make_counter = ref int: heap int := 0;
proc next = (ref int counter) int: counter +:= 1;
ref int c = make_counter, d = make_counter;
for k to 3 do print ((whole (next (c), 0), " ")) od;
print ((whole (next (d), 0), newline))
```

```
1 2 3 1
```

which prints `1 2 3 1`. A function that composes two functions becomes a procedure `(Fn f, g, real x) real: f (g (x))` that receives all three values at each call. Where a function value must be kept, a record that holds the state together with a function that takes the state as a parameter does the job of a closure. Algol 68 Genie offers a further way out, partial parametrization, an extension of the language that Appendix G describes.

7.17. Recursive types

A type declaration may refer to the type it declares, which allows linked data structures. A list of integers is a chain of records, each of which holds a value and a reference to the next record:

```
mode Node = struct (int value, ref Node next);
ref Node no_node = nil;
ref Node list := no_node;
for i to 5 do list := heap Node := (i * i, list) od;     { prepend }
ref Node p := list;
while ref Node (p) isnt no_node do
   print (value of p);
   p := next of p
```

```
od;
print (newline)
```

```
        +25        +16         +9         +4         +1
```

The constant `no_node` is **`nil`** of the type **`ref`** `Node`; naming it makes the comparisons readable. The loop builds the list by putting each new record in front: **`heap`** `Node := (i * i, list)` allocates a record on the heap, initializes it, and yields the reference to it, which is stored in `list`. The second loop follows the references; `value` **`of`** `p` and `next` **`of`** `p` dereference `p` automatically. The records must be allocated with **`heap`**: with **`loc`** inside a procedure, the list would refer to cells that are gone when the procedure returns, and ga68 and a68j would not report it.

The recursion must pass through a reference or a function type. A record that contains itself directly would be infinitely large, and the compiler rejects it:

```
access Genie_Transput
begin
   mode Chain = struct (int value, Chain next);
   skip
end
```

```
bad-mode.a68:3:9: error: chain does not specify a well formed mode
bad-mode.a68:3:9: note: mode chain is infinite
```

Passing through a reference is not enough on its own, though: **`mode`** `A = ` **`ref`** `A` or **`mode`** `U = ` **`union`** `(` **`int`**`, ` **`ref`** `U)` would make a type out of nothing but references, and they are rejected as well. The precise rule is that every cycle in a type definition must pass both through a reference or a function type, and through a record or a function type with parameters. **`struct`** `(` **`int`** `value, ` **`ref`** `Node next)` satisfies it, and so does **`mode`** `F = ` **`proc`** `(F) ` **`int`**. The Report calls the admissible recursive types well-formed modes.

Several types can refer to each other, if they are declared in the same block:

```
mode Person = struct (string name, ref Dept dept),
     Dept = struct (string title, ref Person head);
heap Dept d := ("Research", nil);
heap Person p := ("Ada", d);
head of d := p;
print ((name of head of dept of p, " heads ", title of d, newline))
```

```
Ada heads Research
```

Pointers to links

The type **`ref ref`** `Node` is the type of a variable that holds a link: the variable `list` itself, or the field `next` of a node. A procedure that receives such a reference can change the link, which makes operations on lists simple. The following procedure removes the first node with a given value. It walks along the links, not along the nodes, so that the first node needs no special treatment:

```
proc remove = (ref ref Node head, int v) void:
begin
   ref ref Node link := head;   { the variable that refers to the node }
   while (ref Node (link) isnt no_node) andth value of link /= v do
      link := next of link
   od;
   if ref Node (link) isnt no_node then
      ref ref Node (link) := next of link
   fi
end;
```

```
2 4
```

After `remove (list, 1)`, `remove (list, 3)`, `remove (list, 5)` and `remove (list, 9)` on the list 1 to 5, the list holds `2 4`. `link` is a pointer variable of type **`ref ref ref`** `Node`. **`ref`** `Node (link)` is the node that the current link refers to, and `next` **`of`** `link` is the link inside that node. The assignment **`ref ref`** `Node (link) := next` **`of`** `link` changes the current link so that it skips the node.

> **Pitfall.** The annotation decides what is assigned. With **`ref`** `Node (link) := next` **`of`** `link`, the left-hand side would be the node itself, and the assignment would copy the next node's value and link into it: a different operation, which fails at the last node, whose `next` is **`nil`**. Both versions compile. When an assignment involves several levels of references, it pays to write down the type of each side.

7.18. Unions and recursive types

Unions and recursive types together describe tree-shaped data whose nodes are of different kinds: the syntax tree of a compiler, a document, a JSON value. An arithmetic expression is a number, a variable, or an operator with two operands:

```
access Genie_Transput
begin
   { Arithmetic expressions as trees. }
   mode Expr = union (real, char, ref Binop),
        Binop = struct (char sym, Expr left, right);

   proc bin = (char sym, Expr left, right) Expr: heap Binop := (sym, left, right);

   proc show = (Expr e) string:
      case e in
         (real r): fixed (r, 0, 1),
         (char name): name,
         (ref Binop b): "(" + show (left of b) + " " + sym of b + " "
                            + show (right of b) + ")"
      esac;

   proc eval = (Expr e, real x) real:
      case e in
         (real r): r,
```

```
        (char name): x,
        (ref Binop b):
           begin
              real l = eval (left of b, x), r = eval (right of b, x);
              char o = sym of b;
              if o = "+" then l + r
              elif o = "-" then l - r
              elif o = "*" then l * r
              else l / r
              fi
           end
     esac;

  { 3 x x - 2 x + 1 }
  Expr e = bin ("+", bin ("-", bin ("*", 3.0, bin ("*", "x", "x")),
                               bin ("*", 2.0, "x")),
                       1.0);
  print ((show (e), newline));
  for i from 0 to 3 do
     print (("x = ", whole (i, 0), ": ", fixed (eval (e, i), 0, 1), newline))
  od
end
```

prints

```
(((3.0 * (x * x)) - (2.0 * x)) + 1.0)
x = 0: 1.0
x = 1: 2.0
x = 2: 9.0
x = 3: 22.0
```

The type `Expr` is a union of **`real`**, **`char`** (the name of a variable) and **`ref`** `Binop`, and `Binop` is a record that contains two `Expr` values. The cycle passes through the reference and the record, so the types are well-formed. **`union`** `(`**`real`**`, `**`char`**`, Binop)` without the reference would be infinitely large and is rejected. In Rust the same type is written `enum Expr { Num(f64), Var(char), Bin(Box<Binop>) }`; the `Box` plays the role of **`ref`**.

`show` and `eval` are recursive functions with one alternative of a conformity clause per kind of node. The procedure `bin` allocates a node on the heap; its arguments are united to `Expr` automatically, since an argument is in a strong context.

Operators on unions

It is tempting to make the construction of expressions more readable with operators: **`op`** `+ = (Expr a, b) Expr: bin ("+", a, b)`, so that an expression can be written as `3.0 * x * x - 2.0 * x + 1.0`. Declared in the program above, this operator takes over every `+` of two **`real`** values in its scope. The operands of an operator are in a firm context, where uniting is allowed, and the new `+` is declared in an inner block, so it hides the standard `+` of the library for **`real`** operands. `l + r` in `eval` then builds an `Expr` instead of adding, and the compiler reports

```
error: 'EXPR'mode cannot be coerced to 'real' in a strong-unit
```

Inside the body of an operator on a union, the effect can be worse: an `op = = (union (int, Rational) a, b) bool` whose body compares two `int` values with `=` calls itself, without end. The remedy is to declare an operator for each combination of member types that is needed (`(Rational, Rational)`, `(int, Rational)`, `(Rational, int)`), or to give the operator a new name, such as `PLUS`.

7.19. When two types are the same

Section 4.8 showed that type names are only abbreviations: two record types with the same fields are the same type. This also holds for recursive types. The types

```
mode A = struct (int v, ref A next);
mode B = struct (int v, ref B next)
```

are the same type, since expanding either definition gives the same infinite structure. Deciding this is a non-trivial task for the compiler, but it is invisible to the programmer. Together with the rules for unions, it means that two types are the same if and only if they have the same structure; the Report calls them equivalent modes.

7.20. A complete program

The final program counts the words of a text with a binary search tree. Each node holds a word, a count and two subtrees. The procedure `insert` takes a reference to the variable that holds a subtree, of type `ref ref Tree`, so that it can replace an empty subtree with a new node; `show` traverses the tree in order and prints the words alphabetically.

```
access Genie_Transput
begin
   { Counting words with a binary search tree. }
   mode Tree = struct (string key, int count, ref Tree left, right);
   ref Tree no_tree = nil;
   proc insert = (ref ref Tree t, string w) void:
      if ref Tree (t) is no_tree then
         t := heap Tree := (w, 1, no_tree, no_tree)
      elif w < key of t then insert (left of t, w)
      elif w > key of t then insert (right of t, w)
      else count of t +:= 1
      fi;
   proc show = (ref Tree t) void:
      if t isnt no_tree then
         show (left of t);
         print ((key of t, " ", whole (count of t, 0), newline));
         show (right of t)
      fi;
   ref Tree root := no_tree;
   string text = "the quick brown fox jumps over the lazy dog the end";
   string w := "";
   for i to UPB text + 1 do
      if i > UPB text orel text[i] = " " then
```

```
         (UPB w > 0 | insert (root, w); w := "")
      else
         w +:= text[i]
      fi
   od;
   show (root)
end
```

```
brown 1
dog 1
end 1
fox 1
jumps 1
lazy 1
over 1
quick 1
the 3
```

The test `ref Tree (t) is no_tree` uses the cast of Section 7.9: `t` is a reference to a variable, and the question is whether the variable holds `nil`. In `show`, the parameter is the tree itself, so `t isnt no_tree` needs no cast. The splitting loop uses `orel` (Section 2.9) so that `text[i]` is not evaluated when `i` is beyond the end.

Exercises

The answers are at the end of the book.

1. What are the bounds of `s` after `flex [1:0] char s; s := "abc"; s := s[2:3] + "xyz"`?
2. Why can `ref [] int` not be the type of the parameter of the procedure `append`?
3. Write a procedure that takes a `union (int, real, bool)` and returns it as a `real`: an `int` widened, a `bool` as 1.0 or 0.0.
4. Given `int x := 1; ref int p := x`, which of the following change `x`? (a) `p := 2` (b) `ref int (p) := 2` (c) `x := 2`
5. Why is `q is nil` always false for a variable `q` of type `ref int`, and what is the correct test?
6. In which contexts are the following expressions: (a) `x` in `x := 1`, (b) `x` in `y := x`, (c) `x` in `x + 1`, (d) `b` in `if b then`?
7. Write a procedure that reverses a list of the type `Node` of Section 7.17 in place.
8. `flex [1:0] string words` holds some words. Why is `words[1] +:= "s"` rejected, and how can the program add an `s` to the first word?
9. Which of these declarations are accepted? (a) `union (int, string) u := "a"` (b) `union (int, string) u := "ab"` (c) `union (real, string) u := 2` (d) `union (int, real) u := 2`

10. With `int a := 1, b := 1; ref int p := a, q := b`, what do `p is q`, `ref int (p) is ref int (q)` and `ref int (p) is a` yield? What changes after `q := a`?

11. The `push` of Section 7.3 doubles the capacity. How many elements are copied in total by 1000 calls of `push` on an empty `Vector`, and how many by 1000 calls of `append`?

12. Why is the following procedure wrong, and how do ga68, a68j and Algol 68 Genie behave when it is used?

```
proc adder = (int n) proc (int) int: (int x) int: x + n
```

13. Does the program of Section 7.15 that stores `loc` cells remain correct if the body of the loop is `cells[i] := loc int := i * i; print (i)`? And if it is `int j = i; cells[i] := loc int := j`?

14. Build two lists of type `ref Node` with the same contents 1, 2, 3. What does `a is b` yield? Write an operator `EQ` that compares two lists element by element.

Programming problems

1. **Text in pieces.** Keep a long text as a linked list of pieces, each a `string`, so that joining two texts only links their lists. Write procedures for joining, for the length, for the character at a given position, and for turning the list back into one `string`. Which operations are cheaper than with a single `string`, and which are more expensive?

2. **List operations.** For the lists of Section 7.17, write procedures that return the n-th element, append a copy of one list to another, merge two sorted lists into one, and sort a list by repeatedly merging.

3. **Sets.** Represent sets of integers as sorted linked lists, and write the operators `+` (union), `*` (intersection), `-` (difference) and `ELEM` (membership, as in `3 ELEM s`). Then write the same operators for sets represented as `bits` (Section 6.3) and compare.

4. **Tree sort.** Sort an array of integers by inserting its elements into a binary search tree and reading them back in order. Duplicates must be kept. What is the cost for an array that is already sorted?

5. **Postfix expressions.** Read an expression in postfix notation (`3 4 + 2 *`) and evaluate it with a stack of `real` values implemented as a linked list.

6. **Parsing.** Extend the program of Section 7.18: read a fully parenthesized expression such as `((3 * (x * x)) - (2 * x))`, build its tree, print it, and evaluate it for several values of `x`.

7. **Symbolic differentiation.** Write a procedure that returns the derivative of an `Expr` with respect to `x`, and a procedure that simplifies the result: $0 \cdot e = 0$, $1 \cdot e = e$, $e + 0 = e$, and operations on two numbers are computed.

8. **Optional results.** Write a procedure that searches a list of (name, number) records for a name and returns `union (int, void)`. Use it in a program that reads names and reports each one's number or that it is unknown, with an `out` branch in every conformity clause.

9. **A hash table.** Implement a table from strings to integers as an array `[0 : m - 1] ref Entry` of linked lists, with procedures to insert, look up and delete. Count the words of a text with it and compare the result with the tree of Section 7.20.

10. **Pascal's triangle.** Store the first 15 rows of Pascal's triangle in a jagged array, once in a `flex [1:0] flex [1:0] int` that grows row by row, and once as a `[15] ref [] int` whose rows are allocated with `heap`.

11. **A deck of cards.** Declare the 52 cards of a deck as a fixed array of records, each with a field `ref Card next`, and link them into a circle. A hand is a record with a count and a reference to its first card. Deal cards from the deck to four hands only by changing links, without copying a card. Why is it allowed here to keep references to elements of the array?

12. **A queue.** Implement a queue as a record with references to its first and its last node, so that both adding at the end and removing at the front take constant time. Write a procedure that creates a candidate node on the heap and appends it only if it passes a test; what happens to a rejected node?

13. **Exact or approximate.** Find the largest n whose factorial fits in an `int`, without causing an overflow. Then write a function that returns `union (int, real)`: the exact factorial up to that n, and Stirling's approximation beyond it. How must a caller use the result?

14. **A JSON value.** Define a type for JSON values: a union of `void` (null), `bool`, `real`, `string`, and references to arrays and objects, where an object is a list of (key, value) pairs. Write a procedure that prints such a value in JSON notation with indentation. Check that your types are well-formed.

15. **Cross-reference.** Read an Algol 68 program and build an index of its identifiers: a tree, sorted by name, whose nodes hold a list of the line numbers where the name occurs. Print the index in alphabetical order. Reserved words, which are written in lower case in SUPPER stropping, can be recognized with a table.

8. Concurrency

Algol 68 was one of the first languages to include concurrency in its definition. A program can ask for several parts of itself to run at the same time, and the parts can coordinate with *semaphores*, the synchronization primitive that Edsger Dijkstra had introduced a few years earlier. This chapter presents the two constructs, **par** and **sema**, and uses them to solve three classic problems.

> **Implementation note.** Concurrency is the part of the language where the implementations differ most. Algol 68 Genie runs the units of a **par** clause in threads of the operating system. ga68 currently runs them one after the other, which the Report allows, and does not yet support semaphores. a68j runs them in threads when given the option `-fpar-threads`, and supports semaphores only with that option; without it, a program that uses `LEVEL` is rejected ("no lowering routine installed"). The examples of this chapter were run with Algol 68 Genie and with a68j.

8.1. The problems

Two problems arise as soon as parts of a program run at the same time and share data. The first is *mutual exclusion*. If two parts both execute `total +:= 1` on the same variable, each reads the old value, adds one and stores the result; if both read before either stores, one increment is lost. A sequence of operations that must not be interleaved with another such sequence is a *critical section*, and the program needs a way to let only one part at a time enter it.

The second problem is *communication*. One part, the producer, computes values that another part, the consumer, needs. The consumer must wait until a value is available, and if the values are passed through a buffer of limited size, the producer must wait until there is room.

8.2. Concurrent execution

A list of expressions in parentheses, preceded by **par**, is a *concurrent block* (parallel clause). Its expressions may be evaluated at the same time:

```
int a, b;
par (a := 6 * 7, b := 2 ** 10);       { two units, possibly at the same time }
print ((whole (a, 0), " ", whole (b, 0), newline))
```

```
42 1024
```

The **par** clause ends when all its expressions have ended; it has no value. The expressions of an ordinary list may also be interleaved (Section 3.4). What **par** adds is that each of its

expressions is a separate *process*: one of them can wait on a semaphore while the others go on. Waiting outside a **par** clause has no other process to wait for; the Report leaves it undefined, and Algol 68 Genie rejects `DOWN` and `UP` outside a **par** clause altogether.

A **par** clause has a fixed number of expressions. A variable number of processes is started by recursion, each level starting one worker and the rest of the range:

```
int n = 6;
[n] int squares;
proc worker = (int i) void: squares[i] := i * i;
proc spawn = (int lo, hi) void:
   if lo = hi then worker (lo)
   else par (worker (lo), spawn (lo + 1, hi))
   fi;
spawn (1, n);
for i to n do print ((whole (squares[i], 0), " ")) od;
print (newline)
```

```
1 4 9 16 25 36
```

Each worker writes only its own element of `squares`, so the workers need no coordination, and the program prints `1 4 9 16 25 36`.

8.3. Semaphores

A semaphore is a counter that is changed only by two indivisible operations. The type is **sema**, and a semaphore is created with the operator `LEVEL`, which also gives its initial value: **sema** `mutex = LEVEL 1`. The operations are unary operators:

- `DOWN s` waits until the counter of `s` is greater than zero and then decreases it by one. Dijkstra called this operation P; in today's libraries it is called `acquire` or `wait`.
- `UP s` increases the counter by one, which may let a waiting part continue. Dijkstra called it V; today it is `release` or `signal`.

`LEVEL s` also gives the current value of the counter. A semaphore whose counter is 0 or 1 is a *lock* (a mutex); a semaphore with a larger counter limits the number of parts that may pass at the same time.

The standard library declares **sema** as a record with a single field that is a reference to an **int**, and `LEVEL n` allocates the counter on the heap. A copy of a semaphore therefore shares the counter, like the records with reference fields of Section 7.11:

```
sema a = LEVEL 2;
sema b = a;                  { the same counter, not a copy of it }
par (DOWN b, skip);
print ((whole (LEVEL a, 0), newline))
```

```
1
```

prints 1. This is what makes it possible to pass a semaphore to a procedure. A semaphore that was declared without `LEVEL`, such as an element of `[5] sema forks`, holds no counter, and

using it is a run-time error; that is why the programs below initialize every semaphore. `UP` lets all waiting parts compete for the counter again; which one gets it is not specified, so a part may in principle wait forever while others pass.

Note that `LEVEL`, like every unary operator, binds more tightly than any binary operator: `LEVEL n - 1` means `(LEVEL n) - 1`, which is not defined. The initial value `n - 1` must be written in parentheses, as in the last program of this chapter.

8.4. Mutual exclusion

The following program lets three concurrent parts each add 100 000 to a shared total. Each increment is a critical section, protected by the semaphore `mutex`:

```
int total := 0;
sema mutex = LEVEL 1;
proc add = (int n) void:
   to n do
      DOWN mutex;          { enter the critical section }
      total +:= 1;
      UP mutex             { leave it }
   od;
par (add (100000), add (100000), add (100000));
print ((whole (total, 0), newline))
```

```
300000
```

Without the `DOWN` and `UP`, the result would usually be smaller than 300 000, and different from run to run, since increments would be lost when two parts interleave. Such an error is called a *race condition*; it is the classic problem of concurrent programming, and it is hard to find precisely because it does not show up on every run.

The program takes the lock 300 000 times, which costs time and makes the parts wait for each other most of the time. Parts that share nothing need no lock at all, so the better design lets each part count in a variable of its own and takes the lock only once, to add its result:

```
int total := 0;
sema mutex = LEVEL 1;
proc add = (int n) void:
begin
   int mine := 0;                    { private to this part }
   to n do mine +:= 1 od;
   DOWN mutex; total +:= mine; UP mutex
end;
par (add (100000), add (100000), add (100000));
print ((whole (total, 0), newline))
```

```
300000
```

A critical section should be as short as possible and should do no more than the shared data require.

8.5. Producer and consumer

The second program passes values from a producer to a consumer through a buffer with room for four values. Three semaphores coordinate the two parts: `free` counts the free places, `full` the filled ones, and `mutex` protects the buffer and its two positions.

```
access Genie_Transput
begin
   { A producer and a consumer that communicate through a bounded buffer. }
   int size = 4;
   [0 : size - 1] int buffer;
   int head := 0, tail := 0;
   sema free = LEVEL size,     { the number of free places }
        full = LEVEL 0,        { the number of filled places }
        mutex = LEVEL 1;
   proc put = (int value) void:
   begin
      DOWN free;
      DOWN mutex; buffer[tail] := value; tail := (tail + 1) MOD size; UP mutex;
      UP full
   end;
   proc get = int:
   begin
      DOWN full;
      DOWN mutex; int value = buffer[head]; head := (head + 1) MOD size; UP mutex;
      UP free;
      value
   end;
   int n = 20;
   int sum := 0;
   par (
      for i to n do put (i * i) od,           { the producer }
      to n do sum +:= get od                  { the consumer }
   );
   print (("sum of the first ", whole (n, 0), " squares: ", whole (sum, 0),
           newline))
end
```

```
sum of the first 20 squares: 2870
```

`put` waits for a free place, stores the value and signals a filled place; `get` does the opposite. If the producer is faster, it waits in `DOWN free` when the buffer is full; if the consumer is faster, it waits in `DOWN full` when the buffer is empty. Today's languages offer the same structure ready-made as a blocking queue or a channel, but it is built from exactly these parts.

The order of the two `DOWN`s in `put` matters. If the producer took `mutex` first and then waited in `DOWN free` for a free place, it would wait while holding the lock, and the consumer, which needs the lock to make a place free, could never get it: a deadlock. The rule is to wait for a resource before taking the lock that protects it, and to hold the lock for as short a time as possible.

8.6. The dining philosophers

Dijkstra's best-known synchronization problem has five philosophers sitting at a round table, with one fork between each two of them. A philosopher alternately thinks and eats, and needs both neighboring forks to eat. If every philosopher picks up the left fork first and then waits for the right one, all five may end up holding one fork each and waiting forever, a *deadlock*.

One simple solution is a butler who lets at most four philosophers sit at the table at the same time. Then at least one of them can always get both forks. In the program, each fork is a semaphore with the initial value 1, and the butler is a semaphore with the initial value 4:

```
access Genie_Transput
begin
   { The dining philosophers, with a butler who admits at most four at a time. }
   int n = 5, meals = 1000;
   [0 : n - 1] sema fork;
   [0 : n - 1] int eaten;
   for i from 0 to n - 1 do fork[i] := LEVEL 1; eaten[i] := 0 od;
   sema butler = LEVEL (n - 1);
   proc philosopher = (int i) void:
      to meals do
         DOWN butler;
         DOWN fork[i]; DOWN fork[(i + 1) MOD n];
         eaten[i] +:= 1;                              { eat }
         UP fork[(i + 1) MOD n]; UP fork[i];
         UP butler                                    { think }
      od;
   par (philosopher (0), philosopher (1), philosopher (2),
        philosopher (3), philosopher (4));
   for i from 0 to n - 1 do print ((whole (eaten[i], 0), " ")) od;
   print (newline)
end
```

```
1000 1000 1000 1000 1000
```

Each philosopher eats 1000 times and counts the meals in its own element of `eaten`, which no other philosopher changes; the counts are printed after the **par** clause has ended. Without the butler, the program could hang.

Exercises

The answers are at the end of the book.

1. What is the difference between `(a, b)` and **par** `(a, b)`?

2. What happens if a program executes `DOWN s` on a semaphore declared as **sema** `s = LEVEL 0`, and no other part ever executes `UP s`?

3. Is the semaphore `mutex` of the producer and consumer program necessary as long as there is exactly one producer and one consumer? When does it become necessary?

4. Give another solution of the dining philosophers that needs no butler. (Hint: let one philosopher pick up the forks in the other order.)

5. What happens to the producer and consumer program if the two `DOWN`s in `put` are exchanged, when the producer is faster than the consumer?

6. Why does `sema b = a; par (DOWN b, skip)` change `LEVEL a`?

Programming problems

1. Compute the sum of the elements of a large array with four concurrent parts, each summing a quarter.

2. Build a pipeline of three stages, each a concurrent part connected to the next by a buffer: the first generates numbers, the second squares them, the third prints them.

3. Implement a semaphore-protected counter as a record with a `sema` field and the procedures `increment` and `value`.

4. **A channel type.** Define a record type for a bounded buffer (a reference to an array, references to the two positions, and three semaphores) with procedures `send` and `receive`, so that several pipelines can be built from one definition.

5. **Readers and writers.** Several parts read a shared table and some change it. Readers may read at the same time, but a writer needs the table to itself. Solve it with semaphores, and think about whether a writer can be kept waiting forever.

6. **A barrier.** Build a barrier for n processes from semaphores: no process continues past it until all have reached it. Make it reusable.

7. **Two transfers.** Two processes move money between the same two accounts in opposite directions, and each protects both accounts with a semaphore before it changes them. Show how they can deadlock, and avoid it by always taking the semaphore of the account with the smaller number first.

8. **Parallel merge sort.** Sort an array with merge sort in which the two halves are sorted in a `par` clause, down to a given depth, and compare the time with the sequential version.

9. **A prime sieve pipeline.** Generate the primes with a pipeline of processes: each process holds one prime, passes on the numbers it does not divide, and starts a new process for the first number that reaches it.

9. Input and output

The programs so far used `print` and `read` on the standard output and the standard input. This chapter describes input and output in general: how a program reads and writes files, reads from and writes to strings, reacts to the end of the input and to bad data, converts numbers to text, and lays out its output precisely with formats. The Report calls input and output together transput.

The transput of the Report is a large part of its standard library, and the implementations provide it to different degrees. Algol 68 Genie provides nearly all of it; Section 9.12 lists what it lacks. For ga68 and a68j, the module `Genie_Transput` provides the unformatted part used in this chapter; formatted and binary transput are not available there, and the examples of those sections were run with Algol 68 Genie.

9.1. Books, channels and files

The Report distinguishes three things that current languages usually do not separate:

- A book is the data itself, such as a file on disk, identified by a name. Its text is divided into pages, the pages into lines, and the lines into characters, and a position in a book is given by a page, a line and a character number.

- A channel describes a kind of device and what can be done with it: `stand_in_channel` for reading, `stand_out_channel` for writing, `stand_back_channel` for both.

- A file (here of type `File`) is what a program uses to read or write a book through a channel: a file handle or stream, in today's terms. It records the current position in the book, the procedures to call when something unusual happens, and a few settings.

The procedure `open (f, name, channel)` connects the file `f` with an existing book; `establish (f, name, channel)` creates a new one, and `create (f, channel)` creates a new book without a name, a temporary file. These procedures return 0 on success. `close (f)` ends the connection, `scratch (f)` ends it and deletes the book, and `reset (f)` returns to the start of the book. The files `stand_in` and `stand_out` are open from the start; `print (x)` is short for `put (stand_out, x)`, and `read (x)` for `get (stand_in, x)`.

> **Implementation note.** In ga68 and a68j the type is called `File` and the channels and procedures are those of `Genie_Transput`, since the reserved word **`file`** of ga68 does not yet denote a usable type. In the Report and in Algol 68 Genie, the type is **`file`**. In the Report, `establish` has three more parameters, the number of pages, lines per page and characters per line of the new book; the implementations leave them out.

9.2. Reading and writing files

The procedures `put` and `get` write and read a list of values, like `print` and `read`, but on a given file. The following program writes five lines to a new file and then reads them back:

```
File results;
if establish (results, "squares.txt", stand_out_channel) /= 0 then
   print (("cannot create squares.txt", newline)); stop
fi;
for i to 5 do put (results, (i, " ", i * i, newline)) od;
close (results);

File data;
if open (data, "squares.txt", stand_in_channel) /= 0 then
   print (("cannot open squares.txt", newline)); stop
fi;
on_logical_file_end (data, (ref File f) bool: goto done);
do
   int n, square;
   get (data, (n, square));
   print ((whole (n, 0), " squared is ", whole (square, 0), newline))
od;
done:
   close (data)
```

```
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25
```

The reading loop has no condition: it ends when the input is exhausted. The procedure `on_logical_file_end` installs a procedure that the transput calls when a `get` reaches the end of the book; here it jumps out of the loop to the label `done`. Such procedures are the Report's event routines; today they would be called callbacks or handlers. Section 9.4 describes them in general.

The predefined label `stop` ends the program; `goto stop`, or just `stop`, can be written anywhere.

> **Implementation note.** In Algol 68 Genie, `establish` returns 0 even if the file already exists; the error comes with the first `put` ("file exists"). The program above therefore fails when it is run a second time in the same directory. `open` with `stand_out_channel` opens an existing file for writing and empties it. A program that should work in both cases tries `open` first and calls `establish` only if that fails.

9.3. Reading the standard input

The same technique reads all of the standard input, however much there is. Numbers are separated by blanks and line breaks. One detail matters here: the event routine jumps to a label of the program's block, so it lives only as long as that block (Section 7.16), whereas

`stand_in` lives as long as the whole program. Installing the routine in `stand_in` itself would break the lifetime rule; the Report uses a case like this as its example. The program therefore copies `stand_in` into a local file variable and reads from that:

```
{ Sum all numbers on the standard input, however many there are. }
real sum := 0;
int count := 0;
File input := stand_in;          { a copy, local to this block }
on_logical_file_end (input, (ref File f) bool: goto finished);
do
   real x;
   get (input, x);
   sum +:= x;
   count +:= 1
od;
finished:
   print ((whole (count, 0), " numbers, sum ", fixed (sum, 0, 2), newline))
```

```
4 numbers, sum 11.25
```

with the input

```
1.5 2.5
3


4.25
```

The copy `input` reads the same data as `stand_in`: a `File` value contains references to the book and to the current position, which the copy shares, and its own event routines, which it does not share. The copy can therefore be given an event routine that lives only as long as the block, while reading continues where `stand_in` stands.

A number is read up to the first character that cannot belong to it: `12abc` yields 12 and leaves `abc` for the next read, and `1,024` yields 1. Leading blanks and line breaks are skipped. A character is read as it is, a blank included; at the end of a line, the line break is skipped and the first character of the next line is read.

A **`string`** is read up to the end of the line; the line break itself is not read, and the next `get` of a string would yield an empty string at once. It is skipped with `newline`, which, like `space`, `backspace` and `newpage`, can appear in the list of `get` as well as of `put`. On input they skip the rest of the line, one character, and so on; on output they start a new line, move one position, and start a new page. A program that processes its input line by line reads a line, processes it, and then skips the line break:

```
{ Number the lines of the standard input. }
int n := 0;
File input := stand_in;
on_logical_file_end (input, (ref File f) bool: goto finished);
do
   string line;
   get (input, line);              { the rest of the line }
   n +:= 1;
   print ((whole (n, -3), "  ", line, newline));
```

```
      get (input, newline)            { skip the line break }
   od;
finished:
   skip
```

```
 1  Algol 68 was designed
 2  by IFIP Working Group 2.1.
 3
 4  The Revised Report appeared in 1975.
```

The input was:

```
Algol 68 was designed
by IFIP Working Group 2.1.

The Revised Report appeared in 1975.
```

> **Pitfall.** The shorter `get (input, (line, newline))` looks equivalent, but it loses the last line if the input does not end with a line break: `newline` then reaches the end of the input, the event routine jumps away, and the line that was just read is never printed. Text editors and programs do not always write a final line break.

The layout procedures are ordinary procedures of type **`proc (ref File) void`**. They can be called directly, as `newline (stand_out)`, and a program can write its own procedure of this type and put it in a list, where `put` and `get` call it with the file:

```
proc indent = (ref File f) void: put (f, "    ");
print (("total:", newline, indent, "12 items", newline));
for i to 3 do print (i) od;
newline (stand_out);
print (("done", newline))
```

```
total:
    12 items
         +1         +2         +3
done
```

9.4. Events

An event routine is installed in a file for one kind of event. It receives the file and returns a **`bool`**. If it returns **`true`**, it reports that it has dealt with the event and the transput continues; if it returns **`false`**, the default action follows. A routine can also jump out, as in the examples above.

Installed with	Called when	Default action
`on_logical_file_end`	the data of the book are exhausted	error
`on_physical_file_end`	the medium is full or exhausted	error
`on_line_end`	a line is exhausted	`newline`
`on_page_end`	a page is exhausted	`newpage`
`on_format_end`	a format is exhausted (Section 9.9)	start it again
`on_value_error`	a value cannot be read or written	error
`on_char_error`	a character cannot be converted	error

The Report leaves the program undefined where the table says "error"; the implementations stop it with a message. `Genie_Transput` provides the logical end of the file and the value error. Algol 68 Genie provides all except `on_char_error`; a character that cannot be converted causes a value error.

A routine that returns **`true`** must really have dealt with the event, because the transput then continues at the same place. With Algol 68 Genie and `Genie_Transput`, a value error on input occurs when the text is not a number. (The Report calls the char error routine for a character that cannot belong to a number, and the value error routine for a number that cannot be converted.) If the routine only counts the error, the bad text is still there, and the next number fails in the same way:

```
string text := "12 x3 45 67";
File f;
associate (f, text);
make_term (f, " ");
int errors := 0;
on_value_error (f, (ref File g) bool:
   begin
      errors +:= 1;
      true                    { the error has been dealt with }
   end);
int a := -1, b := -1, c := -1, d := -1;
get (f, (a, b, c, d));
print ((whole (a, 0), " ", whole (b, 0), " ", whole (c, 0), " ",
        whole (d, 0), ", errors: ", whole (errors, 0), newline))
```

```
12 -1 -1 -1, errors: 3
```

prints `12 -1 -1 -1, errors: 3`: one bad word spoiled all the following values. The routine must remove the cause, here by reading the bad word as a string:

```
string text := "12 x3 45 67";
File f;
associate (f, text);
make_term (f, " ");
int errors := 0;
on_value_error (f, (ref File g) bool:
   begin
      errors +:= 1;
      string bad;
```

```
        get (g, bad);          { skip the word that is not a number }
        true                   { the error has been dealt with }
    end);
int a := -1, b := -1, c := -1, d := -1;
get (f, (a, b, c, d));
print ((whole (a, 0), " ", whole (b, 0), " ", whole (c, 0), " ",
        whole (d, 0), ", errors: ", whole (errors, 0), newline))
```

```
12 -1 45 67, errors: 1
```

which prints `12 -1 45 67, errors: 1`. In the same way, a logical file end routine that returns **`true`** without having made more data available is called again, according to the Report; the implementations call it once and continue, and the value that was to be read is then not meaningful. Such a routine should jump away, as in the examples above, or return **`false`**.

9.5. Strings as files

The procedure `associate (f, s)` connects a file with a string variable instead of a book. Reading then takes the characters from the string, and writing adds to it. This converts between numbers and their textual form with all the facilities of the transput:

```
string text := "3 4 5";
File f;
associate (f, text);               { f now reads from text }
int a, b, c;
get (f, (a, b, c));
print ((whole (a * a + b * b - c * c, 0), newline));

string line := "";
File g;
associate (g, line);               { g now writes into line }
put (g, ("a = ", whole (a, 0), ", c = ", whole (c, 0)));
print ((line, newline))
```

```
0
a = 3, c = 5
```

> **Implementation note.** In the Report, `associate` connects a file with an array of arrays of arrays of characters (pages, lines, characters) of fixed size, and writing overwrites it from the start. Algol 68 Genie and `Genie_Transput` use a **`ref string`** instead and append to it, which is what programs need in practice.

9.6. Terminators and other settings

A string read with `get` ends at the end of the line. The procedure `make_term (f, s)` makes it end also at every character of the string `s`; the terminating character itself is not read. The complete program at the end of this chapter uses it to read a name that ends at a blank.

A file can be asked what it allows: `get_possible (f)`, `put_possible (f)`, `bin_possible (f)`, `reset_possible (f)` and `set_possible (f)` return a **`bool`**. After a file has been written, `reset` returns to the start so that it can be read; reading and writing cannot simply alternate.

The Report provides further procedures, which Algol 68 Genie does not have: the current position (`char_number`, `line_number`, `page_number`) and the setting of it (`set_char_number` and a `set` with page, line and character), a new name for a book (`reidf`), and a translation table for characters (`make_conv`). A program that needs the current line number counts the lines itself. `Genie_Transput` provides, besides `make_term`, the functions `eof` and `eoln` of Algol 68 Genie, which test for the end of the file and of the line.

9.7. Printing arrays and records

`put` and `print` accept arrays and records, not only single values, provided that they contain no references, functions or unions. They are broken down into their elements and fields, which are printed one after the other; the Report calls this straightening. An array of records thus prints the fields of every element:

```
mode Point = struct (int x, y);
[] Point path = ((1, 2), (3, 4));
print ((path, newline))
```

```
+1          +2          +3          +4
```

`get` straightens in the same way: `get (f, m)` for a `[2,3] int` variable `m` reads six numbers into it, row by row. For input the Report is stricter: apart from a **`string`** variable itself, the variable may contain no resizable arrays, so a record with a **`string`** field or an array of strings cannot be read with one `get`.

> **Implementation note.** `Genie_Transput` straightens arrays of the primitive types but not records (Section 4.11), and its `get` does not accept arrays; this example was run with Algol 68 Genie, whose **`int`** has 64 bits and is therefore printed in a wider field. Algol 68 Genie also reads a record with **`string`** fields with one `get`.

9.8. Converting numbers to text

The procedures `whole`, `fixed` and `float` of Section 2.14 convert a number into a string of a given width, which is the usual way to lay out numbers without formats:

- `whole (i, w)` writes an integer in a field of width $|w|$. If `w` is positive, a sign is always written; if it is negative, only a minus sign. `w = 0` gives the shortest string.
- `fixed (x, w, d)` writes a real with `d` digits after the point, in a field of width $|w|$, with the same rules for the sign.
- `float (x, w, d, e)` writes a real in scientific notation with `d` digits after the point and an exponent of `e` digits.

```
print (("[", whole (42, 6), "] [", whole (42, -6), "] [", whole (-42, 0),
        "] [", whole (123456, 3), "]", newline));
print (("[", fixed (3.14159, 8, 3), "] [", fixed (3.14159, -8, 3),
        "] [", fixed (3.14159, 6, 4), "] [", fixed (0.015, 0, 4), "]", newline));
print (("[", float (31415.9, 10, 3, 2), "] [", float (31415.9, -10, 3, -2),
        "]", newline))
```

prints

```
[   +42] [    42] [-42] [***]
[  +3.142] [   3.142] [+3.142] [.0150]
[+31.416e+3] [ 31.416e 3]
```

A number that does not fit is not given a wider field, unlike C's `printf`: `whole (123456, 3)` yields three asterisks. `fixed` first gives up digits after the point, so that `fixed (3.14159, 6, 4)` yields `+3.142`; only when the integral part does not fit either does it yield asterisks. `fixed` writes no zero before the point of a number below 1 if the width is 0 (`.0150`). `float` chooses the exponent so that as many digits as possible stand before the point.

9.9. Formatted output

For complete control over the layout, the Report provides *formats*, a small language of their own, comparable to the format strings of `printf` in C or of Fortran's `FORMAT` statement. A format text is written between dollar signs. The procedure `printf` takes a list that begins with a format and continues with the values; `putf` does the same for a given file, and `readf` and `getf` read according to a format.

```
int n = 42;
real x = 3.14159;
printf (($"[" 5d "]" l$, n));          { five digits, leading zeros }
printf (($"[" 5zd "]" l$, n));         { leading zeros suppressed }
printf (($"[" -4zd "]" l$, -n));       { a sign if negative }
printf (($"[" zzd.dd "]" l$, x));      { fixed point }
printf (($"[" d.3de+2d "]" l$, x * 1000));   { floating point }
printf (($"[" 3a "]" x b l$, "abc", true))
```

```
[00042]
[    42]
[-   42]
[  3.14]
[3.142e+03]
[abc] T
```

A format consists of *pictures*, each of which describes how one value is printed, separated by commas. A picture is made of *frames*:

Frame	Meaning
`d`	a digit
`z`	a digit, printed as a blank if it is a leading zero
`+`, `-`	a sign: always, or only if negative
`.`	the decimal point
`e`	the exponent marker of a floating-point number
`a`	a character
`b`	a truth value (`T` or `F`)
`g`	a value in its standard layout, as `print` would print it

A number before a frame repeats it: `5d` is five digits and `3zd` is three suppressible digits followed by one that is always printed. Between the frames, *insertions* add text: a string in quotes is printed as it is, `x` prints a blank, `l` ends the line and `p` the page, and `k` moves to a given column. They too can be repeated, as in `2x` or `3"-"`.

One picture, one value

Everything between two commas is one picture, and one picture prints one value, however many frames it has:

```
printf (($"a = " 2d ", b = " 2d l$, 1));      { one picture: one value }
printf (($"a = " 2d, ", b = " 2d l$, 1, 2))   { two pictures }
```

prints

```
a = 00, b = 01
a = 01, b = 02
```

In the first format, `2d ", b = " 2d` is a single picture of four digits with some text in the middle, and it prints the single value 1 as `0001` with the text inserted. This is easy to get wrong and hard to see; when a format prints something strange, the commas are the first thing to check.

Where the sign goes

A sign frame before the digits puts the sign at the left edge of the field. A sign frame after the `z` frames lets it move to just before the first printed digit:

```
for i to 3 do
   int n = (i | 42, -42, -7);
   printf (($"[" -4zd "] [", 4z-d "] [", 4z+d "] [", +4d "]" l$, n, n, n, n))
od
```

prints

```
[    42] [    42] [   +42] [+0042]
[-   42] [   -42] [   -42] [-0042]
[-    7] [    -7] [    -7] [-0007]
```

A negative value in a picture without a sign frame is a value error.

Choices, general patterns and other kinds of values

A truth value can be printed with two strings of the program's choice, `b("yes", "no")`, and an integer between 1 and n can select one of n strings with `c("one", "two", ...)`. Both make it easy to print plurals and names:

```
for n from 0 to 2 do
   printf (($g(0) " file" b("", "s") " found" l$, n, n = 1))
od;
for month from 1 to 3 do
   printf (($c("January", "February", "March") ", " g(0) l$, month, month))
od;
printf (($b("yes", "no") ", " b l$, 3 > 2, 3 > 4))
```

prints

```
0 files found
1 file found
2 files found
January, 1
February, 2
March, 3
yes, F
```

The general pattern `g` prints a value as `print` would. With parameters, it prints a number like the conversion procedures of Section 9.8: `g(w)` like `whole`, `g(w, d)` like `fixed`, and `g(w, d, e)` like `float`. The parameters can be any integer expressions. A **`bits`** value is printed in base 2, 4, 8 or 16 with a picture such as `2r8d` or `16r4d`:

```
real x = 1234.5678;
printf (($g l$, x));
printf (($g(10) "|" g(-10, 2) "|" g(-12, 3, 2) l$, 42, x, x));
int w = 9;
printf (($g(w, w - 6) l$, x));            { the parameters can be computed }
bits b = 2r1011;
printf (($2r8d, " " 16r4d l$, b, 16rbeef))
```

prints

```
+1.23456780000000e  +3
       +42|   1234.57| 1234.568e+0
+1234.568
00001011 beef
```

The first line shows the standard layout of a **`real`** in Algol 68 Genie, whose **`real`** has 64 bits.

A string is printed with `a` frames, one per character, and the number of frames must equal the length of the string; insertions between the frames are allowed:

```
printf (($a". "5a l$, "JSmith"));          { one string, split by an insertion }
printf (($"[" 5a "]" l$, "Algol"));
printf (($"[" 5a "]" l$, "Al"))           { wrong length: a run-time error }
```

```
J. Smith
[Algol]
5        printf (($"[" 5a "]" l$, "Al"))          COMMENT wrong length: a run-time
    error COMMENT
                         1
a68g: runtime error: 1: error transputting [] CHAR value, in [] "SIMPLOUT"
    collateral-clause starting at "(" in this line.
```

prints `J. Smith` and `[Algol]`, and then stops with an error, since `"Al"` has only two characters for five frames. A string of variable length is printed with `g` or with the computed repetition of the next section.

A part of a format can also be chosen when the program runs. The format pattern `f(...)` contains an expression that yields a format, and inserts that format in its place:

```
for n to 3 do
   printf (($g(0), x f(n = 1 | $"item"$ | $"items"$) l$, n))
od;
printf (($"Smith""Jones" l$));
printf (($"Smith" 1"Jones" l$))
```

prints

```
1 item
2 items
3 items
Smith"Jones
SmithJones
```

> **Pitfall.** Two string insertions with nothing between them are one string: as in a string literal, a doubled quote stands for a quote character, so `"Smith""Jones"` prints `Smith"Jones`, as the fourth line shows. A repetition count, even `1`, keeps them apart. Algol 68 Genie also accepts a space between them, the Report does not.

Repetition and columns

A repetition can be computed while the program runs: in `n (k) (...)`, the part in parentheses is repeated `k` times, which suits arrays of any size. An integer value is printed through a real picture as a real number:

```
format row = $3zd, 2x zd.ddd, 2x zzzd.d l$;
printf (($"   n   sqrt n    n ** 3" l$));
for n to 5 do printf ((row, n, sqrt (n), n ** 3)) od;
[] int v = (7, 11, 13, 17);
printf (($"values:" n (UPB v) (x 2d) l$, v))
```

```
   n   sqrt n    n ** 3
   1   1.000     1.0
   2   1.414     8.0
   3   1.732    27.0
   4   2.000    64.0
   5   2.236   125.0
values: 07 11 13 17
```

The insertion `k` moves to the column given by its repetition count, which can also be computed. That is enough for a simple plot:

```
for i from 0 to 8 do
   real x = i * pi / 8;
   int column = 10 + ROUND (20 * sin (x));
   printf (($zd.dd, n (column) k "*" l$, x))
od
```

prints

```
 0.00    *
 0.39            *
 0.79                  *
 1.18                      *
 1.57                        *
 1.96                      *
 2.36                  *
 2.75            *
 3.14    *
```

How the values and the format are matched

The values of the list are matched with the pictures of the format one by one. When the format is exhausted and values remain, it starts again from the beginning: `printf (($2d l$, 1, 2, 3))` prints three lines. This is the default action of the event `on_format_end`. A list may contain several formats, each of which applies to the values after it. The layout procedures `newline` and `space` cannot appear in the list of `printf`; the insertions `l` and `x` take their place.

A format is a value of the type **`format`** and can be stored in a variable, passed to a procedure, or chosen at run time: a **`proc (int) format`** can return different formats for different cases. A format that computes a repetition from a local name, `n (digits) d`, is subject to the lifetime rule (Section 7.16) like a function, and cannot leave the block of `digits`.

> **Implementation note.** In the Report, the rest of a format that is not used up when the list ends waits for the next `putf` on the same file. Algol 68 Genie stops the program instead ("number of pictures does not match number of arguments"): the values must use the format up to its end.

9.10. Formatted input

`readf` and `getf` read according to a format. Every frame reads one character position: a picture `3zd` reads four characters, and a `z` frame accepts a blank as a zero. The characters of a literal insertion such as `"-"` must be present in the input and are read over; `x` skips one character. This makes it easy to read data in fixed columns, such as the records of this file:

```
Ann Smith  2024-03-15  82
Bob Jones  2023-11-02 100
Cleo Grant 2024-01-30   7
```

```
    format record = $10a, x 4d, "-" 2d, "-" 2d, 3zd l$;
    File input := stand_in;
    on_logical_file_end (input, (ref File f) bool: goto done);
    do
       string name; int year, month, day, score;
       getf (input, (record, name, year, month, day, score));
       printf (($10a, x 2d, "." 2d, "." 4d, 5zd l$, name, day, month, year, score))
    od;
done:
    skip
```

prints

```
Ann Smith  15.03.2024    82
Bob Jones  02.11.2023   100
Cleo Grant 30.01.2024     7
```

The pictures `"-" 2d` skip the dash and read the month. The `10a` reads exactly ten characters, the blanks of a short name included. A field that is one character too short or too long shifts every field after it: with `4zd` for the score, which reads five characters, the program stops at the first record with an error.

9.11. Binary transput

The procedures `put_bin` and `get_bin` write values to a file and read them back in an internal, binary form, without conversion into characters. A file written this way can only be read back by a program that reads the same types in the same order:

```
File f;
if establish (f, "data.bin", stand_back_channel) /= 0 then stop fi;
put_bin (f, (42, 3.5, "xyz"));
reset (f);                                  { back to the start }
int n; real x; [1:3] char s;
get_bin (f, (n, x, s));
print ((whole (n, 0), " ", fixed (x, 0, 2), " ", s, newline));
close (f)
```

```
42 3.50 xyz
```

The channel `stand_back_channel` allows both reading and writing, and `reset` returns to the start of the file before reading. Binary and character transput cannot be mixed on one sequential file until it is reset; Algol 68 Genie enforces this.

Binary transput was meant for intermediate files that a program writes and reads again. Such a file does not need a name, and it should disappear when the program no longer needs it. `create` and `scratch` do exactly that:

```
File scratch_file;
if create (scratch_file, stand_back_channel) /= 0 then stop fi;
for i to 5 do put_bin (scratch_file, i * i) od;
reset (scratch_file);                       { back to the start }
int sum := 0;
```

```
for i to 5 do int n; get_bin (scratch_file, n); sum +:= n od;
scratch (scratch_file);                          { close and delete }
print (("sum of squares ", whole (sum, 0), newline))
```

```
sum of squares 55
```

9.12. What Algol 68 Genie lacks

Algol 68 Genie provides nearly all of the Report's transput. The exceptions, apart from those already mentioned, are:

- the position enquiries `char_number`, `line_number` and `page_number`, `set_char_number`, `reidf`, `make_conv` and the event `on_char_error`;
- `set` moves relative to the current position, with one parameter, instead of to a given page, line and character;
- the insertion `k` cannot move back to an earlier column, and `backspace` and the insertion `y` cannot be used on `stand_out`;
- a `z` frame after a `d` frame always prints its digit, where the Report suppresses some zeros in date-like pictures; pictures with only `d` frames after the first are printed the same everywhere;
- the format is not kept for the next call of `putf` (Section 9.9).

9.13. A complete program

The final program reads a file of names and marks and prints a report. The file `marks.txt` contains:

```
Ann 72 85 90
Ben 88 79 81
Cleo 95 91 70
Dan 60 75 68
```

```
access Genie_Transput
begin
   { Read names and marks from a file and print a report. }
   File data;
   if open (data, "marks.txt", stand_in_channel) /= 0 then
      print (("cannot open marks.txt", newline)); stop
   fi;
   make_term (data, " ");                  { a name ends at a blank }
   on_logical_file_end (data, (ref File f) bool: goto done);
   int students := 0;
   real class_total := 0;
   print (("name      average", newline));
   do
```

```
      string name;
      int a, b, c;
      get (data, (name, a, b, c, newline));
      real average = (a + b + c) / 3;
      print ((name, " " * (10 - UPB name), fixed (average, -7, 2), newline));
      students +:= 1;
      class_total +:= average
   od;
done:
   close (data);
   print (("class     ", fixed (class_total / students, -7, 2), newline))
end
```

```
name      average
Ann         82.33
Ben         82.67
Cleo        85.33
Dan         67.67
class       79.50
```

`make_term (data, " ")` ends a name at the first blank, so that `get` reads the name and then the three numbers. The `newline` at the end of the list skips the line break, so that the next name starts at the beginning of the next line. The expression `" " * (10 - UPB name)` pads the name to ten characters.

Exercises

The answers are at the end of the book.

1. What is the difference between `open` and `establish`?
2. What does an event routine that returns **`true`** tell the transput?
3. Write a procedure `int_to_string` that converts an **`int`** into a string by writing it into a string with `associate` and `put`.
4. What do the formats `$4d$`, `$4zd$` and `$-3zd$` print for the values 7 and −7?
5. Why does the program of Section 9.3 read the line and the line break with two calls of `get`?
6. How many values does each of these formats print, and what do they print for the values 3 and 4? (a) `$2d "/" 2d$` (b) `$2d, "/" 2d$`
7. What do `whole (-5, 3)`, `whole (5, -3)`, `fixed (2.5, 0, 2)` and `fixed (-0.25, 0, 2)` yield?
8. A value error routine returns **`true`** without reading anything. What happens when `get (f, (a, b))` meets the input `x 5`?
9. Write a format that prints a date given as three integers day, month and year as `15 March 2024`.

Programming problems

1. **Line numbers.** Copy a text file to another file with a line number in front of each line, and start a new page with a page number after every 50 lines.

2. **Word count.** Count the characters, words and lines of the standard input, like the Unix program `wc`. Make sure that a last line without a line break is counted.

3. **Exam results.** Each line of a file holds a student number of 6 digits, a name in 20 columns and three marks of 3 digits. Print a line per student with the name, the average with one decimal, and `passed` or `failed` (an average of at least 50 and no mark below 30 passes) chosen with a `b(...)` pattern, with one `getf` and one `printf` per student.

4. **Column totals.** Read a table of numbers separated by commas, with an unknown number of rows, and print it with a format, with a total for each column.

5. **Bad records.** Read records of a fixed layout with `getf`, and install an `on_value_error` routine that prints the number of each bad record and skips it.

6. **Bank statement.** Read transactions (date, text, amount) and print a statement with a running balance, the amounts in a column with a floating sign or with `b("CR", "DR")`, and a heading on every page.

7. **A calendar.** Print the calendar of a month, with the name of the month chosen by a `c` pattern and the first week indented with a computed repetition.

8. **Amounts in words.** Print an amount of money below one million in words, as on a check, using choice patterns for the units, tens and hundreds.

9. **Justification.** Read a text and print it with lines of a given width and straight margins on both sides, by distributing extra blanks between the words as evenly as possible. The last line of a paragraph keeps a ragged right margin.

10. **The last lines.** Print the last n lines of the standard input, which may be of any length, numbered with `whole`.

11. **Merging.** Write two sorted sequences of numbers to temporary files made with `create`, `reset` them, merge them onto the standard output, and `scratch` them.

12. **Number conversion.** Write a procedure that converts a **`real`** into a string with a given number of significant digits, using `associate` or `float`.

13. **Announcements.** Given the children of a family with their names, sexes and years of birth, print sentences such as “Ann is the eldest of three children” or “Tom is the only son”, choosing the words with `b(...)`, `c(...)` and `f(...)` patterns in a single `printf` per child (Algol 68 Genie).

A. The standard library

This appendix lists the parts of the standard library of the Revised Report that this book uses, by topic. The Report defines many of them for every size of a type; the tables show the plain size, and the long and short versions follow the rules of Section 6.2. Names written with underscores here, such as `max_int`, are written with blanks in the Report and in Algol 68 Genie (`max int`).

Types

Type	Values
int, **real**, **bool**, **char**	the primitive types (Chapter 2)
string	**flex** `[1:0]` **char** (Chapter 7)
compl	complex numbers, a record of two **real** fields (Chapter 6)
bits, **bytes**	bit sets and byte strings (Chapter 6)
sema	semaphores (Chapter 8)
file, **channel**, **format**	transput (Chapter 9)
void	no value

Operators

The binary operators of the library with their precedences. Every unary operator binds more tightly than every binary one.

Precedence	Operator	Operands
9	`I`	**int** or **real**; builds a **compl**
8	`**`	**int**, **real** or **compl** base, **int** exponent
8	`UP`, `DOWN`	**bits** and **int**: shift (also `SHL`, `SHR`)
8	`LWB`, `UPB`	**int** (the dimension) and an array
7	`*`	numbers; **int** and **string** (repetition)
7	`/`	numbers; the result is never **int**
7	`%`, `OVER`	**int**: integer division
7	`%*`, `MOD`	**int**: remainder
7	`ELEM`	**int** and **bits** or **bytes**
6	`+`, `-`	numbers; `+` also joins strings and characters
5	`<`, `<=`, `>=`, `>`	numbers, characters, strings, **bits** (`<=`, `>=`)
4	`=`, `/=`	all primitive types, **compl**, **bits**, strings
3	`AND`	**bool**, **bits**
2	`OR`	**bool**, **bits**

Precedence	Operator	Operands
1	`+:=`, `-:=` , `*:=`, `/:=`, `%:=`, `%*:=`	a variable and a value
1	`+=:`	a string or character and a string variable (prepend)

The word forms `PLUSAB`, `MINUSAB`, `TIMESAB`, `DIVAB`, `OVERAB`, `MODAB` and `PLUSTO` may be used instead of the compound assignment symbols, and `EQ`, `NE`, `LT`, `LE`, `GE`, `GT` instead of the comparisons.

The unary operators:

Operator	Meaning
`-`, `+`	negation, identity
`ABS`	absolute value; code of a **`char`**; **`int`** value of **`bits`**; 1 or 0 for a **`bool`**
`SIGN`, `ODD`	sign of a number; whether an **`int`** is odd
`ROUND`, `ENTIER`	nearest **`int`**, largest **`int`** not greater
`REPR`	the character with a given code
`NOT`	negation of **`bool`** or **`bits`**
`BIN`	**`bits`** from an **`int`**
`RE`, `IM`, `ARG`, `CONJ`	parts of a **`compl`**
`LENG`, `SHORTEN`	conversion to the next longer or shorter size
`LWB`, `UPB`	bounds of the first dimension
`LEVEL`, `UP`, `DOWN`	semaphores

Constants

Constant	Meaning
`pi`	π
`max_int`, `max_real`	the largest **`int`** and **`real`**
`small_real`	the smallest **`real`** that changes 1 when added
`bits_width`, `bytes_width`	the sizes of **`bits`** and **`bytes`**
`int_lengths`, `int_shorths`, `real_lengths`, `real_shorths`	the numbers of sizes
`int_width`, `real_width`, `exp_width`	the digits needed to print numbers
`max_abs_char`	the largest character code
`blank`, `null_character`, `flip`, `flop`	the characters `" "`, code 0, `"T"`, `"F"`

Procedures

Procedure	Meaning
`sqrt`, `exp`, `ln`	square root, exponential, natural logarithm
`sin`, `cos`, `tan`	trigonometric functions (radians)
`arcsin`, `arccos`, `arctan`	their inverses
`random`	a pseudo-random **`real`** in $[0, 1)$
`whole`, `fixed`, `float`	numbers to strings (Section 2.14)
`char_in_string (c, pos, s)`	whether `c` occurs in `s`; its index in `pos`
`bits_pack`, `bytes_pack`	**`bits`** from `[] bool`, **`bytes`** from a string

The transput procedures are listed in Chapter 9: `print`, `read`, `put`, `get`, `printf`, `putf`, `readf`, `getf`, `put_bin`, `get_bin`, `open`, `establish`, `close`, `reset`, `associate`, `make_term`, `on_logical_file_end`, `on_value_error`, the layout procedures `newline`, `space`, `backspace`, `newpage`, and the files `stand_in`, `stand_out` and channels `stand_in_channel`, `stand_out_channel`, `stand_back_channel`.

B. The syntax in brief

The Revised Report defines the syntax of Algol 68 with a two-level grammar, which also expresses the type rules. This appendix gives a simplified, context-free summary in the extended Backus–Naur form that most language references use today. It describes the shape of programs; whether a program is correct also depends on the types, which the grammar below does not check. The names of the nonterminals are the terms of this book, with the Report's term in a comment where it differs.

Notation: `{ x }` means zero or more repetitions of `x`, `[ x ]` an optional `x`, `x | y` a choice; terminal symbols are in quotes.

```
program        = [ access ] enclosed .
access         = "access" module_name { "," module_name } .

block          = "begin" sequence "end" | "(" sequence ")" .
                                        (* closed clause *)
sequence       = { declaration ";" } part { exit_part } .
                                        (* serial clause *)
part           = { label ":" } expression
                 { ";" ( declaration | { label ":" } expression ) } .
exit_part      = "exit" label ":" part .

declaration    = mode_decl | prio_decl | op_decl | constant_decl
               | variable_decl | proc_decl .
mode_decl      = "mode" Type_name "=" type { "," Type_name "=" type } .
prio_decl      = "prio" operator "=" digit { "," operator "=" digit } .
op_decl        = "op" operator "=" function_literal
                 { "," operator "=" function_literal } .
constant_decl  = formal_type name "=" expression
                 { "," name "=" expression } .  (* identity declaration *)
variable_decl  = [ "loc" | "heap" ] actual_type name [ ":=" expression ]
                 { "," name [ ":=" expression ] } .
proc_decl      = "proc" name "=" function_literal
                 { "," name "=" function_literal } .

type           = "int" | "real" | "bool" | "char" | "string" | "compl"
               | "bits" | "bytes" | "sema" | "format" | "void"
               | ( "long" | "short" ) { "long" | "short" } type
               | "ref" type
               | "[" [ bounds { "," bounds } ] "]" type
               | "flex" "[" [ bounds { "," bounds } ] "]" type
               | "struct" "(" type names { "," type names } ")"
               | "union" "(" type { "," type } ")"
               | "proc" [ "(" type { "," type } ")" ] type
               | Type_name .
bounds         = [ expression ":" ] expression .
                     (* formal types have no bounds, actual ones have *)
```

```
expression     = assignment | identity | function_literal | jump
               | "skip" | "nil" | formula
               | tertiary ( "andth" | "orel" ) tertiary .  (* unit *)
assignment     = tertiary ":=" expression .
identity       = tertiary ( "is" | "isnt" | ":=:" | ":/=:" ) tertiary .
function_literal = [ "(" type names { "," type names } ")" ] type ":"
                   expression .                 (* routine text *)
jump           = [ "goto" | "go" "to" ] label .
formula        = [ formula operator ] operand .
                    (* binary operators by precedence 1 to 9 *)
operand        = { unary_operator } secondary .
secondary      = primary | selection | generator .
selection      = field "of" secondary .
generator      = ( "loc" | "heap" ) actual_type .
primary        = name | literal | cast | call | slice | enclosed .
cast           = type enclosed .
call           = primary "(" expression { "," expression } ")" .
slice          = primary "[" index { "," index } "]" .
index          = expression | [ expression ] ":" [ expression ]
                 [ "@" expression ] | "@" expression .

enclosed       = block | collateral | parallel | if_expr | case_expr
               | conformity | loop .
collateral     = "(" expression "," expression { "," expression } ")"
               | "(" ")" .              (* also array and record literals *)
parallel       = "par" "(" expression "," expression { "," expression } ")" .
if_expr        = "if" sequence "then" sequence
                 { "elif" sequence "then" sequence }
                 [ "else" sequence ] "fi" .
case_expr      = "case" sequence "in" sequence { "," sequence }
                 [ "ouse" ... | "out" sequence ] "esac" .
conformity     = "case" sequence "in"
                 "(" type [ name ] ")" ":" sequence
                 { "," "(" type [ name ] ")" ":" sequence }
                 [ "out" sequence ] "esac" .
loop           = [ "for" name ] [ "from" expression ] [ "by" expression ]
                 [ "to" expression ] [ "while" sequence ]
                 "do" sequence "od" .
```

The brief forms `( c | a | b )` for **`if`** and `( i | a, b | c )` for **`case`** follow the same pattern, with `|:` for **`elif`** and **`ouse`**. Format texts (Chapter 9), bits and other literals, and comments are lexical matters not shown here. The GNU extensions **`andth`** and **`orel`** are not operators: they bind more weakly than every operator, so that `a < b` **`andth`** `c < d` needs no parentheses, but a chain such as `a` **`andth`** `b` **`andth`** `c` must be written with parentheses, `a` **`andth`** `(b` **`andth`** `c)`. Modules are described in Appendix E.

C. Implementation notes

This appendix collects the differences between the implementations that the book mentions: ga68, the Algol 68 front end of GCC; a68j, which compiles the same language to the Java virtual machine; and Algol 68 Genie (a68g), an interpreter that many Algol 68 programs on the web, among them most Rosetta Code programs, are written for. The information describes the versions used for this book, which the preface lists.

Notation

	ga68 and a68j	Algol 68 Genie
reserved words	lower case: **`begin int`**	upper case: `BEGIN INT`
declared type names	capitalized: `Node`	upper case: `NODE`
word operators	upper case: `ABS`	upper case: `ABS`
names	lower case, underscores: `max_int`	lower case, blanks allowed: `max int`
comments	*`{ ... }`* (nested)	`# ... #`, `CO ... CO`, `COMMENT ... COMMENT`
program	a block or other enclosed construct	the same, or a sequence without brackets

Converting a program from one notation to the other is mechanical: the reserved words and the type names change case, blanks in names become underscores, and comments get the other brackets. A name that is a reserved word in ga68 but not in Algol 68 Genie, such as **`in`**, **`out`**, **`empty`**, **`word`** or **`op`**, must be renamed.

Sizes

	ga68 and a68j	Algol 68 Genie
`int`	32 bits	64 bits
`long int`	64 bits	128 bits
`long long int`	64 bits	72 decimal digits
`real`	32 bits	64 bits
`long real`	64 bits	128 bits
`long long real`	80 bits (ga68), 64 bits (a68j)	63 decimal digits
`bits`	32	64

A program written for Algol 68 Genie that relies on 64-bit **`int`** values must use **`long int`** with ga68 and a68j. Since **`real`** has only about seven decimal digits in ga68 and a68j, numerical programs should use **`long real`** there.

Transput

ga68 and a68j have no transput of the Report in their standard library. The module `Genie_Transput`, which comes with a68j and can be compiled with ga68, provides unformatted transput: `print`, `read`, `put`, `get`, files on disk, files associated with strings, the layout procedures and the events for the end of the file and for value errors. It does not provide formats, binary transput, or the printing of records, **`compl`**, **`short`** and **`long long`** values. Algol 68 Genie provides the transput of the Report almost completely, and adds `eof` and `eoln`, which `Genie_Transput` also has; Section 9.12 lists what it lacks.

Other features

Feature	ga68 and a68j	Algol 68 Genie
`par`	ga68: one unit after the other; a68j: threads with the option `-fpar-threads`	threads
`sema`	not in ga68; a68j with `-fpar-threads`	yes
`compl` operators	a68j only	yes
`bytes`	no	yes
formats	no	yes
lifetime rule (references, procedures, **`loc`** cells)	not checked	checked at run time
bounds of an array field when a record is assigned	not checked	checked
initial value of a variable	zero, **`false`**, blank, empty	undefined (an error when used)
`andth`, **`orel`**	yes (GNU extension)	yes (also `ANDF`, `ORF`)
`XOR` for **`bits`**	yes	yes
`downto`, `until` in loops	no	yes
modules	yes (GNU extension, Appendix E)	no
calling C (**`nest C`**)	ga68 only (Appendix F)	no
partial parametrization	no	yes (Appendix G)

Printing numbers

`print` writes an **`int`** in a field of `int_width + 1` characters with its sign, so the output of ga68 and a68j is narrower than that of Algol 68 Genie. A **`real`** is written in floating-point notation with `real_width` digits. Programs whose output should not depend on the implementation use `whole` and `fixed`.

When ga68 and a68j convert a **`real`** to a string, the last of its seven digits is sometimes wrong (9.81 is printed as `9.810001`); the examples of this book avoid such values.

Running Rosetta Code programs

Most Algol 68 programs on Rosetta Code are written for Algol 68 Genie in upper-case notation. To run one with ga68 or a68j, convert the notation as above, add **access** Genie_Transput and an enclosing block if the program has none, and check whether it relies on 64-bit integers, on formats or on the other features in the table above.

D. Glossary

The terms of the Revised Report and the terms this book uses for them.

Report	This book	Remarks
a posteriori mode	type after an implicit conversion	
a priori mode	type before an implicit conversion	
actual declarer	type with bounds	`[1:10]` **`int`** in a variable declaration
actual parameter	argument	
applied occurrence	use (of a name)	
assignation	assignment	
balancing	finding the common type of branches	Section 3.5
book	file on disk, data	Section 9.1
bounds	bounds	
case clause	**`case`** expression on an integer	like `switch`
cast	type annotation	does not force a conversion
channel	kind of device	
closed clause	block	**`begin`**…**`end`** or `(…)`
coercion	implicit conversion	Section 7.14
collateral clause	unordered list	
collateral elaboration	unordered evaluation	
conditional clause	**`if`** expression	
conformity clause	type switch, pattern match on a union	Section 7.7
context, syntactic position	conversion context	strong, firm, meek, weak, soft
defining occurrence	declaration	
deflexing	removing **`flex`** from a type	the type of the value of a resizable array
denotation	literal	
deproceduring	implicit call of a parameterless function	
dereferencing	automatic reading of a variable	

Report	This book	Remarks
descriptor	array header (bounds and address)	
elaboration	evaluation, execution	
environment enquiry	implementation constant	`max_int`
equivalent modes	the same type	structural equivalence
event routine	callback, handler	`on_logical_file_end`
field selector	field name	
file	file handle, stream	
firmly related	convertible in a firm context	overloads must not be
flexible name	resizable array variable	**`flex`**
formal declarer	type without bounds	`[]` **`int`** in a parameter
formal parameter	parameter	
format text	format string	Chapter 9
formula	operator expression	
generator	allocation	**`loc`**, **`heap`**
hip	expression that fits any expected type	**`skip`**, a jump; **`nil`** fits any reference type
identification (of operators)	overload resolution	
identifier, tag	name, identifier	
identity declaration	constant declaration	**`int`** `n = 10`
identity relation	reference comparison	**`is`**, **`isnt`**
jump	**`goto`**	
loop clause	loop	
mode	type	
mode declaration	type declaration	structural, not nominal
mode indicant	type name	
monadic, dyadic operator	unary, binary operator	
multiple selection	field access on an array of records	Section 4.9
multiple value, row	array	
name	reference, memory location	the value of a variable
nil	null reference	
object, value	value	
operator declaration	operator definition, overloading	
parallel clause	concurrent block	**`par`**
plain mode	primitive type	
pragmat	compiler directive	
priority	precedence	
priority declaration	precedence declaration	**`prio`**
procedure	function	
range	scope of a block	

Report	This book	Remarks
reach	a block without the blocks nested in it	
routine text	function literal, lambda	
row display	array literal	
rowing	making a one-element array	
scope (of a value)	lifetime	Section 7.15
selection	field access	`x of p`
serial clause	sequence (block body)	
skip	undefined value of any type	
slice	indexing, slice	
straightening	flattening for transput	
stropping	marking of reserved words	Section 1.4
structure	record	
structure display	record literal	
transient name	reference into a resizable array	may not be stored
transput	input and output	
trimmer	index range of a slice	`a[2:4]`
unit	expression	
united mode	union, sum type	
uniting	conversion into a union	
voiding	discarding a value	
well-formed mode	admissible recursive type	
widening	widening	**int** to **real** to **compl**, **bits** to `[]` **bool**, **bytes** to `[]` **char**

E. Modules

The Revised Report has no modules: a program is a single text, and the only library is the standard one. Large programs need a way to divide the text into separately compiled parts with defined interfaces. The GNU dialect of Algol 68, which ga68 and a68j implement, adds modules for this purpose, following a proposal by Lindsey and Boom [7]; outside this appendix, the book uses them only for `Genie_Transput`. This appendix describes the essentials. Algol 68 Genie has no modules.

Defining a module

A module is a separate source file of the form

```
module Name =
def
   declarations and expressions
fed
```

The declarations between **def** and **fed** are private, unless they are marked with **pub**; the public ones form the interface of the module. Types, constants, variables, functions, operators and precedences can be public. The sequence is executed once, before the program that uses the module starts, and must end with an expression, as every sequence must (often **skip**).

The following module implements a stack of integers. The list behind it is private; only the three functions are public:

```
module Stack =
def
   { A stack of integers, as a linked list. }
   mode Cell = struct (int value, ref Cell below);
   ref Cell bottom = nil;
   ref Cell top := bottom;
   pub proc push = (int x) void: top := heap Cell := (x, top);
   pub proc pop = int: (int x = value of top; top := below of top; x);
   pub proc is_empty = bool: top is bottom;
   skip
fed
```

Using a module

A program uses modules by naming them after **access**, before its block:

```
access Stack, Genie_Transput
begin
```

```
    for i to 5 do push (i * i) od;
    while NOT is_empty do print (pop) od;
    print (newline)
end
```

```
        +25        +16         +9         +4         +1
```

The public names of all accessed modules are visible in the block, as if they had been declared in a block around it. Two modules that both declare the same public name cannot be accessed together, since the name would be declared twice; a module that is used by two other modules can be accessed by both, and is executed only once. A module can itself access other modules, by writing **access** before its **def**.

Compiling

A module is compiled on its own, and the program is compiled with the compiled modules. With ga68, a module is compiled to an object file, which also contains its interface, and the program is linked with it:

```
$ ga68 -c stack.a68
$ ga68 -I. use-stack.a68 stack.o genietransput.o -o use-stack
```

With a68j, a module is compiled to a library jar, and the directory of the jar is given to the program:

```
$ java -jar a68j-backend.jar stack.a68 -d lib/libstack.jar
$ java -jar a68j-backend.jar -I lib -run use-stack.a68
```

The modules that come with a68j, such as `Genie_Transput`, need no such step.

F. Calling C

A program sometimes needs a function that exists only in another language: a routine of the C library, of the operating system, or of a library written in C. ga68 can call such functions directly, and it can also use variables of a C program. This appendix describes how. The mechanism is a GNU extension; a68j and Algol 68 Genie cannot call C (a68j has a similar mechanism for Java methods, described in its own documentation).

Formal holes

The construct that names a C function or variable is a *formal hole*:

```
nest C "name"
```

It is written where a value is expected and stands for that value. Its type is not written in the hole; it is the type the context requires, so a formal hole may only be written in a strong context (Section 7.14), such as the value of a constant declaration, an argument or the right-hand side of an assignment:

```
proc (int) int c_abs = nest C "abs";
proc (long real) long real cbrt = nest C "cbrt";
proc (real) real cbrtf = nest C "cbrtf";
print ((c_abs (-7), newline));
print ((fixed (cbrt (long 2.0), 0, 12), newline));
print ((fixed (cbrtf (2.0), 0, 12), newline))
```

```
         +7
1.259921049895
1.259921073913
```

Each declaration says which type the C function has in Algol 68 terms. The compiler translates the type into the corresponding C type and generates a call of the C function with that name. The name `C` after **nest** is the language; the word `C` is not reserved and can still name a type, an operator or a module elsewhere in the program. The C library and the mathematical library are linked with every program by the ga68 command, so their functions need nothing else.

The notation goes back to the proposal for modules and separate compilation of 1978 [7] (the same proposal as the modules of Appendix E). There, a formal hole marks a place in one part of a program that another, separately compiled part fills in, so that a program can be developed top-down; the proposal already allowed the part to be written in another language, with an example in Fortran. ga68 provides only the second use.

Types

The types that can cross the border are the simple ones and those built from them:

Algol 68	C
void (as a result)	`void`
bool	`bool`
char	a 32-bit integer (the character code)
int, **short int**, **short short int**	`int`, `short`, `char`
long int, **long long int**	`long`, `long long`
bits, **short bits**, **short short bits**	`unsigned int`, `unsigned short`, `unsigned char`
long bits, **long long bits**	`unsigned long`, `unsigned long long`
real, **long real**	`float`, `double`
ref *T*	a pointer to *T*
proc with parameters and result of these types	a function, or a pointer to one
a record with fields of these types	the corresponding `struct`
string, only as a parameter	three parameters, see below

Other arrays and unions cannot be passed.

> **Pitfall.** Nothing checks that the declared type matches the C function. The compiler trusts the declaration, and the C library has no type information to compare it with. A wrong type gives wrong results, not an error message. The most likely mistake is the floating-point type: since **real** is a C `float`, a function of the C library that takes and returns `double`, such as `cbrt` or `sin`, must be declared with **long real**, as in the example above; declared with **real**, it receives and returns meaningless bit patterns. The `float` variants have an `f` at the end of their name (`cbrtf`).

Variables

A C variable can be used in two ways. Without further marking, the hole stands for the value of the variable at the moment the hole is evaluated, here when the declaration is executed; later changes by the C program are not seen:

```
int limit = nest C "limit";
```

With an ampersand before the name, it stands for the variable itself, and the Algol 68 declaration has a reference type: assignments to it change the C variable, and the C program sees them.

```
ref int calls = nest C "&calls";
```

Without the ampersand, a hole of a **ref** type stands for the value of a C pointer variable, so the Algol 68 name refers to the C object the pointer points to.

Strings and functions as parameters

A C function that receives an Algol 68 string gets three parameters for it: a pointer to the characters, the number of characters, and the distance in bytes from one character to the

next. The characters are 32-bit codes, not bytes, and they need not be next to each other, because the string may be a column of a two-dimensional array of characters, such as `m[, 2]`. A C function that counts the vowels of a string, a C function that calls a function it receives, and a counter variable:

```
/* C side of the example cfun.a68. */
#include <stddef.h>
#include <stdint.h>

int calls = 0;

/* The number of vowels in an Algol 68 string: the characters (UCS-4),
   their number and the distance in bytes between two of them. */
int vowels (const uint32_t *s, size_t len, size_t stride)
{
  int n = 0;
  calls++;
  for (size_t i = 0; i < len; i++)
    {
      uint32_t c = *(const uint32_t *) ((const char *) s + i * stride);
      if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
        n++;
    }
  return n;
}

/* Calls f for 1, ..., n and returns the sum of the results. */
int sum_of (int (*f) (int), int n)
{
  int s = 0;
  calls++;
  for (int i = 1; i <= n; i++)
    s += f (i);
  return s;
}
```

and the Algol 68 program that uses them:

```
ref int calls = nest C "&calls";
proc (string) int vowels = nest C "vowels";
proc (proc (int) int, int) int sum_of = nest C "sum_of";

string s = "parametrization";
print ((vowels (s), " ", vowels (s[1:5]), newline));
proc square = (int i) int: i * i;
print ((sum_of (square, 4), newline));
print (("calls: ", calls, newline))
```

```
         +7          +2
        +30
calls:          +3
```

The C file is compiled with a C compiler and linked with the program:

```
$ cc -c cfun.c
$ ga68 -I. cfun.a68 cfun.o genietransput.o -o cfun
```

An Algol 68 function that is passed to C, such as `square` above, arrives as an ordinary C function pointer. ga68 makes it point to a small piece of code that it places in the stack frame of the block in which the function is declared. A C library must therefore not keep the pointer and call it after that block has ended, even if the function uses no names of the block. The same piece of code is the reason why the linker warns that such a program "requires executable stack"; the warning is expected, but a system that forbids executable stacks will not run the program.

G. Partial parametrization

Section 7.16 showed that an Algol 68 function cannot be returned from the block whose names it uses: a function that composes two other functions, the everyday closure of current languages, is an error. Soon after the Revised Report, Lindsey proposed an extension that removes most of this limitation without changing the lifetime rule [6]. The subcommittee of IFIP Working Group 2.1 that looked after Algol 68 released it in 1976, so that implementations that experimented with such a facility would all provide it in the same form; it is not part of the Revised Report. Algol 68 Genie implements it; ga68 and a68j do not (Appendix C): a68j rejects such a call with a message, and ga68 currently stops with an internal compiler error. The programs of this appendix were run with Algol 68 Genie.

Leaving arguments out

With partial parametrization, a call may leave arguments out, by writing nothing in their places; the commas stay. Such a call does not run the function; it yields a new function that takes the missing arguments and has the given ones stored inside it:

```
proc add = (real x, y) real: x + y;
proc (real) real add3 = add (3, );
print ((fixed (add3 (4), 0, 1), " ", fixed (add (3, ) (10), 0, 1), newline))
```

```
7.0 13.0
```

`add (3, )` is a function of one **real**, which adds 3. It can be called at once, as in `add (3, ) (10)`, or kept and called later. The functional languages call the same idea partial application; in Python it is `functools.partial`, in C++ `std::bind`.

A function can be given its arguments in several stages. Each stage stores the new arguments and yields a function of the remaining ones:

```
proc volume = (real l, w, h) real: l * w * h;
proc (real, real) real base2 = volume (2, , );
proc (real) real base6 = base2 (3, );
print ((fixed (base6 (4), 0, 1), " ", fixed (base2 (1, 1), 0, 1), newline))
```

```
24.0 2.0
```

Closures without the lifetime problem

The stored arguments are part of the new function: they are copied into it, and the function does not refer to the block in which the call was made. This is what makes the idea work. Instead of using the names of an enclosing block, the inner function receives everything as

parameters, and the outer function supplies them by partial parametrization. The function `compose` of Section 7.16, which Algol 68 Genie stops at run time there, becomes:

```
mode Fn = proc (real) real;
proc compose = (Fn f, g) Fn: ((Fn f, g, real x) real: f (g (x))) (f, g, );
proc double = (real x) real: 2 * x;
proc inc = (real x) real: x + 1;
Fn h = compose (double, inc);
print ((fixed (h (1), 0, 1), newline))
```

```
4.0
```

The inner function uses only its own parameters `f`, `g` and `x`. `compose` gives it the first two and returns the result, which no longer depends on `compose`'s parameters. Lindsey's proposal shows the same pattern with an operator that raises the value of a function to a power, so that the cosine "squared" yields $\cos^2 \theta$. The operator `^` below instead applies a function `n` times:

```
mode Fn = proc (real) real;
proc iterate = (Fn f, int n, real x) real:
begin
   real y := x;
   to n do y := f (y) od;
   y
end;
prio ^ = 8;
op ^ = (Fn f, int n) Fn: iterate (f, n, );
proc half = (real x) real: x / 2;
print ((fixed ((half ^ 3) (1), 0, 3), " ", fixed ((cos ^ 20) (1), 0, 6), newline))
```

```
.125 .739184
```

Twenty applications of `cos` to 1 come close to the number x with $\cos x = x$, 0.739085…; more applications get closer.

When the arguments are evaluated

The arguments of a call that leaves arguments out are evaluated when the call is made, not when the resulting function is finally called. A value is stored as it is at that moment; a reference is stored as a reference, so the function sees later changes of the variable it refers to:

```
proc add = (int a, b) int: a + b;
proc incr = (ref int c, int d) int: c +:= d;
int n := 1;
proc (int) int add_n = add (n, );
ref int counter = heap int := 0;
proc (int) int bump = incr (counter, );
n := 10;
print ((add_n (5), newline));
```

```
bump (1); bump (2);
counter +:= 100;
print ((bump (3), newline))
```

```
                +6
               +106
```

`add_n` keeps the value 1 that `n` had when `add_n` was made; the later assignment to `n` does not change it. `bump` keeps the reference `counter`, and every call increases the same cell. A heap cell and a function that keeps a reference to it are the counter of Section 7.16 without the need to pass the counter along.

Pitfall. The lifetime rule still applies to what is stored. A function made by partial parametrization lives only as long as both the original function and every value stored in it, whichever ends first. If a stored reference refers to a **loc** cell of a block, the function must not outlive that block:

```
proc incr = (ref int c, int d) int: c +:= d;
proc make_bump = proc (int) int:
begin
   loc int count := 0;
   incr (count, )
end;
proc (int) int bump = make_bump;
print ((bump (1), newline))
```

```
3        PROC incr = (REF INT c, INT d) INT: c +:= d;
                                             1
a68g: runtime error: 1: REF INT value is exported out of its scope, in VOID
    closed-clause starting at "BEGIN" in line 1.
```

The cell `count` ends with the block of `make_bump`, and the function returned holds a reference to it. By the rules of the proposal, returning the function from the block is the error. Algol 68 Genie does not check it there; it stops the program only when the function is used, here in `incr`, whose result is the cell that no longer exists. The counter must be a **heap** cell, as in the previous example.

Calls with all arguments

In Lindsey's proposal, every call is a partial parametrization: a call that supplies all arguments yields a function without parameters, which is then called at once if the context requires a value, the usual case. If the context requires a function without parameters, the call is not executed but kept: `proc int tick = incr (counter, 1)` would declare a function that increases `counter` by 1 each time it is called.

Implementation note. Algol 68 Genie does not implement this part of the proposal. It runs a call as soon as all arguments are given, and rejects the declaration of `tick`:

```
5        PROC INT tick = incr (counter, 1);
                         1
```

```
a68g: error: 1: INT cannot be coerced to PROC INT in a strong-unit, in
    closed-clause starting at "BEGIN" in line 1.
```

Leaving out at least one argument, as `bump` does above, works in Algol 68 Genie.

Answers to the exercises

The numerical answers have been checked by running them. The checks are programs in the directory `examples/answers` of the book's sources, one for each chapter.

Chapter 2

1. `12` and `1 000` are **`int`** literals (blanks are allowed between digits); `.5`, `3e2` and `0.0` are **`real`** literals. `-3` is not a literal but an expression: the unary operator `-` applied to the literal `3`. `1,000` is two literals, `1` and `000`, separated by a comma. `2.` is not a literal, since a digit must follow the point.

2. `17 % 5` is 3, `-17 % 5` is -3 (the unary minus binds first, and `%` truncates toward zero), `17 MOD 5` is 2, `-17 MOD 5` is 3 (the remainder is never negative), and `17 / 5` is the **`real`** 3.4.

3. `2 + 3 * 4 ** 2` is $2+3\cdot 16 = 50$. `-3 ** 2` is $(-3)^2 = 9$, since unary operators bind more tightly than binary ones.

4. **`int`** `n = 10` binds `n` to the value 10; `n` has the type **`int`** and cannot be assigned to. **`int`** `n := 10` allocates a cell, stores 10 in it and binds `n` to the reference to the cell; `n` has the type **`ref int`**.

5. **`ref real`** `x =` **`loc real`** `:= 1.5, y =` **`loc real`**. Writing **`ref real`** before `y` as well is also correct.

6. (a) is correct: `i` is dereferenced and the **`int`** is widened to **`real`**. (b) is wrong: there is no implicit conversion from **`real`** to **`int`**. (c) is correct. (d) is wrong: `ROUND` is defined for a **`real`** operand only, and an operand is not widened.

7. Both are 3. The expression `a + b` is evaluated first, giving 3; this is assigned to `b`, and the value of `b` is then assigned to `a`.

8. Since a capital letter begins a type name, `maxCount` is read as the identifier `max` followed by the type name `Count`. Identifiers are written in lower case: `max_count`.

9. 2. `r` is a second name for the cell of `x`.

10. `((a < b) AND ((x > 0) = (y > 0))) OR (NOT done)`: the comparisons `<` and `>` (precedence 5) bind more tightly than `=` (4), which binds more tightly than `AND` (3) and `OR` (2), and the unary `NOT` binds most tightly of all.

11. 0.25 and 2: `-7 MOD 3` is 2, since `MOD` never yields a negative result. `NOT n = 0` means `(NOT n) = 0`, and `NOT` is not defined for an **`int`**.

Chapter 3

1. 10 and 11.

2. It prints 20 and then 1. The inner block declares a new `x` with the value $y \cdot 10 = 20$; after that block, `x` is the outermost `x` again.

3. (a) never, since the initial value is already greater than the final value; (b) five times (1, 3, 5, 7, 9); (c) five times (10, 8, 6, 4, 2); (d) never.

4. `if x > 0 then "plus" elif x < 0 then "minus" else "zero" fi`.

5. **`real`**: the two branches are balanced, and the common type of **`int`** and **`real`** is **`real`**; the 1 is widened.

6. The operands of an operator are evaluated in an unspecified order (collateral elaboration), so either call may come first.

7. `int p := 1; while p <= 1000 do print (p); p *:= 2 od`.

8. It prints 6, 12 and 24. The number of repetitions is computed once, before the first iteration, so the loop runs three times, and the changes of `k` do not affect it.

9. It prints 1 forever. With a step of 0 the final value is ignored.

10. Yes. A jump is acceptable as a value of any type, here **`string`**. For `k = 5` the **`out`** part is chosen, control goes to `bad`, and `name` never gets a value.

11. In `real v = (b  x | n)|` the context gives the type **`real`** to both branches: `x` is dereferenced, and `n` is dereferenced and widened. A **`ref int`** cannot become a **`ref real`**, so in the second declaration the `n` branch cannot be converted, and all three compilers report that **`ref int`** cannot be coerced to **`ref real`**.

12. The two declarations are carried out in an unspecified order, so `b` may be computed before `a` has a value. Written with a semicolon, `int a = 1; int b = a + 1`, the order is fixed.

13. No number is both less than 1 and greater than 9, so the condition is always false, and the loop ends after the first `read`. The condition must be `n < 1 OR n > 9`.

Chapter 4

1. ```
   [1900:2000] real rainfall;
   for y from LWB rainfall to UPB rainfall do rainfall[y] := 0 od
   ```

2. `a[2:3]` is the array `(6, 7)` with the bounds 1 to 2, so `LWB a[2:3]` is 1. `a[3:]` has the elements 7, 8, 9 and the bounds 1 to 3, so its `UPB` is 3. `a[2:3 @ 0]` has the bounds 0 to 1, and its element 0 is 6.
   ```

3. All three are one-dimensional arrays of **int** (references to them, since `m` is a variable). `m[2, ]` is the second row, with the bounds 1 to 4; `m[, 3]` is the third column, with the bounds 1 to 3; `m[2:3, 2]` is the part of the second column in rows 2 and 3, renumbered to the bounds 1 to 2.

4. `(4, 5, 6)`: the assignment copies the elements of `b` into `a`, and the later change of `b` does not affect `a`.

5. ```
   mode Date = struct (int day, month, year);
   Date d = (3, 11, 1975)
   ```

6. Indexing binds more tightly than selection, so the expression means `name of (class[1][1])`, which tries to index a record. It must be written `(name of class[1])[1]`.

7. `y of ps := (0, 0, 0, 0, 0, 0, 0, 0, 0, 0)`. Note that `y of ps := 0` does not work: the 0 is converted into an array with one element, and the bounds do not match (a run-time error).

8. 2 to 11 (the second dimension is left out and keeps its bounds), 1 to 3 (a range is renumbered), and 0 to 9 (the first dimension keeps its bounds).

9. `1 1 2 3 4 6`. The right-hand side is a value that is complete before anything is stored, so the overlap does no harm.

Chapter 5

1. (a) `proc (real, int) real`; (b) `proc bool`; (c) `proc (ref [] int) void`.

2. With a loop:

   ```
   proc digit_sum = (int n) int:
   begin
      int sum := 0, rest := n;
      while rest > 0 do sum +:= rest MOD 10; rest := rest % 10 od;
      sum
   end
   ```

   Recursively: `proc digit_sum = (int n) int: (n < 10 | n | n MOD 10 + digit_sum (n % 10))`.

3. The parameter `n` is a constant of type **int**, which cannot be assigned to. Written as `proc inc = (ref int n) void: n +:= 1`, it takes a reference to the caller's variable, as in `inc (count)`.

4. `proc apply_twice = (proc (int) int f, int x) int: f (f (x));` the call `apply_twice ((int n) int: 2 * n, 5)` gives 20.

5. `prio ABOVE = 5; op ABOVE = (int a, b) bool: a > b`. Since `+` has the higher precedence 6, `1 + 2 ABOVE 2` means `(1 + 2) ABOVE 2`, which is **true**.
   ```

6. A unary operator binds more tightly than a binary one, so the expression means `(SQUARE 2) MAX 3`, which is 4.

7. An assignment is one of the expressions that are discarded as they are: its value, the variable `p`, is thrown away without calling the function that `p` holds. Every other expression that yields a function without parameters, such as `p` on its own, an element `ps[1]` or a call `g (1)`, is called when it is discarded.

8. 9: the precedence belongs to the symbol `+`, so in that block `1 + 2 * 3` is `(1 + 2) * 3` for integers too.

9. Yes. Two declarations of a binary operator clash only if both pairs of operand types are related; here the first operands (**`int`** and **`real`**) are not, since an **`int`** operand is not widened.

10. ```
    op * = ([] real a, real k) [] real:
    begin
       [LWB a : UPB a] real r;
       for i from LWB a to UPB a do r[i] := a[i] * k od;
       r
    end
    ```

11. The items of a `print` list are evaluated in an unspecified order (Section 3.4), so `d` may be read before or after `month` has changed it. The call must come first, in a declaration of its own: **`string`** `m = month (d); print ((m, " ", d))`.

12. ```
    proc sum = (int lo, hi, proc (int) real term) real:
    begin
       real s := 0;
       for k from lo to hi do s +:= term (k) od;
       s
    end
    ```

    `sum (1, 100, (int k) real: 1 / (k * k))` gives about 1.635, and `sum (1, 3, (int k) real: v[k] * w[k])` the scalar product of three-element arrays. `v[k] * w[k]` on its own is a **`real`**, not a function of `k`: the call is rejected, and there is no `k` at that place anyway. The function literal is called by `sum` for each `k`, and its body is evaluated with that `k`.

Chapter 6

1. $(1+i)(1-i) = 2$, so the real part is 2.0; `ABS (0 I 2)` is 2.0.

2. There is no implicit conversion from **`int`** to **`long int`**. Correct is **`long int`** `n := ` **`long`** `5` or **`long int`** `n := LENG 5`.

3. 10, 16 and **`true`**: bit 32 is the least significant bit.

4.
```
proc popcount = (bits b) int:
begin
   int count := 0;
   for i to bits_width do if i ELEM b then count +:= 1 fi od;
   count
end
```

5. With the environment enquiry `int_lengths`: **long long int** is a distinct, longer type only if `int_lengths` is at least 3.

6. No. The comparisons of **bits** are set comparisons: `>=` asks whether the first value contains all bits of the second, and `2r100` does not contain the bits of `2r011`.

7. `2 * (3 I 4)`, the complex number $6 + 8i$, since `I` has the precedence 9.

Chapter 7

1. 1 to 5: `s[2:3]` is `"bc"`, and the assignment gives `s` the bounds of `"bcxyz"`.

2. A parameter of type **ref [] int** would also accept a variable with fixed bounds, to which a longer array cannot be assigned. Only **ref flex [] int** guarantees a variable whose bounds may change. Moreover, in the Report a **ref flex [] int** is a different type, which cannot be passed as a **ref [] int** at all.

3.
```
proc to_real = (union (int, real, bool) v) real:
     case v in (int i): i, (real r): r, (bool b): (b | 1 | 0) esac
```

4. (b) and (c). (a) is an error: the left-hand side `p` has the type **ref ref int**, so the right-hand side must be a **ref int**, and the compiler reports that an **int** cannot be converted to **ref int**.

5. One side of an identity relation is soft, the other strong. **nil** can only be the strong side, so `q` is the soft side and is not dereferenced: the variable `q` itself, which is never **nil**, is compared. The correct test is **ref int (q) is nil**.

6. (a) soft, (b) strong, (c) firm, (d) meek.

7.
```
proc reverse = (ref ref Node list) void:
begin
   ref Node prev := no_node, cur := list;
   while ref Node (cur) isnt no_node do
      ref Node rest = next of cur;
      next of cur := prev;
      prev := cur;
      cur := rest
   od;
   list := prev
end
```

8. `words[1]` is a reference into a resizable array, a transient reference, and `+:=` is an operator whose left operand would receive it. `words[1] := words[1] + "s"` is allowed.

9. (a) is rejected: `"a"` is a **`char`**, which is not a member of the union; **`string`** `("a")` works. (b) is accepted. (c) is rejected: `2` is an **`int`**, and uniting cannot follow widening. (d) is accepted.

10. `F`, `F`, `T`: `p` **`is`** `q` compares the two pointer variables, **`ref int`** `(p)` **`is ref int`** `(q)` compares `a` with `b`, and **`ref int`** `(p)` **`is`** `a` compares `a` with `a`. After `q := a` the results are `F`, `T`, `T`: the variables `p` and `q` are still different cells.

11. With `push`, the array is replaced when it holds 4, 8, 16, …, 512 elements, which copies $4 + 8 + \cdots + 512 = 1020$ elements, about one per element. With `append`, the k-th call copies the $k - 1$ elements before it, $0 + 1 + \cdots + 999 = 499\,500$ in all.

12. The function `(`**`int`** `x) `**`int`**`: x + n` uses `n`, a parameter of `adder`, so its lifetime ends when `adder` returns, and it may not be returned. Algol 68 Genie stops with "value is exported out of its scope"; ga68 and a68j run the program, and the function returned by `adder (3)` even yields 7 for the argument 4.

13. The first loop body declares nothing, since `print (i)` is not a declaration: the program remains correct. The second declares `j`, so each cell lives only during one execution of the body, and the program is wrong; Algol 68 Genie stops it.

14. `a` **`is`** `b` is **`false`**: the two lists consist of different cells, although their contents are equal.

```
prio EQ = 4;
op EQ = (ref Node a, b) bool:
   if a is nil then b is nil
   elif b is nil then false
   else value of a = value of b andth (next of a EQ next of b)
   fi
```

`a EQ b` is **`true`**.

Chapter 8

1. Both leave the order of `a` and `b` unspecified, and in both they may be interleaved. With **`par`**, each is a separate process, so one of them can wait on a semaphore while the other goes on.

2. The part waits forever; if no other part can continue either, the program hangs (a deadlock). Formally, the Report leaves the rest of the program undefined once no process can continue.

3. With one producer and one consumer, `mutex` is not strictly necessary: `tail` is changed only by the producer and `head` only by the consumer, and `free` and `full` ensure that they never use the same element of the buffer at the same time. With several producers or several consumers, two of them could change the same position at once, and `mutex` becomes necessary.

4. Let philosopher 0 pick up the right fork first and then the left one, while all others pick up the left fork first. Then the five can never each hold exactly one fork while waiting for the next.

5. When the buffer is full, the producer takes `mutex` and then waits in `DOWN free` while holding it. The consumer waits in `DOWN mutex` and can never make a place free: a deadlock.

6. **`sema`** is a record whose only field is a reference to the counter, so `b` is a copy of that reference and refers to the same counter as `a`.

Chapter 9

1. `open` connects the file with an existing book; `establish` creates a new one.

2. That the routine has dealt with the event, so that the transput continues at the same place. If it returns **`false`**, the default action follows; at the end of the file, the Report leaves the program undefined, and the implementations stop it with an error.

3. ```
   proc int_to_string = (int n) string:
   begin
      string s := "";
      File f;
      associate (f, s);
      put (f, n);
      s
   end
   ```

   The result has the standard layout of `print`, with the sign and leading blanks.

4. `$4d$` prints `0007` for 7; for −7 it is an error, since the picture has no sign frame. `$4zd$` prints four blanks and 7 for 7, and is also an error for −7. `$-3zd$` prints four blanks and 7 for 7, and a minus sign, three blanks and 7 for −7: the sign frame - prints a blank for a positive value.

5. Reading a string stops before the line break and does not skip it, so the line break must be skipped with `newline`. If both are read in one call, `get (input, (line, newline))`, and the last line has no line break, `newline` meets the end of the input and the event routine jumps away before the line is printed. With two calls, the line is printed first.

6. (a) One value: the frames form a single picture of four digits, and 3 is printed as `00/03`; the format then starts again and prints 4 as `00/04`. (b) Two values, printed as `03/04`.

7. `" -5"`, `"  5"`, `"2.50"` and `"-.25"`. A positive width always writes a sign, a negative one only a minus sign; with width 0, `fixed` writes no zero before the point.

8. Both `a` and `b` fail on the same `x`, since the routine leaves it in place; in the implementations, the routine is called twice, and `a` and `b` keep their old values. The `5` is never read.
   ```

9. `$g(0), " ", c("January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"), " ", g(0)$`

The pictures `" "` have no pattern and use no value.

Bibliography

[1] A. van Wijngaarden, B. J. Mailloux, J. E. L. Peck, C. H. A. Koster, M. Sintzoff, C. H. Lindsey, L. G. L. T. Meertens and R. G. Fisker (eds.), *Revised Report on the Algorithmic Language ALGOL 68*. Acta Informatica 5 (1975), 1–236. Also Springer, Berlin, 1976, and ACM SIGPLAN Notices 12(5) (1977), 1–70.

[2] A. van Wijngaarden (ed.), B. J. Mailloux, J. E. L. Peck and C. H. A. Koster, *Report on the Algorithmic Language ALGOL 68*. Numerische Mathematik 14 (1969), 79–218.

[3] C. H. Lindsey and S. G. van der Meulen, *Informal Introduction to ALGOL 68*, revised edition. North-Holland, Amsterdam, 1977.

[4] A. D. McGettrick, *ALGOL 68: A First and Second Course*. Cambridge University Press, Cambridge, 1978.

[5] A. S. Tanenbaum, A tutorial on ALGOL 68. ACM Computing Surveys 8(2) (1976), 155–190.

[6] C. H. Lindsey, Specification of partial parametrization proposal. ALGOL Bulletin 39 (February 1976), AB39.3.1, 6–9.

[7] C. H. Lindsey and H. J. Boom, A modules and separate compilation facility for ALGOL 68. ALGOL Bulletin 43 (December 1978). Also report IW 105/78, Mathematisch Centrum, Amsterdam, 1978.

[8] C. H. Lindsey, A history of ALGOL 68. ACM SIGPLAN Notices 28(3) (1993), 97–132 (preprints of the conference History of Programming Languages II).

[9] M. van der Veer (ed.), *Learning Algol 68 Genie*. The manual of Algol 68 Genie, 2008–2023.

[10] J. E. Marchesi, *The GNU Algol 68 Compiler*. The manual of ga68, part of the GNU Compiler Collection, 2025–2026.

Index

actual declarer (Report), 32
actual parameters (Report), 44
allocation, 8
array, 31
array literal, 32
assignment, 8

balancing (Report), 22
block, 1
book (Report), 93

cast, 73
channel (Report), 93
closed clause (Report), 19
closure, 48
coercion (Report), 12
collateral clause (Report), 21
collateral elaboration (Report), 21
comment, 3, 15
communication, 87
compound assignment, 13
concurrent block, 87
conformity clause (Report), 68
constant, 7
contexts (Report), 13
conversion contexts, 13
critical section, 87

deadlock, 91
deflexes (Report), 63
denotation (Report), 6
deproceduring (Report), 45, 74
dereferencing, 12
dereferencing (Report), 12, 74

environment enquiries (Report), 59
equivalent modes (Report), 82
event routines (Report), 94

field selector (Report), 36
fields, 36
file (Report), 93
firm, 13
firmly related (Report), 49
flexible name (Report), 63
formal declarer (Report), 32
formal hole, 125
formal parameters (Report), 44
formats, 100
frames, 100
function literal, 43

generator (Report), 8

higher-order functions, 47

identification (Report), 50
identifier (Report), 6
identifiers, 6
identity declaration (Report), 7
identity relations (Report), 72
implicit conversion, 12
index, 31
insertion sort, 38
insertions, 101

label, 26
lambda, 47
lifetime rule, 76
literal, 6
lock, 88
loop variable, 23

mode (Report), 5
mode declaration (Report), 36
multiple selection (Report), 37
multiple value (Report), 31

mutual exclusion, 87

name (Report), 7, 8
nominal typing, 37
null reference, 72

orthogonality, 1, 63
overload resolution, 50
overloading, 49

parallel clause (Report), 87
pictures, 100
precedence, 10
primitive types, 5
priority (Report), 10
priority declaration (Report), 49
procedure (Report), 43
process, 88

race condition, 89
range (Report), 20
reach (Report), 20
record, 31, 36
reference, 8
reserved words, 2
routine text (Report), 43
row (Report), 31, 38
row display (Report), 32
rowing, 38
rowing (Report), 38, 74

scope, 20
scope (Report), 20, 76
selection (Report), 36
semaphores, 87
serial clause (Report), 19
slice, 34
slice (Report), 31
sort (Report), 75
straightening (Report), 99
strong, 13
strong (Report), 12
stropping, 2
stropping (Report), 2
structural equivalence, 37
structure (Report), 36
structure display (Report), 36

sum type, 69
syntactic position (Report), 75
syntactic positions (Report), 13

tag (Report), 6
tagged union, 69
this (Report), vii
transient name (Report), 67
transput (Report), 93
trimmer (Report), 34
type, 5
type declaration, 36

union, 68
unit (Report), 12
uniting, 68
uniting (Report), 68, 74
units (Report), 19

variable, 7
voiding (Report), 20, 74

well-formed modes (Report), 79
widening, 12
widening (Report), 12, 74